

"TITU MAIORESCU" UNIVERSITY OF BUCHAREST

FACULTY OF INFORMATICS

BACHELOR THESIS

Cryptographic Schemes Applicable to Post-Quantum Cryptography

SCIENTIFIC COORDINATOR:

Associate Professor Dr. Marius Rogobete

GRADUATE:

Brandon Aron

JUNE SESSION

2025

Contents

1. Introduction.....	5
1.1 Context	5
1.2 Motivation	6
1.3 Structure of the Thesis.....	7
2. Concept and State-of-the-Art	9
2.1 Definitions	9
2.2 Fundamental Quantum Algorithms that Affect Cryptography.....	10
2.2.1 Shor's Algorithm	10
2.3.1 Lattice-Based Schemes	16
2.3.2 Code-Based Schemes.....	17
2.3.3 Hash-Based Schemes.....	18
2.3.4 Isogeny-Based Schemes	18
2.3.5 Multivariate Schemes.....	19
2.3.6 Symmetric-Key Schemes.....	20
3.1 Benchmarking Methodology and Relevant Metrics	21
3.2 Implementation of Cryptographic Algorithms	22
3.3 Implementation of Quantum Algorithms	24
3.3.3 Resource Estimation for Cryptographically Relevant Algorithms	25
3.3.4 Quantum Error Correction and Fault Tolerance	26
3.4 Performance Metrics.....	26
3.4.1 Noise and Hardware Error Characterisation	27
3.4.2 Circuit Complexity and Optimisation Metrics	28
3.4.3 Execution Performance and Resource Consumption	28

3.4.4 Algorithmic Success and Output Quality Assessment.....	29
3.4.5 Comparative Analysis and Contextualisation of Results.....	30
4. Architecture and Design of the Application: “Keystone”	31
4.1 Application Requirements and Specifications.....	31
4.1.4 Target User Profiles	34
4.1.5 Integration Requirements with External Systems.....	34
4.2 Use Cases and Specification Allocation	35
4.2.3 Use Case 3: Demonstrating a Hybrid Cryptographic System.....	37
4.2.4 Use Case 4: Real-Time Analysis and Research-Oriented Data Export	38
4.2.5 Use Case 5: Comparative Analysis in an Educational Context	39
4.2.6 Architectural Mapping and Design Validation	40
4.3 Architectural Diagrams	41
4.4 Hexagonal Architecture	49
4.4.1 Architectural Rationale and Design Motivation	49
4.4.2 Architectural Layer Specification	50
4.5 User Interface and UX Design.....	54
4.5.1 Design Philosophy and UX Principles.....	54
4.5.2 Benchmark Execution and Visualisation Interface	54
4.5.3 Quantum Algorithm Execution Interface	55
4.5.4 Hybrid Encryption Module Interface	56
4.5.5 Accessibility Features and User Adaptation	57
4.5.6 Performance Optimisation and Responsiveness	57
4.6 Backend Integration and Management.....	58
4.6.1 Electron + Node.js Orchestration Platform	59
4.6.2 Native Execution Infrastructure (Cryptography)	59

4.6.3 Benchmark Task Management	60
4.6.4 Quantum Integration and External APIs	61
4.6.5 Memory Management and Performance Optimisation	61
4.6.6 Error Handling and System Reliability	62
4.6.7 Extensibility and Future Integration Support	62
4.7 Application Testing and Validation Strategies	63
5. Evaluation and Experimental Results	69
5.1.1 Performance of Key Encapsulation Mechanisms (KEM)	69
5.1.2 Performance of Digital Signature Algorithms (DSA)	75
5.1.3 Classical Algorithms	80
5.2 Security Considerations	84
5.3 Performance Trade-offs	85
5.4 Quantum Algorithm Execution	86
5.4.1 Quantum Algorithm Results	86
5.4.2 Backend Comparison	102
5.5 Methodology and Result Validation	109
6. Conclusions	111
6.1 Key Results	111
6.2 Original Contributions of the Thesis	111
6.3 Future Development Directions	112
6.4 The Importance of Post-Quantum Cryptography in the Contemporary Context..	113
6.5 Final Observations	114
7. References	114

1. Introduction

1.1 Context

For over half a century, quantum computing has evolved from a theoretical curiosity into an operational reality within laboratories. Its foundations were laid by figures such as Feynman, who anticipated the limitations of classical machines in simulating quantum systems [3], and Deutsch, who formalised the concept of a universal quantum computer [4]. A decisive moment came in 1994, when Shor introduced his polynomial-time algorithm for factoring integers and computing discrete logarithms [1]. Two years later, Grover demonstrated a quadratic speedup for searching unstructured databases [2]. These developments exposed vulnerabilities at the core of public-key cryptographic systems and initiated a global effort to develop scalable quantum hardware.

Progress in quantum computing was long constrained by qubit decoherence, the tendency of quantum bits to lose their state when interacting with the environment. This obstacle began to shift with the invention of quantum error-correcting codes and the proof of the threshold theorem, which showed that scalable computation becomes viable once physical error rates are reduced below a critical value, typically around one percent. Subsequent demonstrations involved trapped ion qubits and superconducting circuits. To describe this transitional phase, in which quantum devices are functional yet still unstable, Preskill introduced the term Noisy Intermediate-Scale Quantum, or NISQ [5].

While quantum processing continues to mature, other branches of the quantum ecosystem have already produced tangible results. One example is the development of an algorithm that efficiently simulates the dynamics of coupled oscillators using a Hamiltonian approach, significantly reducing the cost of solving the Schrödinger equation in realistic scenarios. This technique paves the way for direct applications of quantum computation in areas such as material design, molecular dynamics, and the engineering of complex systems [6]. Meanwhile, the United Kingdom has achieved a secure data transmission spanning over 410 kilometres via a quantum network operating over classical

fibre infrastructure. This system combined photonic key distribution with entanglement to enable real-world communication, including trials in medical and industrial contexts [7].

Taken together, these advances suggest a shift in trajectory. Quantum technologies are moving beyond theoretical demonstration and entering the domain of practical application, where they may soon influence data simulation and security in distributed computing environments.

1.2 Motivation

Shor's algorithm undermines the foundations of modern cryptography. Once a sufficiently powerful quantum computer becomes operational—a system referred to as a Cryptographically Relevant Quantum Computer, or CRQC—schemes such as RSA, Diffie-Hellman, and elliptic curve cryptography lose their security assumptions. Grover's algorithm introduces an additional vulnerability, halving the effective brute-force resistance of symmetric schemes in the presence of a quantum adversary.

These schemes are not theoretical abstractions. They underpin critical protocols across the global digital infrastructure, from TLS to code signing mechanisms. For this reason, cybersecurity agencies have issued clear guidance urging the transition toward cryptographic algorithms resistant to quantum attacks [9]-[11].

Past cryptographic migrations have required more than a decade to complete. That timeline now conflicts with an emerging risk: the possibility that sensitive data could be intercepted today and decrypted later, once quantum hardware becomes capable. This scenario is known as harvest-now, decrypt-later. To mitigate this, NIST has recommended beginning the transition to post-quantum algorithms before such hardware becomes technically feasible [9].

In 2016, NIST launched an international competition to standardise post-quantum cryptographic schemes. Of the 69 submissions in the initial round, four finalists were announced in 2022: Kyber, Dilithium, Falcon, and SPHINCS+ [12]. In 2024, preliminary

drafts of the FIPS 203 through 205 [13]-[15] standards were released. As of March 2025, HQC has been added as an additional key exchange mechanism.

Agencies now recommend an agile approach. Organisations are encouraged to adopt interchangeable cryptographic primitives and to deploy hybrid schemes that combine classical and post-quantum algorithms during the transition period. They are advised to map out their cryptographic dependencies, allocate budgets for system upgrades, and adopt architectures that allow compromised algorithms to be replaced without rewriting entire systems.

This thesis presents Keystone, a practical application that integrates the execution of Shor and Grover algorithms on IBM Quantum hardware via cloud access. The system performs comparative benchmarking across nine cryptographic schemes, including both classical and post-quantum variants. It features a hybrid encryption system that combines ML-KEM, AES-256-GCM, and ML-DSA. In addition, the application provides performance visualisation tools, asynchronous execution, dataset management utilities, and an extensible framework intended to support future experiments.

1.3 Structure of the Thesis

This thesis is organised into seven chapters, following a logical progression from theoretical foundations to practical implementation, culminating in experimental validation and applicable conclusions.

Chapter 2 presents the conceptual background and current state of the field. It introduces the key definitions of post-quantum cryptography and quantum computing, followed by an analysis of the quantum algorithms that threaten classical cryptography, specifically Shor's algorithm for factoring and discrete logarithms, and Grover's algorithm for unstructured search. The chapter continues with an overview of the main families of post-quantum schemes, including lattice-based schemes such as Kyber, Dilithium, and Falcon, code-based schemes like Classic McEliece, hash-based schemes such as SPHINCS+, and emerging designs based on isogenies and multivariate polynomials.

Chapter 3 details the methods used for cryptographic evaluation. It describes benchmarking techniques for execution time, memory consumption, and throughput, as well as integration strategies for classical and post-quantum algorithms, with an emphasis on the use of the liboqs and OpenSSL libraries. The chapter also addresses the analysis of quantum circuits, integration with IBM Quantum backends, and the challenges associated with error correction in realistic executions.

Chapter 4 documents the architecture and design of the Keystone application, which represents the practical contribution of the thesis. It outlines the initial requirements, use cases, architectural decisions, and design diagrams, including the adoption of hexagonal architecture. Both the backend — featuring native cryptographic modules and asynchronous routines for quantum and performance benchmarking — and the user interface — designed to make complex cryptographic concepts accessible — are examined in detail. The chapter also includes testing strategies and security considerations relevant to evaluation platforms.

Chapter 5 presents the experimental validation of the proposed solutions. It reports comparative benchmarks between classical and post-quantum algorithms across different security levels and operational contexts. It also analyses the results of the quantum algorithms, including a comparison between two hardware backends. The chapter formalises the experimental methodology, validation procedures, and statistical tools used.

Chapter 6 synthesises the results and contributions of the thesis. It presents the main achievements, including the Keystone platform and the evaluation methods, along with insights drawn from the comparative analysis. The chapter identifies the study's limitations and proposes directions for future research, highlighting the urgency of adopting post-quantum cryptography in light of current threat landscapes.

Chapter 7 contains the complete bibliography, covering both the academic literature and relevant technical sources.

This structure guides the reader from foundational concepts to concrete applications, offering a unified perspective on the cryptographic transition in the quantum era.

2. Concept and State-of-the-Art

2.1 Definitions

Post-Quantum Cryptography (PQC)

Post-quantum cryptography refers to public-key encryption schemes and digital signature algorithms that remain secure even in the presence of an adversary equipped with a large-scale quantum computer [9]. These schemes rely on hardness assumptions for which no efficient attacks are currently known, whether classical or quantum. Such assumptions include problems based on lattices, error-correcting codes, multivariate equations, and cryptographic hash functions [7].

Quantum Computing

A quantum computer operates on qubits, units of quantum information that can exist in superposition between the classical states 0 and 1. The phenomena of entanglement and interference allow quantum algorithms to achieve drastic reductions in computational complexity for certain problem classes. The complexity class BQP groups the problems that can be solved efficiently using quantum machines [34].

Comparison of Cryptographic Assumptions

Classical cryptographic schemes such as RSA, Diffie-Hellman, and elliptic curve cryptography (ECC) assume that factoring large integers and computing discrete logarithms are computationally intractable within reasonable time. Shor's algorithm invalidated this assumption in the quantum model, placing both problems within the BQP class and thereby compromising their associated security guarantees [1].

Post-quantum cryptography departs from these assumptions and instead relies on problems for which no sub-exponential quantum algorithms are currently known. The only broadly applicable quantum advantage against symmetric schemes is offered by Grover's algorithm, which reduces the search space quadratically. This effect can be countered by doubling the key size in symmetric algorithms [2], [23].

2.2 Fundamental Quantum Algorithms that Affect Cryptography

2.2.1 Shor's Algorithm

Shor's algorithm solves the problems of integer factorisation and discrete logarithms in polynomial time on a quantum computer [1]. For a composite integer N and a base a such that $\gcd(a, N) = 1$ the objective is to find the period r of the function: $f(x) = a^x \bmod N$

Setup:

1. Let $n = \lceil \log_2 N \rceil$
2. Allocate two quantum registers:
 - A control register with $t = 2n$ qubits, initialised to $|0\rangle^{\otimes t}$
 - A work register with n qubits, initialised to $|1\rangle$

Quantum Steps:

1. Hadamard Superposition

Apply a Hadamard gate to each qubit in the control register to obtain:

$$|0\rangle^{\otimes t} \xrightarrow{H^{\otimes t}} \frac{1}{2^{t/2}} \sum_{x=0}^{2^t-1} |x\rangle$$

2. Controlled Modular Exponentiation

For each $k \in \{0, \dots, t-1\}$, apply the controlled unitary gate $C = U^{2^k}$, where:

$$U|y\rangle = |a \cdot y \bmod N\rangle$$

The resulting entangled state becomes:

$$\frac{1}{2^{t/2}} \sum_{x=0}^{2^t-1} |x\rangle |a^x \bmod N\rangle$$

3. Inverse Quantum Fourier Transform (QFT):

Apply the inverse QFT on the control register:

$$|x\rangle \xrightarrow{QFT^{-1}} \frac{1}{2^{t/2}} \sum_{y=0}^{2^t-1} e^{-2\pi i xy/2^t} |y\rangle$$

4. Measurement:

Measure the control register to obtain a value $y \in \{0, \dots, 2^t - 1\}$

Classical Post-Processing:

- Approximate the measured fraction $\frac{y}{2^t}$ by a rational number $\frac{s}{r}$ using continued fractions.
- Verify whether r satisfies $a^r \equiv 1 \bmod N$
- If r is even and $a^{r/2} \not\equiv \pm 1 \bmod N$, compute $\gcd(a^{r/2} \pm 1, N)$

If this yields a non-trivial factor of N , return it. Otherwise, repeat the procedure with a different base a .

```

# --- Constructia circuitului ---
def build_shor_circuit_n15(n_control, n_work, a):
    """Builds the quantum circuit for Shor's algorithm for N=15, a=7."""
    log_stderr("Building Shor Circuit (N=15, a=7)...")
    ctrl = QuantumRegister(n_control, name='ctrl')
    work = QuantumRegister(n_work, name='work')
    creg = ClassicalRegister(n_control, name='c') # Masuram rezultatul pe bitii de control
    qc = QuantumCircuit(ctrl, work, creg)

    qc.x(work[0]) # Initializam registrul de lucru cu |1>
    qc.h(ctrl)    # Aplicam transformata Hadamard pe bitii de control
    qc.barrier()

    log_stderr("Applying controlled modular exponentiation gates...")
    for k in range(n_control):
        power = 2**k
        try:
            U_gate = U_a_pow_mod15(a, power, n_work)
            if U_gate.definition.size() > 0:
                C_U_gate = U_gate.control(1)
                qc.append(C_U_gate, [ctrl[k]] + work[:])
                log_stderr(f"    Applied c-{{U_gate.name}} controlled by ctrl[{{k}}]")
            else:
                log_stderr(f"    Skipped c-{{U_gate.name}} (Identity) controlled by ctrl[{{k}}]")
        except ValueError as e:
            log_stderr(f"ERROR generating gate: {{e}}")
            raise # Ridicam exceptia pentru a fi capturat in main

    qc.barrier()
    log_stderr("Applying inverse QFT...")
    qft_inv = QFT(num_qubits=n_control, inverse=True, do_swaps=True, name="IQFT")
    qc.append(qft_inv.to_instruction(), ctrl)

    qc.barrier()
    qc.measure(ctrl, creg) # Masuram registrul de control in registrul clasic
    log_stderr("Circuit construction complete.")
    return qc

```

Code Snippet 1. Shor's Circuit for $N = 15$, $a = 7$

The implementation defines two quantum registers, `ctrl` and `work`, along with a classical register used to store phase measurements. The work qubit is initialised in the state $|1\rangle$, while the control qubits are placed into a Hadamard superposition. Within a for loop, the circuit applies controlled gates of the form $U_{a^{2^k}}$, each performing modular multiplication by $a^{2^k} \bmod 15$. Each instance of `U_a_pow_mod(15)` is validated: if the gate is non-trivial, the control is applied; otherwise, it is reported as an identity operation. Logical

barriers are inserted to enhance transpiler readability. The inverse Quantum Fourier Transform (labelled as "IQFT") is then applied, followed by measurement of the phase register into the classical register `c`. The `log_stderr()` messages describe each step in the execution console, useful for debugging when using IBM Q backends.

Resource Use and Probability of Success

A standard implementation of Shor's algorithm requires approximately $O(n^3)$ Toffoli gates due to the high cost of classical modular arithmetic when translated into reversible quantum operations. However, this complexity can be reduced to $O(n^2 \log n)$ when optimised modular multipliers are used, significantly improving efficiency for larger instances.

In terms of memory, the algorithm requires around $3n$ logical qubits: n for the work register, $2n$ for the control register (to provide sufficient resolution for the quantum Fourier transform), and additional ancilla qubits for arithmetic operations.

The algorithm has a high probability of success. Each run yields a useful value with probability at least $4/\pi^2 \approx 0.405$, which implies that only a few repetitions are needed on average to produce a valid result.

The overall circuit complexity is $O(t^3)$ elementary gates and $t + n$ qubits. Classical post-processing has complexity $O(t^2)$.

2.2.2 Grover's Algorithm

For an unstructured domain of size $M = 2^n$ containing $K \geq 1$ valid solutions, Grover's algorithm finds one such solution using $O(\sqrt{M/K})$ oracle queries. This matches the known lower bound for unstructured search problems [2].

Problem Formulation:

There exists an oracle function O_f that acts as follows:

$$O_f|x\rangle = (-1)^{f(x)}|x\rangle$$

Where $f(x) = 1$, if and only if x is a marked solution. The goal is to find a value x such that $f(x) = 1$.

Grover Iteration:

1. Initial State:

The system begins in a uniform superposition over all M states:

$$|\psi_0\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} |x\rangle$$

2. Oracle Application:

The oracle O_f inverts the phase of the marked states.

3. Diffusion Operator:

The diffusion operator is defined as:

$$D = 2|\psi_0\rangle\langle\psi_0| - \mathbb{I}$$

This step performs an inversion about the average amplitude.

A single Grover iteration is the composition $G = D O_f$. After j iterations, the quantum state undergoes a rotation in the two-dimensional subspace spanned by the uniform state $|s\rangle$ and the solution state $|w\rangle$, with a rotation angle of 2θ , where $\sin^2\theta = K/M$.

The optimal number of iterations is approximately:

$$j^* = \left\lceil \frac{\pi}{4} \sqrt{M/K} \right\rceil$$

After this number of iterations, measuring the system yields a marked solution with probability at least $1 - O(1/M)$

```

# --- Circuitul Grover ---
# Construim circuitul complet cu numarul optim de iteratii
def build_grover_circuit(marked_states):
    log_stderr("Building Grover Circuit...")
    if not marked_states:
        raise ValueError("Marked states list cannot be empty.")

    num_qubits = len(marked_states[0]) # Presupunem ca toate starile marcate au aceeaasi lungime
    log_stderr(f" Number of qubits (n): {num_qubits}")

    oracle = grover_oracle(marked_states, num_qubits)
    grover_op = GroverOperator(oracle)

    # Calculam numarul optim de iteratii
    num_marked = len(marked_states)
    n = num_qubits
    # Ne ocupam de cazul in care num_marked este 0 sau >= 2^n
    if num_marked == 0:
        optimal_iterations = 0
    elif num_marked >= 2**n:
        optimal_iterations = 0
    else:
        optimal_iterations = math.floor(math.pi / (4 * math.asin(math.sqrt(num_marked / 2**n))))
    log_stderr(f" Optimal number of Grover iterations: {optimal_iterations}")

    # Construim circuitul
    qc = QuantumCircuit(n, name="GroverSearch")
    # 1. Cream superpozitia
    qc.h(range(n))
    qc.barrier()

    # 2. Aplicam operatorul Grover iterativ
    if optimal_iterations > 0:
        qc.compose(grover_op.power(optimal_iterations), inplace=True)

    # 3. Masuram toate bitii
    qc.measure_all() # Adaugam un registrul clasic numit 'meas'
    log_stderr("Grover circuit construction complete.")
    return qc, num_qubits

```

Code Snippet 2. Grover's Circuit with automatic optimal iterations

The function receives a list of marked states (`marked_states`) and raises an exception if the list is empty. The number of qubits is inferred from the length of the first marked binary string, which is assumed to be uniform across all entries.

A phase oracle is created using `grover_oracle()`, and a `GroverOperator` is instantiated. The optimal number of iterations is then computed as $k = \lfloor \frac{\pi}{4} \sqrt{M/K} \rfloor$, where $N = 2^n$ and M

is the number of marked states. Special cases, such as $M = 0$ or $M \geq N$, yield zero iterations to prevent over-rotation and preserve correctness.

The circuit prepares all qubits in a uniform superposition, applies the GroverOperator repeatedly for the computed number of iterations, and then performs a full measurement of all bits. The function returns both the measurement circuit and num_qubits for external validation.

2.3.1 Lattice-Based Schemes

Lattice-based cryptography (LBC) is currently considered the most versatile and promising family of post-quantum algorithms [9], [12]. Its security is rooted in the hardness of certain mathematical problems defined over point lattices, such as the Shortest Vector Problem (SVP), Closest Vector Problem (CVP), Learning With Errors (LWE), and Short Integer Solution (SIS) [26], [27]. The LWE problem, introduced by Regev [26], has become a foundational component for constructing secure and efficient post-quantum cryptographic schemes.

LBC offers resistance against both classical and quantum attacks, combined with performance characteristics that support practical integration. This combination has positioned LBC schemes at the forefront of global standardisation efforts. Two notable examples selected by NIST include:

- **CRYSTALS-Kyber**, a key encapsulation mechanism (KEM)
- **CRYSTALS-Dilithium**, a digital signature scheme [12]

Kyber, standardised under FIPS 203 [13], is based on the Module Learning With Errors (MLWE) problem. It is designed to strike a robust balance between security, computational efficiency, and compact key and ciphertext sizes, making it suitable for protocols such as TLS [24].

Dilithium, standardised under FIPS 204 [14], is also based on MLWE, but it incorporates Module Short Integer Solution (MSIS) for enhanced security properties. Its primary goal is to provide compact signatures with fast signing and verification times.

Recent optimisations include support for specialised instruction sets, such as AVX2, which enhance performance on modern processor architectures [25].

The security of these schemes is measured against the most advanced lattice reduction algorithms, particularly the Blockwise Korkine-Zolotareff (BKZ) algorithm [18]. Parameter sets are selected to ensure substantial security margins against such attacks.

Another notable scheme is Falcon, selected by NIST and standardised as FIPS 206 [30]. Falcon is based on NTRU lattices and employs an optimised version of the GPV (Gentry-Peikert-Vaikuntanathan) framework, achieving very compact signatures and fast verification. Unlike Dilithium, Falcon relies on floating-point arithmetic and requires high-precision implementations, which complicates secure deployment in the absence of dedicated libraries or hardware.

2.3.2 Code-Based Schemes

Classic McEliece is one of the oldest cryptographic schemes, proposed as early as 1978. Despite its age, it remains one of the most resilient candidates for post-quantum cryptography, owing to its proven resistance against both classical and quantum attacks. Unlike other schemes, its foundation lies in binary Goppa codes, whose internal structure is extremely difficult to identify or exploit without the private key. Its security rests on the practical impossibility of distinguishing a well-constructed Goppa code from an arbitrary one and solving the decoding problem without access to the internal parameters.

A major obstacle to its widespread adoption is the massive size of its public keys, ranging from 260 KB to 1 MB depending on the security level. This makes it unsuitable for environments with strict storage or bandwidth constraints. However, it is well suited for contexts where security outweighs the need for frequent communication, such as critical infrastructure, enterprise-grade VPNs, or offline devices.

2.3.3 Hash-Based Schemes

Signature schemes built on hash functions (Hash-Based Signatures – HBS) represent a distinct branch of post-quantum cryptography. They rely solely on the cryptographic properties of hash functions — collision resistance, preimage resistance, and second preimage resistance. These concepts are well understood, time-tested in real-world applications, and, crucially, immune to quantum algorithms such as Shor's [31].

Early examples include Lamport's one-time signatures (OTS) and Merkle's signature scheme (MSS), which laid the groundwork for the field. Lamport introduced an efficient but single-use scheme: one key per message. Merkle extended this idea by organising multiple signatures in a binary tree structure, allowing many messages to be signed from a single trusted root. The problem? These schemes were stateful: the signer had to track key usage precisely. Reusing a key would fatally compromise security [31].

A breakthrough came with SPHINCS+, selected by NIST for standardisation under FIPS 205 [32]. It inherits ideas from schemes like XMSS and LMS, but unifies them in a fully stateless architecture, removing the need for key management. The result is a robust, predictable scheme that is far easier to deploy in real-world applications [33].

That said, SPHINCS+ has its trade-offs: signature sizes and generation times are significantly larger than those of lattice-based schemes, which may limit its suitability in performance-critical systems.

2.3.4 Isogeny-Based Schemes

Isogeny-based cryptography rests on a mathematically subtle but promising concept: the difficulty of constructing an isogeny — a group-structure-preserving map between two isogenous elliptic curves. This area caught researchers' attention for its ability to yield extremely compact keys and signatures [34].

Among the most studied schemes in this family were SIDH and its key encapsulation variant, SIKE — both finalists in the NIST standardisation process, largely due to their small key sizes [12].

However, in 2022, two independent attacks — one by Castryck and Decru, the other by Maino and Martindale — shattered the security assumptions of these schemes. By exploiting structural weaknesses in SIDH, attackers managed to break the schemes in polynomial time. As a result, SIKE was officially removed from the NIST competition during the fourth round [9].

These events deeply undermined confidence in isogenies as a viable post-quantum cryptographic foundation. Although alternatives like CSIDH still exist, they remain in early stages of development and lack robust validation. Consequently, the Keystone application continues to monitor this area, but prioritises schemes built on sound theoretical foundations and supported by practical, peer-reviewed implementations.

2.3.5 Multivariate Schemes

Multivariate Public Key Cryptography (MPKC) is based on the inherent difficulty of solving systems of multivariate polynomial equations over a finite field — known as the MQ problem [37]. This family is used predominantly for digital signatures, valued for its fast key generation and verification speed, though hindered by the large size of public keys.

One of the earliest significant schemes in this category was UOV (Unbalanced Oil and Vinegar) [38]. It introduced an efficient signing structure by dividing variables into two classes — "oil" and "vinegar" — to allow controlled construction of quadratic polynomials. Several optimisations and extensions have since been proposed based on this core idea. Rainbow, a layered variant of UOV, reached the finalist stage in the NIST competition due to its compact signatures and solid performance [39]. However, in 2022, a MinRank-style attack successfully extracted the private key, leading to Rainbow's removal from the standardisation process [40].

Other MPKC schemes, such as GemSS, remain under evaluation, but confidence in the family as a whole has diminished. The large key sizes and fragility of certain constructions have raised doubts regarding their long-term viability. From this standpoint, Keystone continues to monitor the field but prioritises schemes with rigorous security guarantees and robust key management mechanisms [37-39].

2.3.6 Symmetric-Key Schemes

Symmetric cryptography, exemplified by algorithms such as AES, uses the same key for both encryption and decryption. Remarkably, these schemes are unaffected by Shor's algorithm, though they are impacted by Grover's [2], [23].

Grover's algorithm offers a quadratic speedup for brute-force search, effectively reducing the security level from 2^k to $2^{k/2}$. Thus, AES-128 provides only 64 bits of quantum security, which is below the generally accepted threshold. For this reason, AES-256 is recommended to ensure 128 bits of post-quantum security [40].

Both NIST and the broader cryptographic community currently consider AES-256, alongside strong hash functions such as SHA-256 and SHA-3, to be quantum-resistant [9], [41]. In authentication schemes (e.g. HMAC) or key derivation contexts, longer hash outputs are recommended, though present standards provide adequate protection.

Thanks to its efficiency, transparent security model, and long-standing robustness, symmetric cryptography remains a cornerstone — even in the post-quantum era. Accordingly, the Keystone application employs AES-256 for encryption and data authentication, complementing KEMs and digital signature schemes in scenarios where hybrid cryptography is required.

3.1 Benchmarking Methodology and Relevant Metrics

In the transition toward post-quantum cryptography, performance evaluation is essential. It provides actionable insights into computational efficiency, resource consumption, and practical deployability. This research employs a rigorous benchmarking methodology aligned with comparative standards for both classical and post-quantum schemes, following established cryptographic performance practices and guidelines derived from the NIST PQC standardisation process [28].

The benchmark targets execution time and memory usage for fundamental cryptographic operations, tailored to the structure of each algorithm.

For KEMs, the evaluated operations include:

- **Key Generation:** generation of the public–private keypair.
- **Encapsulation:** generation of a shared secret and ciphertext using the public key.
- **Decapsulation:** recovery of the shared secret from the ciphertext using the private key.

For digital signature algorithms (DSA), the following are measured:

- **Key Generation:** generation of the public–private keypair.
- **Signing:** production of a digital signature using the private key.
- **Verification:** validation of the signature using the public key and message.

Symmetric algorithms (e.g. AES) are evaluated by measuring encryption and decryption performance in different modes (e.g. GCM, CBC). Classical asymmetric algorithms (e.g. RSA, ECC) are tested on operations equivalent to KEMs (ECDH) or signature schemes (ECDSA), depending on context.

Each operation is executed thousands of times (e.g. 10,000) to minimise system noise and produce reliable averages.

Timing is performed using high-resolution clocks: `QueryPerformanceCounter` on Windows and `std::chrono::steady_clock` on POSIX-compliant systems. The following temporal metrics are recorded for each operation:

- **Minimum Execution Time:** shortest time observed for a single run.
- **Maximum Execution Time:** longest time observed for a single run.

- **Average Execution Time:** arithmetic mean across all iterations.
- **Throughput:** operations per second, derived from the average execution time.

Beyond timing, memory usage is critical—particularly in constrained environments.

The benchmark also includes:

- **Peak Memory Usage:** the maximum size of the resident set (e.g. Peak Working Set Size on Windows). This is captured using platform-specific APIs like `GetProcessMemoryInfo` (Windows) and `getrusage` (Linux/macOS).
- **Average Memory Usage:** useful to assess sustained memory pressure, though generally less critical.

Additionally, the size of generated materials is recorded:

- **Public Key Size:** byte-length of the public key.
- **Private/Secret Key Size:** byte-length of the secret or private key.
- **Ciphertext Size (KEM):** size in bytes of the encapsulated ciphertext.
- **Signature Size (DSA):** size in bytes of the generated signature.

The software supports parameter set selection (e.g. Kyber-512/768/1024), enabling direct comparisons between security levels and performance trade-offs. Results are structured as JSON objects to allow seamless parsing via `lowDB` and interactive visualisation with `plotly.js`. Data is memoised locally to avoid redundant recomputation and ensure a smooth user interface experience.

3.2 Implementation of Cryptographic Algorithms

The algorithms evaluated in this study are integrated via `liboqs` (for post-quantum schemes) and `OpenSSL` (for classical schemes), relying on validated, standardised, and optimised implementations.

Post-Quantum Implementations (PQC)

All PQC schemes are sourced from liboqs, the C library developed by the Open Quantum Safe (OQS) project [29]. It provides consistent interfaces and support for most algorithms selected or under evaluation by the NIST PQC process. The benchmark includes:

- **CRYSTALS-Kyber (ML-KEM)**: a lattice-based KEM selected as NIST's primary standard (FIPS 203) [13]. Variants Kyber-512, Kyber-768, and Kyber-1024 are tested.
- **CRYSTALS-Dilithium (ML-DSA)**: a lattice-based digital signature algorithm, standardised as FIPS 204 [14]. Variants Dilithium 2, 3, and 5 are benchmarked.
- **FALCON**: a NIST finalist, based on NTRU lattices, producing extremely compact signatures. It is set to become FN-DSA (FIPS 206) [30].
- **SPHINCS+**: a hash-based signature scheme, standardised as FIPS 205 [15]. Benchmarked variants include SPHINCS+-SHA256-128f and SPHINCS+-SHAKE256-128s.
- **Classic McEliece**: a code-based KEM, NIST finalist, known for its historically unmatched resistance to cryptanalysis and large public key sizes.

The benchmark tool interfaces directly with the C-level API of liboqs for performing key generation, encapsulation/decapsulation, and signing/verification operations as appropriate per algorithm.

Classical Implementations

For reference, classical cryptographic algorithms are tested using OpenSSL [31], the de facto standard in production environments. These include:

- **AES**: symmetric cipher tested in GCM mode (authenticated encryption), with 128-, 192-, and 256-bit keys.
- **RSA**: public-key algorithm evaluated with 2048-, 3072-, and 4096-bit keys.
- **ECDH**: key exchange mechanism tested on P-256 and P-384 elliptic curves.
- **ECDSA**: digital signature algorithm tested on the same curves.

These implementations ensure that classical performance metrics mirror real-world deployment scenarios. Each function is called through native APIs to extract relevant benchmark metrics without modification.

3.3 Implementation of Quantum Algorithms

The practical implementation of quantum algorithms—especially cryptanalytic ones like Shor and Grover—poses both engineering and theoretical challenges. These stem from the fragility of quantum states, the physical constraints of current hardware, and the difficulty of mapping abstract algorithmic logic onto executable instructions for real qubits [43].

3.3.1 Fundamental Constraints of Quantum Hardware

Quantum computation is fundamentally bounded by several interdependent physical factors:

- **Qubit quality and coherence:** Qubits, the carriers of quantum information, suffer from decoherence. Coherence times (T_1 and T_2) define the computational window before information degrades. Superconducting qubits in modern systems reach tens to hundreds of microseconds [43]. Gate operations further introduce noise, with fidelity exceeding 99% for single-qubit gates and lower values for two-qubit gates. In the absence of full error correction, usable computational volume is effectively bounded by the product of qubit count and circuit depth.
- **Connectivity and topology:** Most quantum processors lack full qubit interconnectivity. Algorithms requiring operations between non-adjacent qubits incur SWAP gate overhead, which increases latency and exposure to noise. The cost varies based on physical layout (e.g., linear chain, 2D grid).
- **SPAM errors:** Errors in state preparation and measurement (SPAM) remain significant, ranging from 0.1% to 1% in current systems. These contribute materially to final output uncertainty [43].

3.3.2 Circuit Compilation and Transpilation

The transformation of a high-level quantum algorithm into a hardware-executable circuit involves a multi-stage pipeline known as transpilation [43]:

- **Gate synthesis:** High-level logical gates (e.g., Toffoli, Fredkin) are decomposed into native universal gate sets (typically single-qubit rotations and CNOT or CZ). The goal is to minimise depth and gate count. This process draws on group theory and optimal control.
- **Qubit allocation:** Logical qubits are mapped to physical qubits based on hardware topology. Heuristic or exact allocation algorithms are employed to reduce SWAP insertion cost.
- **Routing:** SWAP gates are inserted where non-adjacent interactions are required, based on the selected qubit layout.
- **Scheduling:** Gate execution is temporally ordered to obey hardware constraints like gate durations and permissible parallelism.
- **Optimisation and error mitigation:** Techniques such as gate resynthesis, dynamical decoupling, and noise extrapolation are applied to reduce depth and increase resilience.

This process is computationally demanding, often NP-hard, and requires advanced heuristic strategies for real-world devices.

3.3.3 Resource Estimation for Cryptographically Relevant Algorithms

A critical step in evaluating the quantum threat is estimating the resources required to run relevant cryptanalytic algorithms, especially Shor's for factoring and discrete logarithms.

Shor's Algorithm: Breaking RSA-2048 requires millions of physical qubits in a fault-tolerant model. Early estimates suggested $\sim 2n$ logical qubits for an n -bit modulus, but recent simulations based on surface codes and error rates of 10^{-3} show that around 20

million physical qubits are needed to factor RSA-2048 [55]. The number of Toffoli gates—main contributors to overall circuit complexity—is $O(n^3)$ in its naive form but can be reduced to $O(n^2 \log n)$ or better via arithmetic optimisations [43].

Grover's Algorithm: Offers quadratic speedup for unstructured search, though its impact on symmetric cryptography is relatively easy to mitigate by doubling the key length. For AES-256, approximately 2^{128} Grover iterations are required [23].

3.3.4 Quantum Error Correction and Fault Tolerance

Scalable quantum computation critically depends on **quantum error correction (QEC)**, required to counteract decoherence and operational noise [50]. QEC codes protect quantum information by encoding a single logical qubit into an ensemble of physical qubits, allowing errors to be detected and corrected without direct state measurement. The **threshold theorem** states that if the error rate falls below a critical value, quantum computation can proceed reliably [53]. However, achieving this threshold requires significant overhead in resources and remains one of the primary roadblocks to building practical cryptanalytic quantum systems.

3.4 Performance Metrics

Quantum algorithm implementations in the Keystone application collect a comprehensive set of metrics, grouped into four categories: noise characterisation, circuit complexity analysis, execution performance measurement, and algorithmic success evaluation. These metrics provide essential data for estimating the practical feasibility of quantum attacks on classical cryptographic schemes under realistic conditions.

3.4.1 Noise and Hardware Error Characterisation

Modern quantum platforms are marked by significant noise levels, which directly impact algorithmic stability. Keystone automatically extracts relevant metrics from IBM Quantum backends to accurately characterise the execution environment.

Quantum coherence metrics

- **Relaxation time T1 (μ s):** The time a qubit takes to lose its energy to the environment. Measured for each qubit and averaged across the device. Typical values range from 100–300 μ s. Retrieved using `qubit_properties()` or inherited methods.
- **Dephasing time T2 (μ s):** The duration a qubit maintains phase coherence before collapsing. Usually shorter than T1, it imposes strict limits on circuit depth. Automatically extracted, with backward-compatible fallbacks across multiple API versions.

Gate fidelity and readout errors

- **Gate error rate (%):** A global indicator of gate fidelity, especially for two-qubit gates (CX, CNOT, ECR), which are the most error-prone. Averaged over all available qubit pairs. Typical rates range from 0.1% to 2%.
- **Readout error (%):** The probability that a qubit is measured incorrectly. Measured per qubit and aggregated via averaging. Keystone uses direct backend access and robust fallbacks for cross-version compatibility.

Quantum volume and system characterisation

- **Quantum volume:** An IBM-defined metric reflecting a quantum system's practical computational capacity. It incorporates qubit count, gate fidelity, connectivity, and executable depth. Automatically extracted from backend configuration, it denotes the largest square circuit that can be executed with a success probability above $2/3$.

3.4.2 Circuit Complexity and Optimisation Metrics

The compilation and optimisation of quantum circuits directly influence execution feasibility, especially for cryptanalytic algorithms. Keystone evaluates circuit complexity both before and after transpilation, to quantify the impact of optimisations.

Circuit depth: The number of sequential gate layers applied post-transpilation. This represents the ideal execution time under maximum parallelism. Depth is directly correlated with coherence requirements.

- **Shor** is transpiled with optimisation level 2, offering a balance between compilation duration and circuit quality.
- **Grover** uses optimisation level 3 to maximise success rate. Typical values range between 100–1000 layers.

Gate count: Quantum gates are counted after transpilation, with an emphasis on two-qubit gates (CX), as they are primary sources of both error and latency. The total count reflects both logical complexity and resource cost.

Transpilation impact: Transpilation includes gate decomposition into native instructions, qubit mapping onto physical architecture, and routing optimisation to minimise SWAP operations. Each step directly affects circuit depth, execution fidelity, and algorithm success probability.

3.4.3 Execution Performance and Resource Consumption

Quantum algorithm execution involves measurable durations and resource consumption, which are critical for estimating the real-world viability of quantum attacks.

Timing metrics

- **Total time (seconds):** Includes all phases — circuit construction, optimisation, backend queue time, execution, and postprocessing. Reflects overall real-world execution cost.

- **QPU time (seconds):** The actual time spent on the quantum processor, excluding classical stages. This is a direct measure of quantum resource consumption and becomes increasingly relevant for complex algorithms.
- **Circuit duration (μ s):** Time elapsed from the first gate application to the final measurement on hardware. This is the most granular execution metric and is strongly tied to system coherence. It is extracted via metadata and alternative API endpoints for robust cross-platform support.

Resource allocation and execution metadata

- **Job identifier:** A unique code assigned to each run, useful for monitoring, tracking, and reproducibility.
 - **Number of shots:** Number of repetitions per execution. Impacts both statistical precision and total runtime.
 - **Shor:** 4096 shots
 - **Grover:** 8192 shots
 - **Backend specification:** Documents the selected backend—name, generation, status.
 - In real executions, Keystone automatically selects the least-busy backend that meets hardware requirements.
 - For simulations, **AerSimulator** is used to ensure consistent conditions.
-

3.4.4 Algorithmic Success and Output Quality Assessment

Beyond hardware metrics, Keystone integrates criteria for algorithmic success and quantitative evaluation of output quality—both essential to understanding the real impact of quantum algorithms on cryptographic security.

In the case of **Shor’s algorithm**, success is defined by identifying a non-trivial factor of the target number ($N = 15$). This requires correct period extraction from the quantum distribution and successful classical postprocessing to derive the factors. The outcome is evaluated binarily: the factors were either identified or not. Complementarily, the quantum

measurement distribution is analysed to detect interference peaks associated with the theoretical periodicity of the modular function. The clarity of this interference is then correlated with the noise level during execution.

For **Grover's algorithm**, success is measured by the identification of the marked state. An execution is considered successful when the most frequently measured state corresponds to the expected solution. This outcome signals effective amplitude amplification.

Additionally, the accuracy of the resulting distribution is evaluated through the ratio of correct to incorrect measurement frequencies, allowing estimation of algorithmic efficiency under noise.

To strengthen the analysis, each execution produces a raw histogram of the measurement distribution, supporting post-hoc interpretation, statistical validation, and cross-experiment comparison. All execution errors are also systematically documented—from hardware failures and timeouts to rejected parameter validations—allowing full diagnostic visibility into failed runs and a deeper understanding of practical algorithmic limitations.

3.4.5 Comparative Analysis and Contextualisation of Results

Keystone's extended metrology framework enables not only granular evaluation of individual executions but also structured comparisons across platforms, backends, and algorithms, providing a solid empirical foundation for extrapolation and post-quantum risk modelling.

Simulator-based executions offer an idealised benchmark, free from noise and architectural constraints, while executions on physical hardware reveal concrete limitations introduced by gate fidelity, connectivity, and readout errors. Comparing the two enables direct quantification of noise impact on algorithmic performance.

By repeating executions on the same backends at different points in time, one can observe the technological evolution of quantum platforms. Improvements in gate fidelity, reductions in readout error, or increases in quantum volume are all clear signals of

progress — factors that feed into more realistic projections for the emergence of quantum systems capable of viable cryptanalytic attacks.

Finally, by correlating circuit depth, logical complexity, and success rate, Keystone allows estimation of resource requirements for larger cryptographic instances, such as factoring a 1024-bit number. These extrapolations, far from speculative, are empirically grounded and contribute to rigorous, reproducible post-quantum risk mapping. In this sense, Keystone’s metrological architecture becomes more than just a measurement tool: it forms a predictive map, where each documented execution contributes to our understanding of the future threat landscape in cryptography.

4. Architecture and Design of the Application: “Keystone”

4.1 Application Requirements and Specifications

4.1.2 Functional Requirements

Keystone’s functionality is structured along four operational axes, each reflecting a critical direction in the transition toward post-quantum cryptography:

FR1: Cryptographic Benchmarking

- Support for full performance evaluation of PQC algorithms: Kyber (ML-KEM), Dilithium (ML-DSA), FALCON (FN-DSA), SPHINCS+ (SLH-DSA), Classic McEliece;
- Comparison against classical primitives: RSA, ECDH, ECDSA, AES;
- Execution of standardised benchmarks: key generation, encryption/decryption, signing/verification across all supported security levels;
- Production of robust statistical metrics via configurable iteration and methodologies aligned with NIST guidelines.

FR2: Quantum Resource Analysis

- Implementation of Shor's algorithm to estimate the resources required for RSA factorisation;
- Implementation of Grover's algorithm and quantification of its impact on symmetric cryptography;
- Execution on IBM Quantum hardware via cloud integration;
- Asynchronous job management, adaptive to backend latency and queue conditions.

FR3: Hybrid Cryptographic System

- Combination of ML-KEM with AES-GCM for authenticated encryption;
- Integration of ML-DSA for digital signatures;
- Full cryptographic pipeline: encryption → signing → decryption → verification;
- Flexible parameterisation for each component: algorithm, security level, output format.

FR4: Metrics and Reporting

- Generation of detailed reports in structured JSON format for further analysis or automated processing;
- Graphical visualisation of results and comparison functions between algorithms and runs;
- Export of quantum executions: circuit depth, gate count, resource consumption, fidelities;
- Fast management of dataset collections.

4.1.3 Non-Functional Requirements

NFR1: Modular Architecture and Extensibility

- Clean hexagonal architecture for clear separation of responsibilities and ease of maintenance;
- Well-defined interfaces to integrate new cryptographic algorithms or quantum backends;
- Plugin-based extensibility without modifying core logic;
- Backward compatibility when adding algorithms or modifying parameters.

NFR2: Performance and Scalability

- Sub-second response time for benchmark initialisation and runtime monitoring;
- Asynchronous processing of quantum jobs without blocking the interface;
- Memory usage optimised for execution on standard desktop/laptop systems.

NFR3: Security and Reliability

- Secure storage of API credentials for quantum services;
- Handling of cryptographic material in accordance with best practices;
- Validation and sanitisation of all user-configurable inputs;
- Full error handling and graceful degradation in case of connectivity issues or external service unavailability.

NFR4: Usability and Accessibility

- Intuitive graphical interface accessible to researchers, analysts, and students;
- Comprehensive documentation and contextual assistance for cryptographic concepts and result interpretation;
- Cross-platform compatibility: Windows, macOS, and Linux.

4.1.4 Target User Profiles

Keystone addresses a diverse spectrum of users within the ecosystem of post-quantum cryptographic research and application:

- **Cryptographic researchers:** from academia or industry, interested in comparative evaluation of algorithms, performance analysis, and post-quantum security estimation;
 - **Security analysts:** professionals responsible for cryptographic migration strategies, risk assessments, and technology selection in institutions or corporate environments;
 - **Students and educators:** academic users seeking to explore PQC concepts in practice, algorithmic behaviour, and foundational principles of quantum computing.
-

4.1.5 Integration Requirements with External Systems

Full functionality of the Keystone application assumes integration with the following external libraries and platforms:

- **Open Quantum Safe (liboqs):** the primary library for post-quantum algorithms, providing standardised and optimised implementations of PQC schemes;
- **OpenSSL:** a well-established library for classical algorithms (RSA, ECDSA, AES), used as a comparative baseline in benchmarking;
- **IBM Quantum Services:** cloud platform for executing quantum algorithms on real hardware;
- **Qiskit:** quantum development framework used for building, optimising, and managing quantum circuits.

The detailed architectural design that supports these integrations will be analysed in Section 4.4, where the clear separation between domain logic, application services, and execution infrastructure forms the foundation for maintaining the system's robustness and extensibility.

4.2 Use Cases and Specification Allocation

4.2.1 Use Case 1: Cryptographic Algorithm Benchmarking

Scenario:

A researcher seeks to compare the performance of CRYSTALS-Kyber (ML-KEM) and RSA for key exchange mechanisms. They select the Kyber-768 parameter, set 10,000 iterations for statistical robustness, and launch the benchmark to obtain execution time and memory consumption metrics.

Component Allocation:

- **Interface Layer:**

The BenchmarkRunner component provides the configuration interface, allowing selection of algorithm, parameter presets, and execution monitoring. The implementation includes dropdown controls, iteration settings, and real-time progress visualisation.

- **Application Layer:**

Orchestration services manage execution flow, validate parameters, coordinate iteration cycles, and aggregate results using BenchmarkParams and BenchmarkResult entities.

- **Domain Layer:**

Cryptographic entities model algorithms, security parameters, and performance metrics, enforcing strict typing and business logic validation.

- **Infrastructure Layer:**

Platform-specific adapters execute the benchmarks via native libraries (liboqs for PQC, OpenSSL for classical) and access system APIs for timing and memory measurement.

The clear separation of responsibilities enabled by the hexagonal architecture ensures that the interface remains agnostic of algorithmic implementation. Benchmarking logic is fully isolated from infrastructure details, supporting portability and maintainability.

4.2.2 Use Case 2: Resource Estimation for Quantum Algorithms

Scenario:

A quantum computing researcher aims to estimate the practical resources required to break RSA-2048 by running Shor's algorithm. They perform a demonstrative simulation factoring $N=15N = 15N=15$ on IBM Quantum hardware and analyse circuit depth, gate count, and qubit requirements, extrapolating toward real cryptographic parameters.

Component Allocation:

- **Interface Layer:**

The quantum execution interface allows circuit visualisation, parameter configuration, and result display. Circuit properties and quantum measurements are presented in an accessible format.

- **Application Layer:**

Quantum orchestration services handle circuit construction, job dispatch to quantum backends, and processing of measurement outputs.

- **Domain Layer:**

The `QuantumJob` and `CircuitMetrics` entities model essential data — backend selection, circuit depth, fidelity metrics, and statistical results — defining the semantics of quantum executions.

- **Infrastructure Layer:**

Quantum adapters (e.g., `shor_n15.py`) interface with IBM Quantum through Qiskit, managing circuit building and execution, transpilation, backend selection, and noise characterisation.

This use case highlights the importance of the port-adapter model in quantum contexts, where backends differ widely in API and execution semantics. Hexagonal architecture enables smooth transitions between simulated and hardware-based execution without disrupting domain logic.

4.2.3 Use Case 3: Demonstrating a Hybrid Cryptographic System

Scenario:

A security architect exploring post-quantum migration requires a practical demonstration of a hybrid system that combines post-quantum and classical primitives. They configure a complete cryptographic flow using ML-KEM-768 for key encapsulation, AES-256-GCM for symmetric encryption, and ML-DSA-65 for digital signing. The system runs the full cycle: encrypt → sign → verify → decrypt, all on a sample message.

Component Allocation:

- **Interface Layer:**

The `EncryptionRunner` component allows algorithm selection, security parameter configuration, and monitoring of the multi-stage cryptographic process.

- **Application Layer:**

Orchestration services coordinate the full flow: key derivation, routing the message through the cryptographic modules, and managing error handling.

- **Domain Layer:**

Hybrid entities define functional relations between the KEM, the symmetric

encryption scheme, and the signature scheme. These ensure cryptographic coherence and parameter consistency across stages.

- **Infrastructure Layer:**

Native adapters (Node.js + C++) under the `encryption/` directory (e.g., `kyber_node_addon.cpp`, `dilithium_node_addon.cpp`) implement critical cryptographic operations with native performance and a clean JavaScript API.

The presence of C++ addons highlights the architectural decision to isolate high-performance logic in the infrastructure layer, keeping the domain and application layers language-agnostic and free of runtime-specific concerns.

4.2.4 Use Case 4: Real-Time Analysis and Research-Oriented Data Export

Scenario:

A performance analyst wishes to monitor in real-time a set of benchmark runs for lattice-based signature schemes and export the results for later statistical analysis. They track live metrics, observe throughput evolution, and save the final results in JSON format for inclusion in academic publications.

Component Allocation:

- **Interface Layer:**

The `MetricsCard` and `Speedometer` components display live progress and resource utilisation directly within the `BenchmarkRunner` view.

- **Application Layer:**

Aggregation and export services handle data collection, formatting, and serialization, ensuring data integrity and statistical validity.

- **Domain Layer:**

Result entities define the structure and validation rules for performance data,

maintaining semantic consistency across algorithm types and benchmark scenarios.

- **Infrastructure Layer:**

Monitoring adapters access OS-level APIs for memory measurement, persist the collected data via file services, and manage inter-process communication to update the interface in real time.

The separation between low-level data collection and its domain-level representation allows results to be reported in a standardised format, independent of how or where measurements are displayed.

4.2.5 Use Case 5: Comparative Analysis in an Educational Context

Scenario:

A student or instructor wants to conduct a side-by-side comparison of post-quantum signature schemes in a classroom setting. They configure multiple benchmark scenarios, visualise performance metrics live, and generate interpretable data suitable for didactic use, presentations, or lab reports.

Component Allocation:

- **Interface Layer:**

The same MetricsCard and Speedometer modules are used to make live metrics visually accessible in the BenchmarkRunner UI.

- **Application Layer:**

Services manage real-time data aggregation and export, streamlining the pipeline from benchmarking to educational reporting.

- **Domain Layer:**

The result objects maintain well-defined data schemas and apply validation logic to guarantee accurate and meaningful performance comparisons.

- **Infrastructure Layer:**

Monitoring modules interface with OS-level instrumentation to measure usage, log results to disk, and push updates to the UI asynchronously.

This case showcases how a consistent architectural foundation can serve both advanced research and pedagogical needs, simply by adapting visualisation and export layers without altering the core benchmarking logic.

4.2.6 Architectural Mapping and Design Validation

The use cases analysed above validate several core architectural decisions:

- **Dependency Inversion:**

In every scenario, high-level logic—domain entities and application services—remains entirely decoupled from infrastructure. Cryptographic libraries, quantum APIs, and OS interfaces can be replaced without touching the core logic. Algorithms can be swapped, quantum backends migrated, and the application remains portable.

- **Single Responsibility Principle:**

Each layer upholds a distinct and focused role. The interface handles user interaction. The application layer coordinates data flow. The domain layer defines rules and invariants. Infrastructure connects to external systems.

- **Extensibility:**

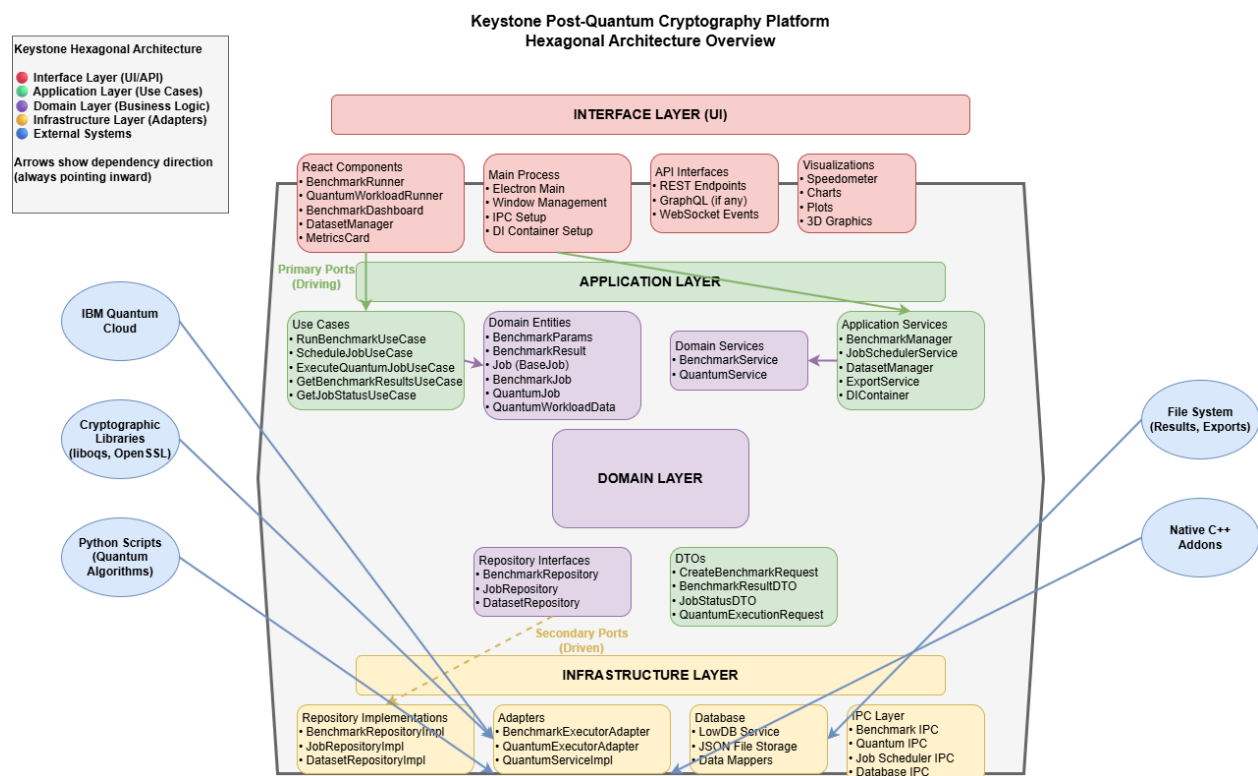
The system's modularity allows for seamless integration of new algorithms, quantum backends, or additional forms of analysis. These extensions are implemented via adapters, requiring no changes to existing components.

This structure affirms the principles outlined in Section 4.4. The hexagonal model emerges as the system’s backbone, enabling the coherent integration of diverse requirements within a stable, compartmentalised, and future-ready framework.

4.3 Architectural Diagrams

4.3.1 Hexagonal Diagram

Keystone’s architecture is illustrated through a suite of independent diagrams, each capturing a distinct layer of the software model. When read in sequence, they reveal the transition from static concepts to dynamic behaviours, culminating in integration with external ecosystems.

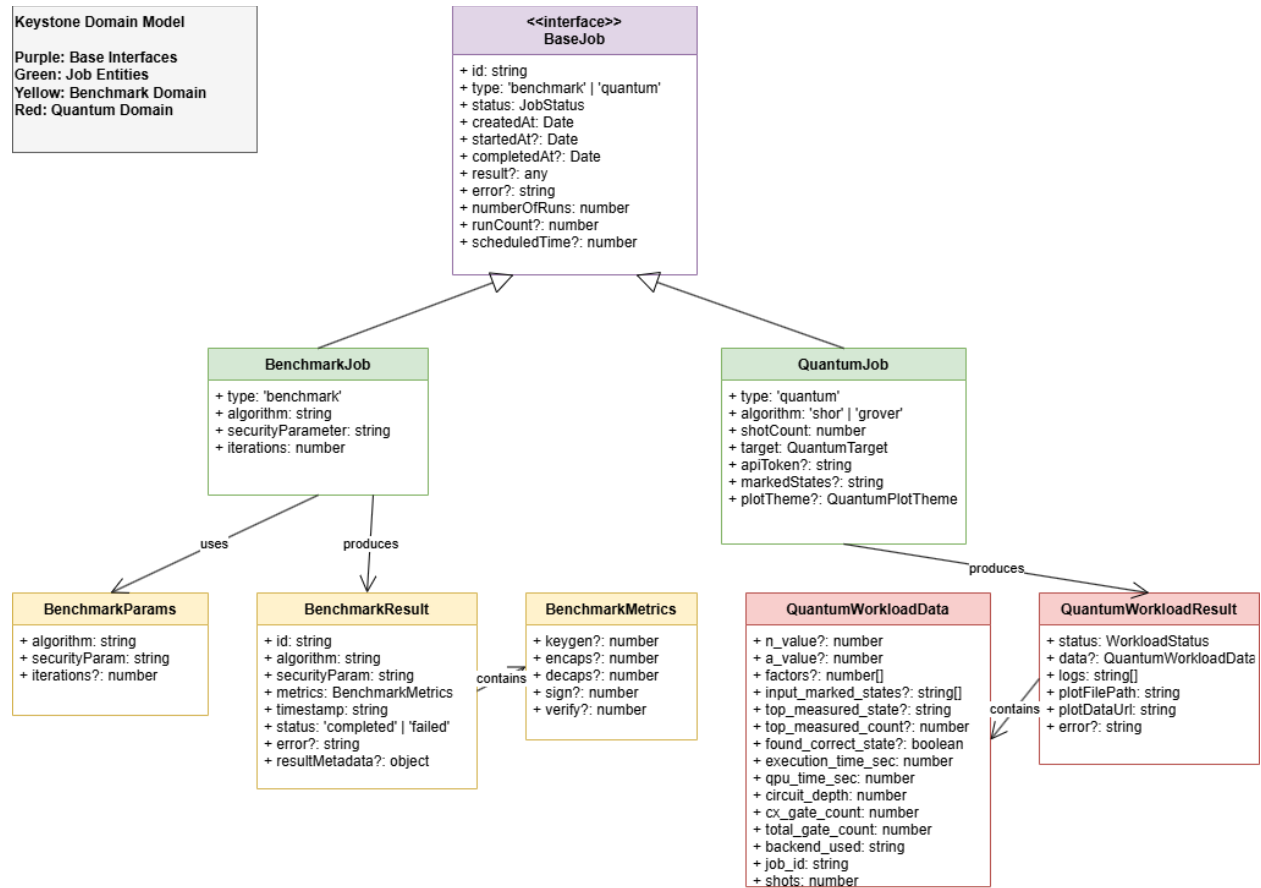


The diagram illustrates the separation of four distinct layers: Interface (UI), Application, Domain, and Infrastructure—framed by external systems such as IBM

Quantum Cloud, cryptographic libraries, Python scripts, output files, and C++ addons. The flow of dependencies always points inward, enforcing the principle of dependency inversion. Green ports (driving) expose use cases like `RunBenchmarkUseCase`; yellow ports (driven) anchor infrastructure adapters such as LowDB repositories, IPC services, and JSON mappers. The result is a fully isolated domain core, accessed exclusively through application services.

The diagram also highlights how React/Electron handles the interface, while application services orchestrate cryptographic or quantum executions without reliance on I/O specifics. As a consequence, the interface can be replaced without affecting business logic, and the infrastructure can evolve (e.g., switching from LowDB to PostgreSQL) without altering the domain entities.

4.3.2 Domain Model Diagram



The BaseJob interface defines attributes common to all jobs—identity, state, timestamps, and results. It is extended by both BenchmarkJob and QuantumJob, underscoring the structural parity between classical tests and quantum tasks. BenchmarkJob consumes security parameters and yields a BenchmarkResult, which itself contains BenchmarkMetrics (e.g., number of keys generated, encapsulations, signatures, verifications). On the quantum side, QuantumJob outputs a QuantumWorkloadResult that encapsulates QuantumWorkloadData, holding detailed circuit measurements.

The model employs small, well-scoped aggregates to simplify persistence without relying on an ORM and to support strict Domain-Driven Design.

4.3.3 Sequence Diagram – Use Case “Run Benchmark”

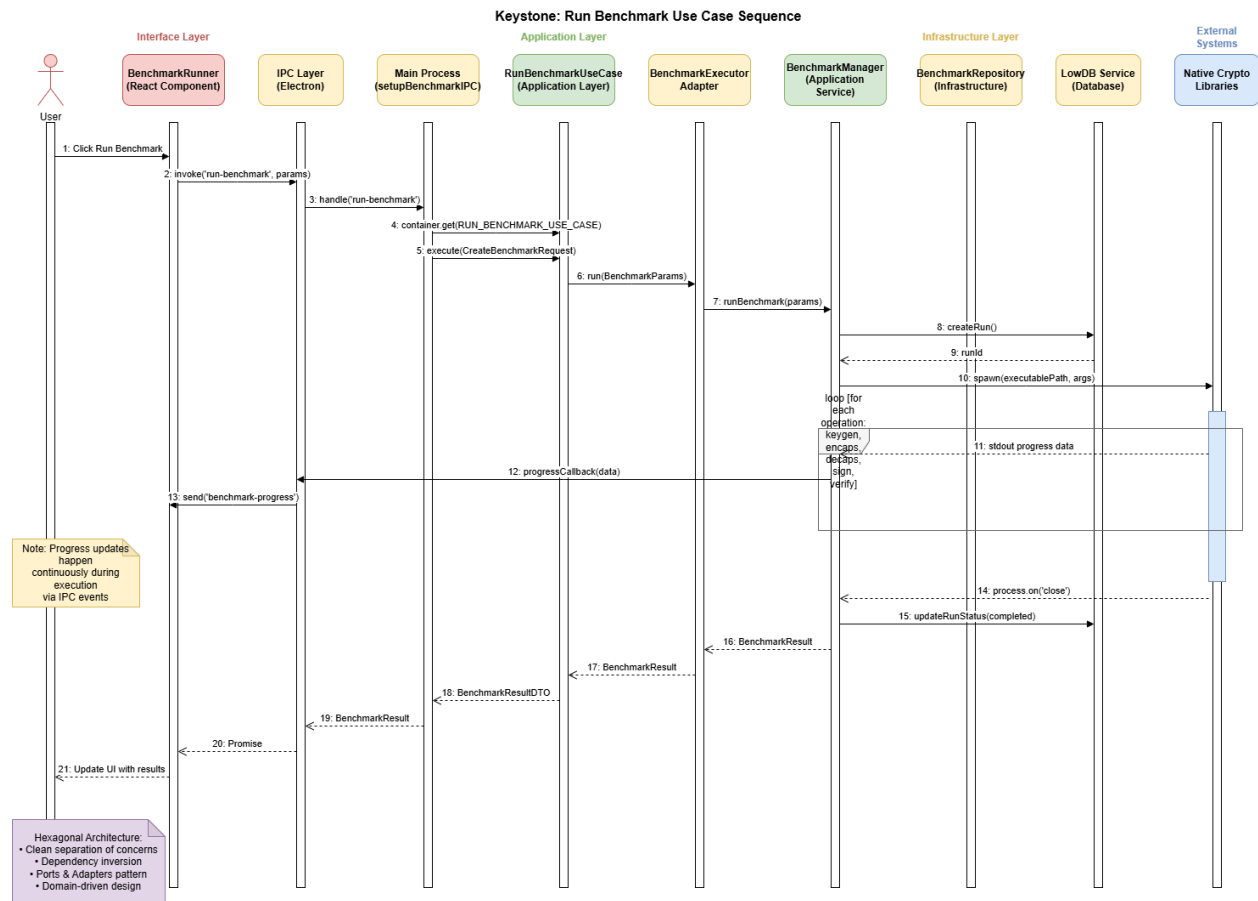


Figure 3

The flow begins in the BenchmarkRunner React component. An IPC event travels through Electron and reaches RunBenchmarkUseCase, where a CreateBenchmarkRequest is instantiated. This request is delegated to the BenchmarkExecutor adapter, which spawns a native binary via the infrastructure repository. Progress is streamed through stdout, captured and relayed via IPC to the UI in real time. At the end, results are mapped into a BenchmarkResultDTO and returned as a promise, keeping the UI asynchronous yet consistent with domain logic.

4.3.4 Sequence Diagram – Use Case “Run Quantum Workload”

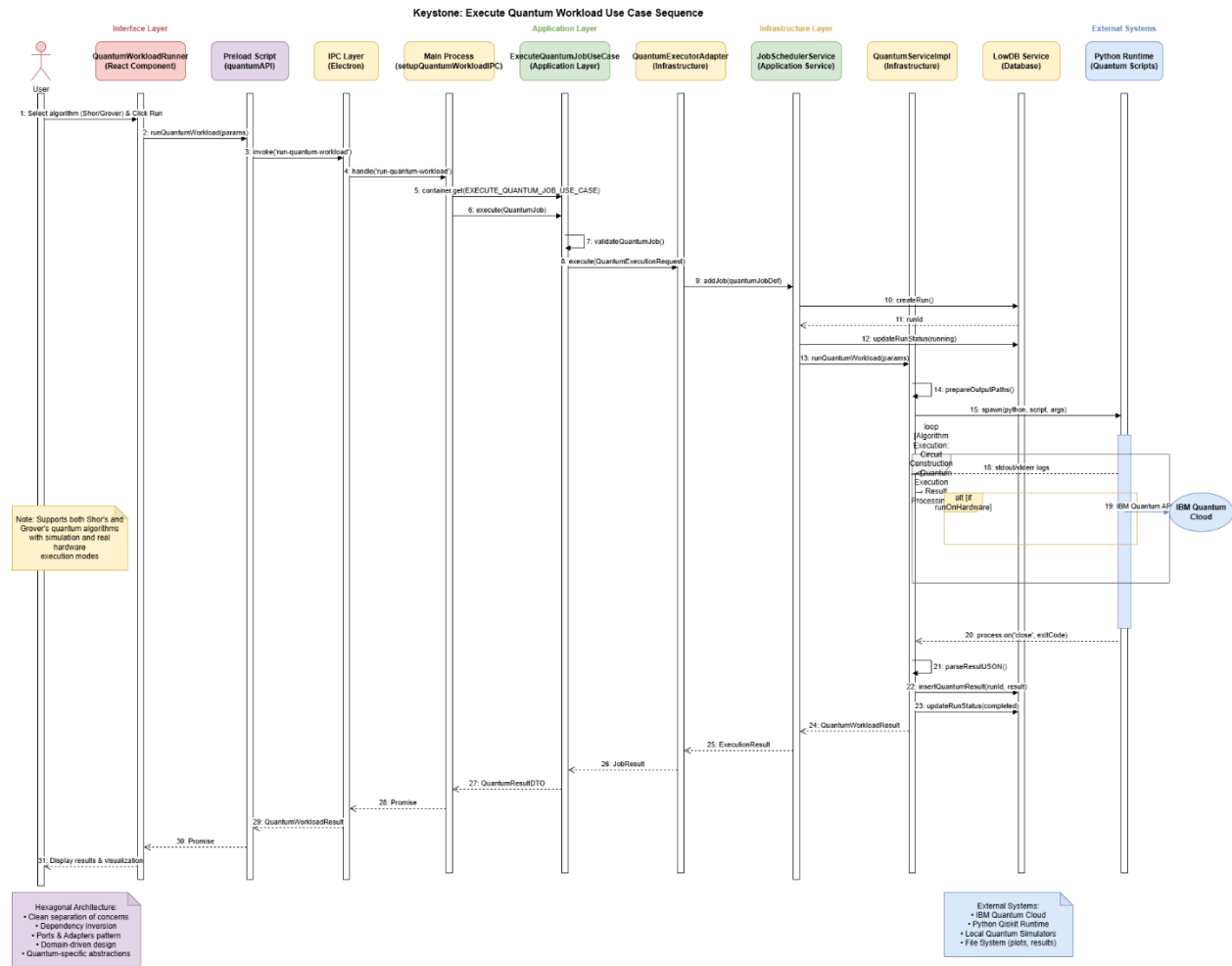


Figure 4

This diagram builds upon the previous mechanism by adding a pre-load script that grants access to the Python runtime. `ExecuteQuantumJobUseCase` validates parameters, then calls `QuantumExecutorAdapter`, which decides between local simulation and remote execution via IBM Quantum Cloud. Raw results undergo a preprocessing pipeline—noise filtering and shot aggregation—before being wrapped in a `QuantumWorkloadResult`. Meanwhile, `JobSchedulerService` manages the execution queue, enforcing concurrent job limits to maintain interface SLOs.

4.3.5 Sequence Diagram – Use Case “Run Encryption”

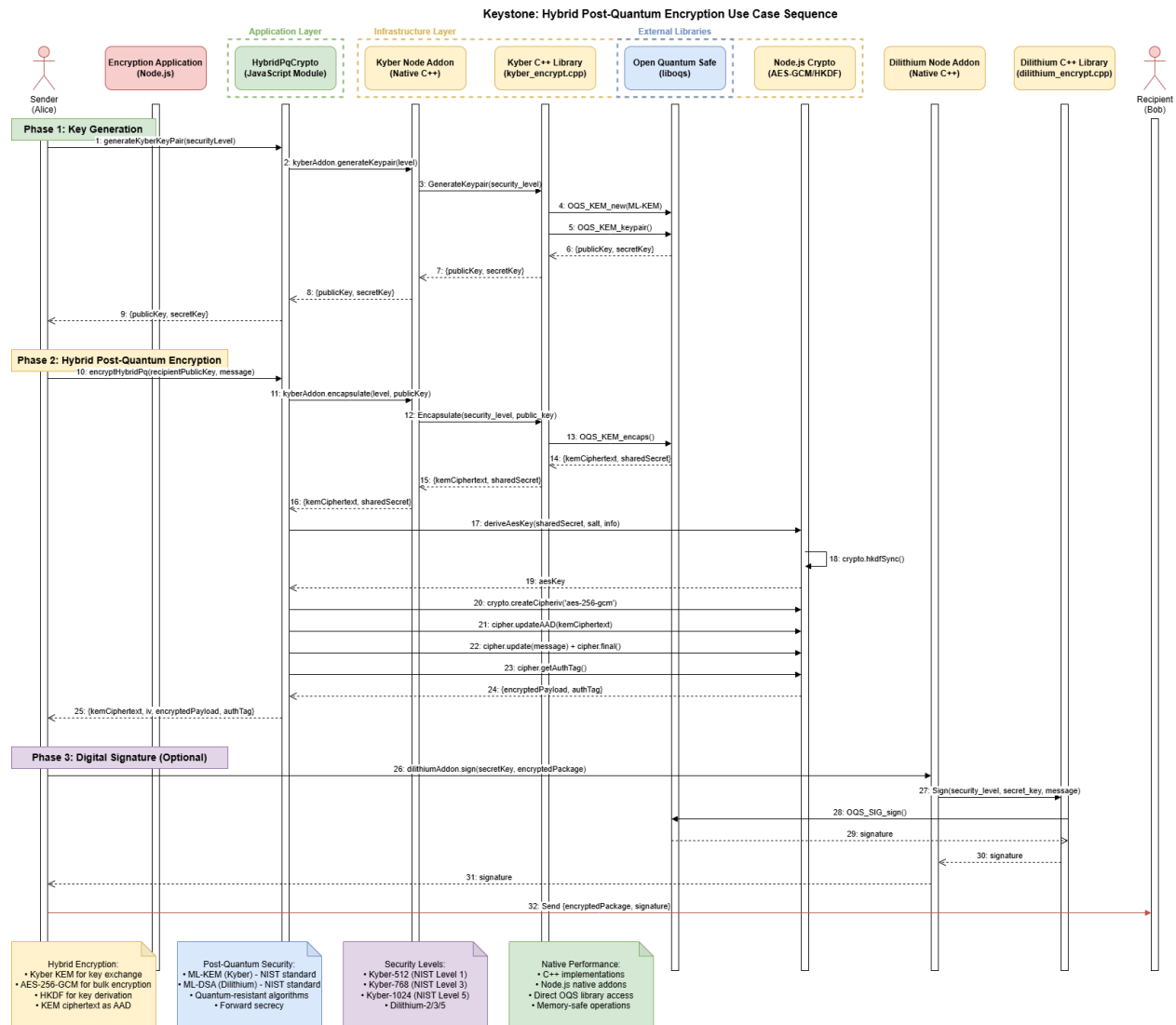


Figure 5

The flow splits into three phases. First, the Kyber C++ addon generates KEM key pairs, temporarily stored in LowDB and exposed via the JavaScript HybridPqcCrypto module. In phase two, a hybrid payload is composed: KEM for key exchange, AES-GCM for confidentiality, and the KEM tag included as AAD. The payload’s hash is computed using libsodium through OpenQuantumSafe to avoid memory-unsafe side-channels. Optionally, a Dilithium signature ensures non-repudiation; OQS_SIG_sign() is invoked in a separate thread to prevent blocking Node.js’ event loop.

4.3.6 State Diagram – Job Lifecycle

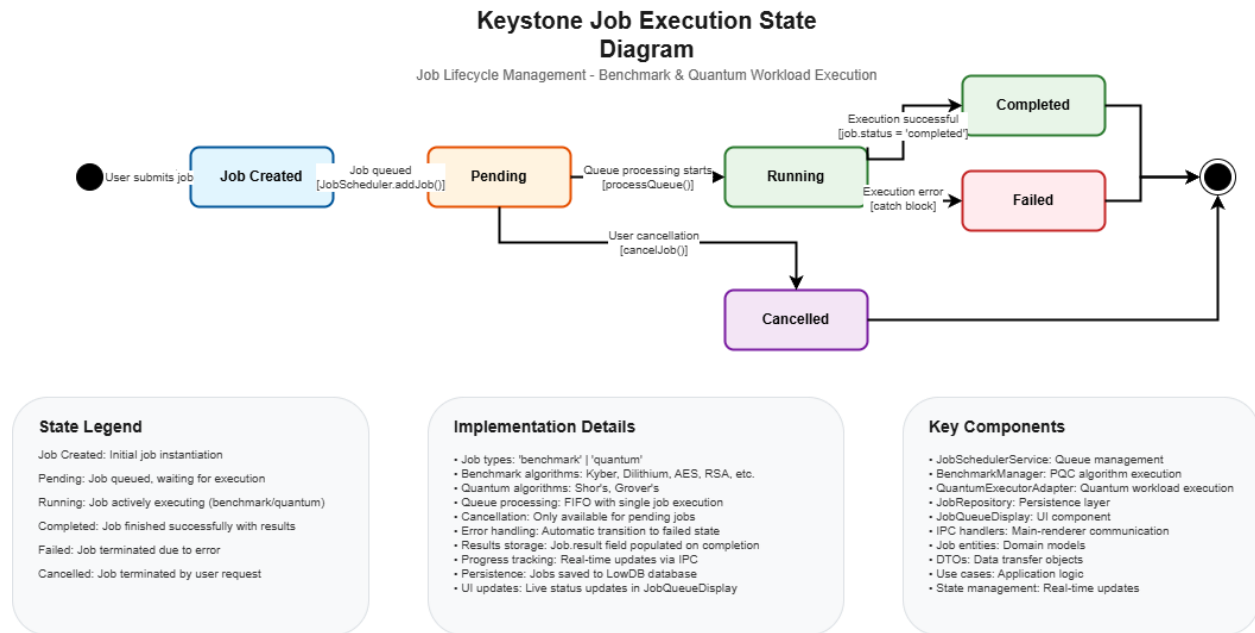


Figure 6

A job moves through five distinct states: Created → Pending → Running → (Completed | Failed) or Cancelled. The transition from Pending to Cancelled is only permitted before execution begins, preventing abrupt termination of native processes. Transitions are governed by JobSchedulerService. Successful jobs have their results attached in the result field of the entity, while failed jobs trigger a fallback into the Failed state. Smooth persistence in LowDB ensures the job queue is restored upon application restart.

4.3.7 Package Diagram

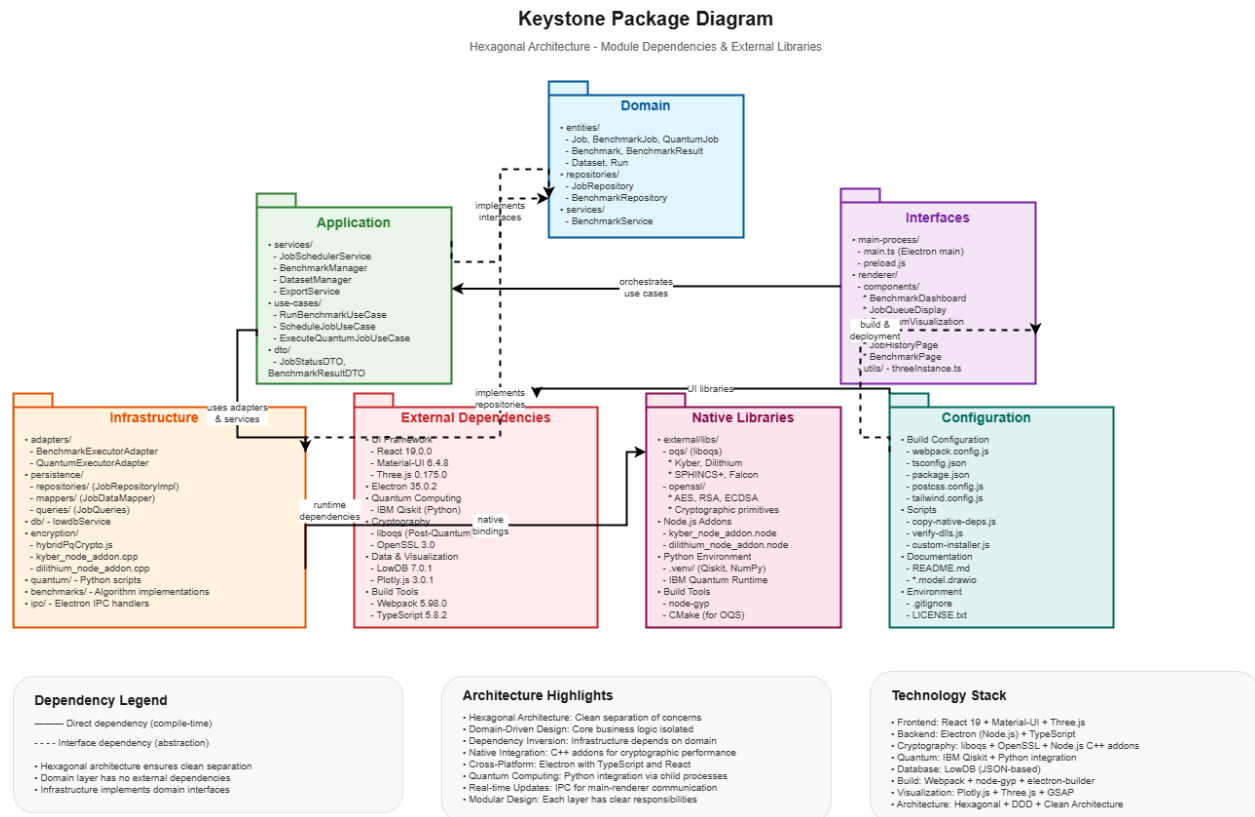


Figure 7

The blocks illustrate a modular implementation of the hexagonal architecture.

- The **Domain** layer (blue) is dependency-free, exposing only entities and repository interfaces.
- The **Application** layer (green) orchestrates use cases and depends on repositories via inversion of control.
- The **Infrastructure** layer (orange) implements adapters such as IPC handlers, native executors, and the LowDB persistence layer.
- The **Native Libraries** package contains OQS bindings and custom C++ crypto modules.
- The **External Dependencies** layer (red) introduces React, Electron, and IBM Qiskit into the runtime.

- The **Configuration** package (turquoise) encapsulates Webpack settings and deployment scripts.

Solid arrows represent compile-time dependencies; dotted arrows indicate interface-level connections.

4.4 Hexagonal Architecture

The architecture of the Keystone application combines two complementary models: the hexagonal structure proposed by Alistair Cockburn [57] and the principles of Clean Architecture formulated by Robert C. Martin. Together, they offer a clear separation of responsibilities and a robust integration scheme with external systems. The result is a flexible, coherent, and easily extensible framework, purpose-built for the complex needs of cryptographic benchmarking and quantum algorithm execution.

4.4.1 Architectural Rationale and Design Motivation

This architectural model was chosen due to the specific requirements of cryptographic and quantum applications:

- **Maintainability and scalability:** Algorithms change often. New proposals, optimisations, and standard revisions arise frequently. The architecture allows replacing or extending cryptographic logic without propagating changes across components. The business logic remains stable. Adaptors can be swapped in isolation.
- **Dependency clarity:** The application interacts with native libraries, quantum APIs, and OS-level components, each with their own semantics and error patterns.

Layered separation limits the impact of these variations. Domain code is not dependent on infrastructure. Infrastructure code contains no business rules.

- **Rigorous testability:** The separation of logic allows full unit testing. Components can be mocked or replaced with simulations. Even quantum executions can be tested without access to real hardware. Every adaptor is testable in isolation.
-

4.4.2 Architectural Layer Specification

Keystone's implementation follows a four-layer structure, each with clearly defined responsibilities and interactions.

4.4.2.1 Domain Layer

The domain is the core logic of the application. It contains no dependencies on infrastructure, UI, or external libraries. This layer defines the rules, entities, and services that capture the system's semantics.

Main entities:

- **BenchmarkParams:** defines execution parameters like algorithm, security level, and number of iterations.
- **BenchmarkResult:** contains performance metrics and measured results, with built-in validations.
- **QuantumWorkloadData:** represents collected quantum execution data: circuit depth, fidelity, and measured distributions.
- **Job:** base interface for async executions (**BenchmarkJob**, **QuantumJob**);
- **AlgorithmDefinition:** algorithm classification (**KEM**, **Signature**, **Symmetric**).

Domain services:

- BenchmarkService: validates and orchestrates benchmarking logic;
- QuantumService: manages quantum executions and result interpretation.

Algorithmic constants:

- SUPPORTED_ALGORITHMS: includes Kyber, Dilithium, Falcon, McEliece, SPHINCS+, AES, RSA, etc.;
 - SECURITY_PARAMS: mappings between algorithms and available security levels.
-

4.4.2.2 Application Layer

This layer orchestrates all use cases. It coordinates domain entities without depending on the technical infrastructure or user interface. It contains flow logic and turns user intent into coherent operations.

Use cases:

- RunBenchmarkUseCase: full benchmark execution flow;
- ScheduleJobUseCase: asynchronous workload scheduling;
- ExecuteQuantumJobUseCase: building, submitting, and analysing quantum jobs;
- GetBenchmarkResultsUseCase: querying and aggregating results;
- GetJobStatusUseCase: monitoring job progress.

Services:

- BenchmarkManager: runs the benchmark and emits progress events;
- JobSchedulerService: handles queues, priorities, and parallel executions;
- DatasetManager: dataset import/export, statistical analysis;
- DIContainer: runtime-configured dependency injection container.

DTOs:

- BenchmarkDTO, JobDTO, QuantumDTO: transfer structures with parametric validation.
-

4.4.2.3 Infrastructure Layer

This layer adapts external libraries and system services to the interfaces defined in the domain. It bridges logic and actual execution on hardware or software platforms.

Cryptographic adapters:

- BenchmarkExecutorAdapter: interface with native libraries for execution;
- C++ addons (kyber_node_addon.cpp, dilithium_node_addon.cpp) for high-performance execution via liboqs;
- binding.gyp: build configuration for native addons.

Quantum adapters:

- QuantumExecutorAdapter: integrates IBM Quantum backends through Python scripts;
- shor_n15.py, grover_search.py: specialised implementations for transpilation and circuit analysis.

System adapters:

- persistence: result and configuration storage;
 - db: structured data access;
 - ipc: inter-process communication (Electron main <-> renderer).
-

4.4.2.4 Interface Layer

This layer handles direct user interaction and exposes the application's capabilities externally. It includes both the graphical interface and integration points for other systems.

Main UI components:

- **BenchmarkRunner:** allows algorithm configuration and real-time execution monitoring.
- **QuantumWorkloadRunner:** launches and tracks quantum algorithm executions.
- **EncryptionRunner:** provides a full demonstration of a hybrid cryptographic workflow.
- **BenchmarkDashboard:** displays aggregated results, execution history, and relevant metrics.

Supporting UI components:

- **BenchmarkResultCard, MetricsCard, Speedometer:** real-time data display and continuous visual feedback.
- **RecentBenchmarks:** provides quick access to previous runs directly from the interface.

External integration:

- **Handlers** in the main process control system-level capabilities, such as native addon execution or hardware metadata access.
 - **API handlers** expose functionality to external interfaces, useful for scripting or app integration.
 - **Renderer components** (Electron) build and manage the main interface while maintaining logical separation from the backend.
-

4.5 User Interface and UX Design

4.5.1 Design Philosophy and UX Principles

Keystone's interface was designed with a dual purpose: to serve advanced cryptographic research while remaining accessible in educational contexts. The design prioritises clarity, real-time responsiveness, and user autonomy.

- **Thematic aesthetics and intuitive navigation:**

The visual identity suggests a digital lab. The colour scheme uses Material UI with violet accents (#9747FF). Icons are clear, and spacing is precisely calibrated. Even complex flows are navigable without visual clutter.

- **Real-time feedback and transparency:**

Benchmarking may take seconds or minutes. To sustain engagement, the interface offers continuous feedback: synchronised animations, live metrics, and clear visual cues at every step.

- **Contextual, adaptable information:**

Users vary in experience. The UI provides concise explanations where needed—algorithm classes, security levels, paradigm differences. Additional info is always available, never obtrusive.

- **Adaptive, configurable design:**

The interface adapts to user preferences and device types. It supports light/dark modes, controllable animations, and variable information density. The goal is maximum readability and complete control in any context.

4.5.2 Benchmark Execution and Visualisation Interface

This is the core interface of the application, where users configure and launch algorithm executions. It starts with the `BenchmarkRunner` component, which groups algorithms by cryptographic family: lattice-based, hash-based, code-based, and classical.

Security parameters are selected from a validated list with brief descriptions of protection levels. Iteration count is user-configurable, with sensible defaults calibrated by algorithm complexity: 10,000 for Kyber and AES, 1,000 for signature schemes like Dilithium, Falcon, or ECDSA, and 100 for heavier algorithms such as McEliece, SPHINCS+, or RSA.

Execution is visualised by a **Speedometer** animation—an elastic-needle dial built with GSAP. The needle moves in real time and reflects progress from key generation to signing or decryption. Animated transitions include a soft elastic effect for dynamic feedback. The gauge background changes colour according to state. On completion, it turns green and pulsates to indicate success. Below the speedometer, a panel shows the algorithm name, current operation, and active security parameter.

The BenchmarkDashboard provides a performance overview: min/max/avg execution times in microseconds, throughput in operations per second, and both average and peak memory consumption. The layout uses three columns: metrics on the left, the speedometer in the centre, and system resource usage on the right. This layout ensures a natural reading flow and allows real-time monitoring at a glance.

4.5.3 Quantum Algorithm Execution Interface

This interface centres around the QuantumWorkloadRunner component, which allows specific configuration for each algorithm. For Shor's algorithm, the default parameter N is set to 15—a didactic and circuit-light choice. For Grover's algorithm, the user manually provides the target state to be searched within the quantum space.

The execution backend is selected from the IBM Quantum network. The system automatically assesses resource availability, queue lengths, and hardware capabilities, then suggests an optimal backend accordingly. The number of **shots** is configured per algorithm: 4,096 for Shor and 8,192 for Grover, ensuring statistically stable results. Users

may also enable a **quantum circuit viewer**—useful for educational purposes or technical validation of the generated structure.

Executions are asynchronous and cloud-based, subject to hardware-specific queuing. The interface continuously monitors the job's queue position, estimated execution time, and real-time status. Upon completion, results are fetched automatically, including measurement distributions and detailed circuit data (e.g., gate count, logical depth, average fidelity). If execution fails, the system displays clear error messages and retries automatically when possible.

4.5.4 Hybrid Encryption Module Interface

The EncryptionRunner component demonstrates the real-world applicability of post-quantum cryptography in a full workflow: encryption, signing, verification, and decryption of a message. The process begins with algorithm selection via intuitive dropdown menus:

- **ML-KEM (Kyber):** Security levels 512, 768, and 1024
- **ML-DSA (Dilithium):** Levels 2, 3, and 5

Each option includes brief explanations comparing them to classical standards, helping users make informed choices. All keys are generated simultaneously through a single action. After generation, the app displays key sizes in bytes, giving instant feedback on resource usage. Input messages are entered in a text field with a character counter, encoding options, and live preview.

The resulting cryptographic material is presented transparently with fine-grained control. Public keys are shown in full (base64); private keys are hidden by default but can be toggled with a `VisibilityIcon`. The encrypted packet's structure is fully detailed:

- Ciphertext from the KEM
- AES initialisation vector

- Encrypted payload with authentication tag
- Digital signature from ML-DSA

Each field includes its own copy button. On successful copy, a visual toast confirms the action. This mechanism supports rapid testing, secure sharing, and effective documentation of each cryptographic step.

4.5.5 Accessibility Features and User Adaptation

The Keystone interface is designed to accommodate a wide range of preferences and usage scenarios. The focus is on personalisation, clarity, and adaptability, regardless of the user's experience level or technical context.

For visual adaptation, the app provides a global toggle between dark and light modes, managed through the Material-UI theme provider, ensuring a consistent theme across all components. Users can disable background animations, useful both on slower systems and in contexts where a more sober interface is preferred. Information density can also be manually adjusted to suit small screens or a preference for more breathable UI layouts.

The interface includes tooltips and expandable sections that explain technical terms without cluttering the screen. For quick usage scenarios, preconfigured benchmarks are available that can be launched with a single click.

Cross-platform compatibility is ensured through the Electron architecture, which guarantees a consistent experience on Windows, macOS, and Linux, while respecting the accessibility conventions of each operating system.

4.5.6 Performance Optimisation and Responsiveness

Keystone remains smooth and responsive even during intensive cryptographic

executions. This is achieved through an architecture optimised for efficiency and a series of frontend-level techniques.

Benchmark execution is fully asynchronous. Each process runs in isolation, and progress updates are transmitted through IPC channels, keeping the interface continuously responsive. Performance metrics are sent incrementally using optimised protocols, reducing re-rendering overhead. Under heavy load, the system activates an internal scheduler that visually degrades only non-essential elements while maintaining full interactivity.

Rendering and result display are memory-optimised. Long lists use virtual scrolling, meaning only visible elements are rendered. Complex data and charts load only when viewed. Animations built with GSAP are hardware-accelerated and designed to minimise CPU load.

Long-term stability is supported by explicit cleanup mechanisms. Heavy components use `useRef` to avoid unnecessary re-renders, and data transformations are memoised using `useMemo`, recalculating only when sources change. On component unmount, resources are rigorously released, preventing memory buildup during extended sessions.

4.6 Backend Integration and Management

Keystone's backend architecture orchestrates multiple execution environments: high-performance native cryptography, asynchronous quantum algorithms, and extensive metric processing. Throughout this system, clear separation of concerns is maintained in line with hexagonal architecture principles. Each component is isolated yet interoperable, allowing coherent integration without rigid dependencies or side effects between layers.

4.6.1 Electron + Node.js Orchestration Platform

Keystone uses Electron to combine a modern graphical interface with a high-performance native backend, split across separate processes. Communication between the frontend and internal logic is handled through an event-driven IPC (Inter-Process Communication) system.

- **IPC:** The file `src/infrastructure/ipc/ipc.ts` defines over 30 specialised channels, covering operations like `kyber-generate-keypair`, `run-benchmark`, and `get-cpu-usage`. Each execution runs in isolation, avoiding main thread (renderer) blocking.
 - **Preload security model:** The file `preload.js` defines a controlled API exposed via `contextBridge`, limiting internal Node.js exposure. Each IPC call is filtered through a whitelist, and unauthorised commands are explicitly rejected.
 - **Dependency injection:** The file `di-container.ts` manages the lifecycle of backend services, allowing them to be replaced with mocks for testing or reconfigured dynamically at runtime. This mechanism ensures logic consistency and avoids circular references.
-

4.6.2 Native Execution Infrastructure (Cryptography)

To bypass JavaScript's performance limitations, Keystone runs cryptographic operations in C++ using native addons that interface directly with optimised libraries like `liboqs` and `OpenSSL`.

- **Custom C++ addons:** The files `kyber_node_addon.cpp` and `dilithium_node_addon.cpp` provide direct execution interfaces, bypassing JavaScript interpretation entirely. Conversion between Node.js structures (`Buffer`) and C++ vectors is handled via `RAII` models, ensuring memory safety and full control over resource lifecycles.

- **Build system:** Configured through `binding.gyp` and `CMakeLists.txt`, it handles automatic dependency resolution and platform-specific compilation. Both static and dynamic linking are supported, depending on the OS.
 - **Cross-platform compilation:** The toolchain is auto-detected, whether Visual Studio, GCC, or Clang. Settings are platform-conditioned, with dynamically generated paths and links for both development and production modes.
-

4.6.3 Benchmark Task Management

Each benchmark execution in Keystone is treated as an isolated process, launched and monitored individually to ensure both application safety and operational transparency. This approach guards against local crashes and provides detailed visibility into each execution phase.

Execution management is orchestrated by `BenchmarkManager`, which uses `child_process.spawn` to launch independent processes without blocking the main control thread. Each benchmark is registered with a unique ID, execution state, and metadata used for cancellation, tracking, or post-processing.

Real-time monitoring relies on live parsing of output using regular expressions. It extracts execution times in milliseconds, memory usage in kilobytes or megabytes, and detailed stats for each operation (e.g., signing or encryption). For complex algorithms, the system accepts JSON progress streams, allowing structured incremental updates.

Data persistence and integrity are ensured through integration with **LowDB**, which journals all executions and updates state in real time. To maintain consistency, the system simulates transactions: upon detecting a failure, all changes are automatically rolled back, ensuring that only valid, complete executions are stored.

4.6.4 Quantum Integration and External APIs

Quantum algorithm execution in Keystone involves coordinated interplay between the Node.js backend, specialised Python scripts, and IBM Quantum's cloud services. Each execution is handled as a fully asynchronous flow, decoupled from the core logic yet monitored end-to-end.

The Python scripts (`shor_n15.py`, `grover_search.py`) are launched from Node.js via `spawn`, with real-time capture of `stdout` and `stderr` streams. Virtual environments are activated automatically before execution, keeping the global system isolated from quantum-specific dependencies.

Interaction with IBM Quantum occurs through an authenticated API. Backend selection is automated, based on available qubits and current queue load. Requests include configurable retry logic, and polling is adapted with timeouts to ensure resilience against network latency.

Authentication data is securely stored via Electron's `safeStorage` mechanism. The IBM token is encrypted locally and decrypted only at runtime, within an isolated context inaccessible from the UI.

Quantum jobs are processed entirely in the background. Each job is tracked independently, and upon completion, results are sent automatically to the database and trigger interface updates—no manual refresh required.

4.6.5 Memory Management and Performance Optimisation

To prevent overload from intensive cryptographic executions, Keystone isolates each task in its own memory space. This approach allows the main application to function unhindered even if a subprocess fails or consumes excessive resources.

- **Process isolation:** Every benchmark runs in its own process. If a crash occurs, the process is safely terminated without affecting the global app.
 - **Native memory management:** C++ code uses RAII models and explicit cleanup. Every malloc is matched with a free, and exception handling includes automatic resource release mechanisms.
 - **System monitoring:** The `setupSystemMetricsIPC` function provides data on per-core CPU usage and aggregate memory consumption. These metrics guide real-time allocation decisions, especially under high computational load.
-

4.6.6 Error Handling and System Reliability

Keystone integrates a multi-layered error capture and handling system, designed to ensure continuous operation even in the face of instability or unpredictable behaviour from external components.

- **Controlled capture and propagation:** IPC channels return structured responses including error codes and detailed messages. Native C++ addons catch exceptions and serialise them for forwarding to the Node.js backend.
 - **Graceful fallback:** Local failures—such as temporary API unavailability or a missing algorithm—do not block overall functionality. Critical operations are guarded with retry logic and configured fallbacks.
 - **Full logging:** All operations are logged at distinct levels (info, warn, error). Even `stderr` output is saved, enabling detailed post-mortem analysis in case of failure.
-

4.6.7 Extensibility and Future Integration Support

Keystone's backend architecture was designed from the ground up for extensibility, anticipating both the evolution of cryptographic standards and the diversification of

quantum technologies. Modularity underpins this vision, allowing new features to be added without global restructuring.

- **Cryptographic extensibility** is straightforward. Adding a new algorithm requires only a compatible infrastructure adaptor. Higher layers—domain entities, use cases, and UI—need no changes. This functional decoupling ensures codebase stability and enables fast iteration without architectural risk.
- **Quantum backend integration** is made possible through full interface abstraction. Platforms like Google Cirq, Azure Quantum, or Amazon Braket can be integrated via minimal adaptors. Each native API is mapped to Keystone semantics, allowing uniform result processing and consistent error handling regardless of the source.

Multi-platform portability support

Conditional compilation is handled through platform-specific directives (`#ifdef`, CMake options), allowing Keystone to be built for diverse OSes and hardware architectures. Adaptors for files, crypto libraries, and toolchains are platform-aware and configurable.

Delivery scenarios span multiple distribution modes. Keystone can be compiled as a standalone application (local executable), exported to a web variant via WebAssembly, or run headless for automated testing or distributed benchmarking servers. All variants are supported without major structural changes.

This flexibility positions Keystone as a truly adaptable platform. It can quickly incorporate cryptographic innovations, adapt to emerging research demands, and deploy across varied environments—from classrooms to industrial-grade audit and analysis infrastructures.

4.7 Application Testing and Validation Strategies

As an application designed for research in post-quantum cryptography, Keystone demands a rigorous, layered testing strategy tailored to its hexagonal architecture. The

primary testing framework is **Jest**, selected for its seamless integration with TypeScript and Node.js, fast execution, and robust support for isolation, mocking, snapshots, and coverage reporting.

The test structure mirrors the application architecture. The `src/tests` directory replicates the functional layout of the codebase, enabling clear mapping between modules and their associated tests. Tests for domain logic, use-case orchestration, infrastructure adapters, and interface components are grouped in a way that ensures full isolation while allowing intuitive navigation across layers.

- **tests/domain** validates entities and services in complete isolation, with no external dependencies.
- **tests/application** verifies orchestration logic using mocked domain services.
- **tests/infrastructure** contains both isolated tests (using mocks for native libraries and quantum backends) and real integrations when possible.
- **tests/interfaces** covers UI components and APIs through unit, behavioural, and snapshot tests.

End-to-end tests complete the testing pyramid by simulating real-world flows—for instance, running a full benchmark or executing a quantum job. These tests provide final validation of system behaviour from the user's perspective.

The strategy targets four key objectives:

1. **Functional correctness**
2. **Regression prevention**
3. **Reliability across diverse environments**
4. **Support for extensibility and refactoring**

In critical zones—such as domain logic and application orchestration—**Test-Driven Development (TDD)** is applied. Tests are written before implementation, encouraging modular design and full coverage from the first version.

Jest offers a robust execution environment:

- Each test runs in a sandboxed context.
- Mocks are used to isolate external adaptors.
- Snapshots verify UI and response structure integrity against unintended changes.
- Coverage reports highlight untested paths, guiding further test priorities.

Automated validation is integrated into the CI pipeline. On every commit or pull request, the following steps are triggered:

- Clean dependency installation
- Static analysis via ESLint and the TypeScript compiler
- Full Jest suite execution
- Coverage evaluation

If thresholds are not met, the build is automatically aborted.

In parallel with functional validation, some tests monitor **performance**. These include time-based assertions to catch regressions in cryptographic execution. Stability is measured by comparing output consistency across repeated runs, ensuring determinism within statistical bounds. Error scenarios are also simulated—such as backend unavailability or external library failure—to verify resilience and proper exception reporting.

This layered strategy, aligned with the hexagonal architecture and powered by Jest, ensures not just current functionality but also future-proofing. In a research-grade cryptographic application—where correctness, transparency, and reproducibility are non-negotiable—this approach is not optional but foundational.

4.8 Application Security Considerations

In a context involving the handling of cryptographic keys, sensitive materials, and interaction with external backends, security in Keystone is treated as an architectural concern—not merely a functional feature. The implemented measures aim to protect both

in-flight data and the logical isolation of components, while preserving the integrity of the software infrastructure.

Sensitive data is handled exclusively in volatile memory. Private keys are never exposed in the user interface without explicit consent. All cryptographic material is retained only for the minimum operational duration and is explicitly overwritten where possible. Intermediate data generated during benchmarks or hybrid operations is not written to disk unless explicitly controlled. In such cases, the user is notified and involved in the decision. The IBM Quantum token is securely stored locally using `electron.safeStorage`, encrypted automatically and accessible only from within the application.

To validate the integrity of the Keystone infrastructure, a partial **reverse engineering analysis** was performed on the compiled binary using **Ghidra**. The objective was to identify critical initialisation routines and verify integrity during runtime.

```

FUN_144a4f1e4                                     XREF[2]:      entry:144a4f1d4(c)
144a4f1e4 48 89 5c      MOV      qword ptr [RSP + 0x18],RBX
          24 18
144a4f1e9 55          PUSH     RBP
144a4f1ea 48 8b ec      MOV      RBP,RSP
144a4f1ed 48 83 ec 30    SUB      RSP,0x30
144a4f1f1 48 8b 05      MOV      RAX,qword ptr [DAT_14b77e040]      = 32h    2
          48 ee d2 06
144a4f1f8 48 bb 32      MOV      RBX,0x2b992ddfa232
          a2 df 2d
          99 2b 00 00
144a4f202 48 3b c3      CMP      RAX,RBX
144a4f205 75 74          JNZ      LAB_144a4f27b
144a4f207 48 83 65      AND      qword ptr [RBP + 0x10],0x0
          10 00
144a4f20c 48 8d 4d 10    LEA      RCX,[RBP + 0x10]
144a4f210 ff 15 6a      CALL     qword ptr [->KERNEL32.DLL::GetSystemTimeAsFile... = 0b7424be
          11 cf 06
144a4f216 48 8b 45 10    MOV      RAX,qword ptr [RBP + 0x10]
144a4f21a 48 89 45 f0    MOV      qword ptr [RBP + -0x10],RAX
144a4f21e ff 15 f4      CALL     qword ptr [->KERNEL32.DLL::GetCurrentThreadId]    = 0b741e14
          0e cf 06
144a4f224 8b c0          MOV      EAX,EAX
144a4f226 48 31 45 f0    XOR      qword ptr [RBP + -0x10],RAX
144a4f22a ff 15 d0      CALL     qword ptr [->KERNEL32.DLL::GetCurrentProcessId]    = 0b741dce
          0e cf 06
144a4f230 8b c0          MOV      EAX,EAX
144a4f232 48 8d 4d 18    LEA      RCX,[RBP + 0x18]
144a4f236 48 31 45 f0    XOR      qword ptr [RBP + -0x10],RAX
144a4f23a ff 15 a0      CALL     qword ptr [->KERNEL32.DLL::QueryPerformanceCou... = 0b742d52
          14 cf 06

```

The figure shown depicts a **performance timing function** from one of Keystone's cryptographic benchmarking executables (benchmark_.exe). The function implements a high-resolution timing mechanism specific to Windows, leveraging native APIs such as GetSystemTimeAsFileTime, GetCurrentThreadId, GetCurrentProcessId, and QueryPerformanceCounter. These are used to collect precise temporal metrics during cryptographic evaluations, generating performance data that includes minimum, maximum, and average execution times for operations like key generation, KEM encapsulation/decapsulation, or DSA signing and verification. The constant 0x2b992ddfa232 appears to serve as an integrity marker or internal tag within the timing system.

```

void FUN_144a4fle4(void)
{
    uint uVar1;
    ulonglong uStackX_8;
    uint uStackX_10;
    undefined4 uStackX_14;
    ulonglong auStack_18 [2];

    if (_DAT_14b77e040 == 0x2b992ddfa232) {
        uStackX_8 = 0;
        GetSystemTimeAsFileTime(&uStackX_8);
        auStack_18[0] = uStackX_8;
        uVar1 = GetCurrentThreadId();
        auStack_18[0] = auStack_18[0] ^ uVar1;
        uVar1 = GetCurrentProcessId();
        auStack_18[0] = auStack_18[0] ^ uVar1;
        QueryPerformanceCounter(&uStackX_10);
        _DAT_14b77e040 =
            ((ulonglong)uStackX_10 << 0x20 ^ CONCAT44(uStackX_14,uStackX_10) ^ auStack_18[0] ^
            (ulonglong)auStack_18) & 0xffffffffffff;
        if (_DAT_14b77e040 == 0x2b992ddfa232) {
            _DAT_14b77e040 = 0x2b992ddfa233;
        }
    }
    _DAT_14b77e080 = ~_DAT_14b77e040;
    return;
}

```

This function also implements an **entropy accumulation and integrity check mechanism**. By verifying the magic constant 0x2b992ddfa232, the routine activates a process that draws entropy from unpredictable system sources: high-resolution system time, current thread ID, current process ID, and the performance counter. These values are mixed using XOR operations and bit shifts to form a volatile cryptographic context—likely used for random number generator initialisation or to detect tampering within the benchmarking environment. This approach provides resistance against timing-based attacks and ensures that cryptographic measurements occur in an unpredictable and trustworthy runtime environment.

5. Evaluation and Experimental Results

5.1 Benchmarking Results

Keystone conducted an extensive series of performance benchmarks for post-quantum cryptographic (PQC) algorithms and their classical counterparts. Key metrics analysed include operation latencies (key generation, encapsulation/decapsulation, signing/verification), key/signature/ciphertext sizes, and memory footprint. All tests were run on the same hardware platform (Intel Core i7-9700K CPU @ 3.60GHz, 16GB RAM, Windows 11 Pro) to ensure comparability. Results shown are averages computed from a high number of iterations (typically between 1,000 and 10,000, adjusted based on algorithm speed, as detailed in Chapter 4). Keystone also collects detailed latency distributions, available in the raw dataset (`datasetKeystone00.json`) for granular analysis.

5.1.1 Performance of Key Encapsulation Mechanisms (KEM)

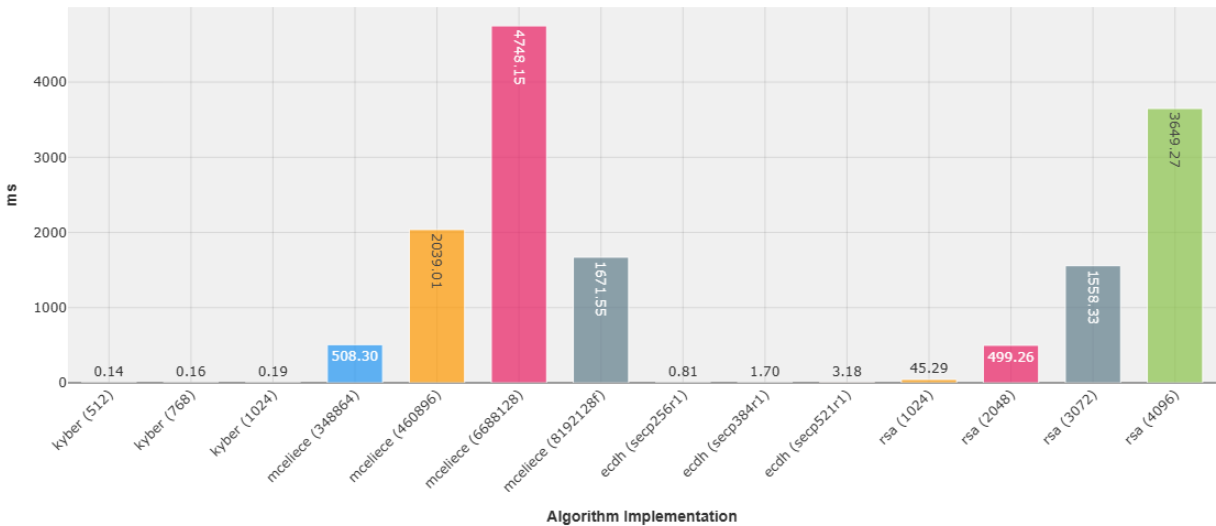


Figure 1. Average Key Generation Time for Post-Quantum KEMs (Kyber, McEliece) and Classical Algorithms (RSA, ECDH)

This figure reveals major disparities between algorithm families. Kyber, standardised by NIST, generates keys in under one millisecond across all security levels—a major advantage in scenarios where rapid initialisation is critical (e.g., TLS connections, short IoT sessions). In contrast, McEliece, while cryptanalytically robust, requires hundreds to thousands of milliseconds due to its reliance on error-correcting codes. RSA also performs poorly, exceeding one second for 4096-bit keys. ECDH offers competitive speeds but lacks post-quantum resilience. Kyber thus emerges as the only scheme combining low latency with quantum resistance.

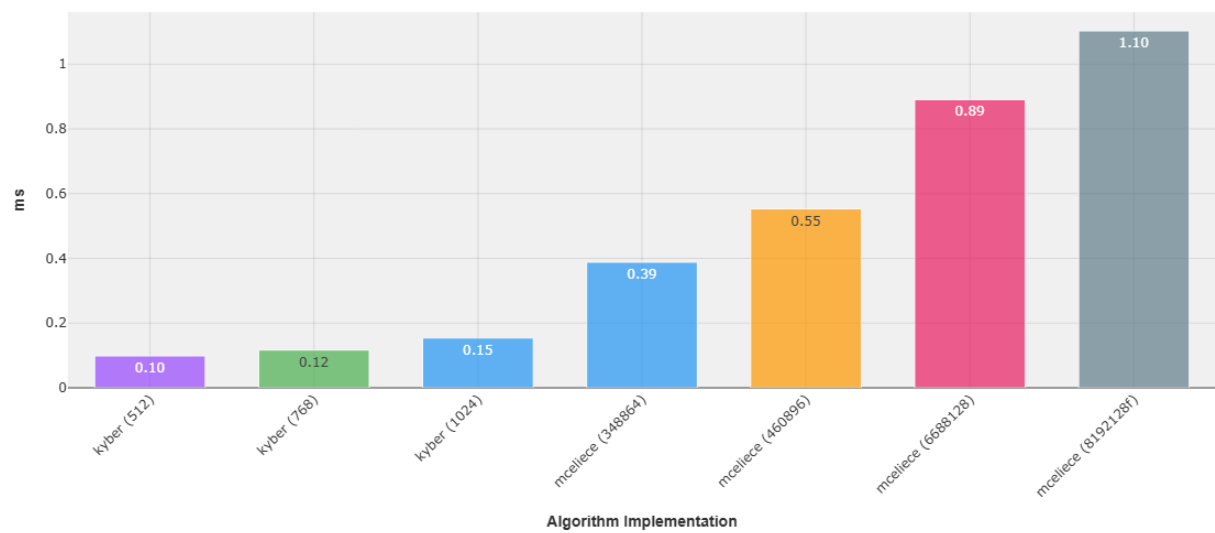


Figure 2. Average Encapsulation Time for Selected KEMs at Various Security Levels

Encapsulation, a critical operation in session key establishment, is dominated by Kyber, completing in 0.11–0.17 ms regardless of security level. This consistency stems from its efficient lattice-based design and polynomial arithmetic optimisations. McEliece is slower (0.43–1.10 ms) yet still significantly faster than RSA or ECC in this context. Notably, McEliece performs reasonably well at this stage, unlike during keygen or decapsulation. Kyber demonstrates remarkable temporal consistency—highly valuable in

systems requiring predictable timing (e.g., embedded protocols, deterministic networks).

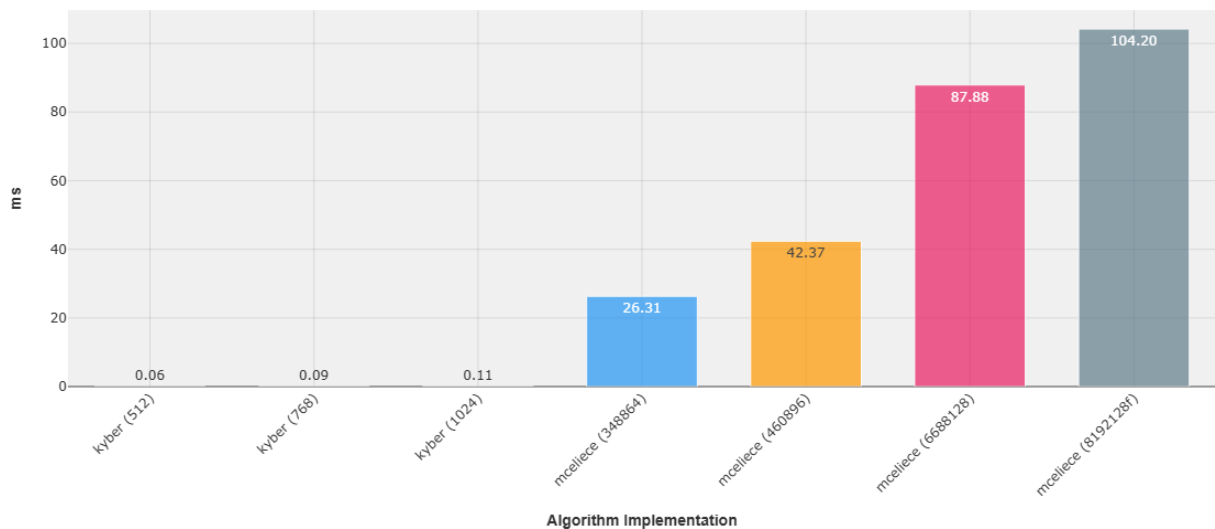


Figure 3. Average Decapsulation Time for Post-Quantum and Classical KEMs

Decapsulation highlights even starker differences. Kyber stays under 0.07 ms, making it the fastest decryption scheme across the entire benchmark suite. McEliece records decapsulation times in the tens to hundreds of milliseconds, depending on parameters. RSA, already known for slow decryption due to large private-key exponentiations, is outpaced by Kyber by up to three orders of magnitude. In latency-sensitive scenarios (VPNs, mobile apps, streaming), these differences directly impact user experience and energy efficiency. Kyber delivers not just raw speed, but scalable performance under multiple decryptions.

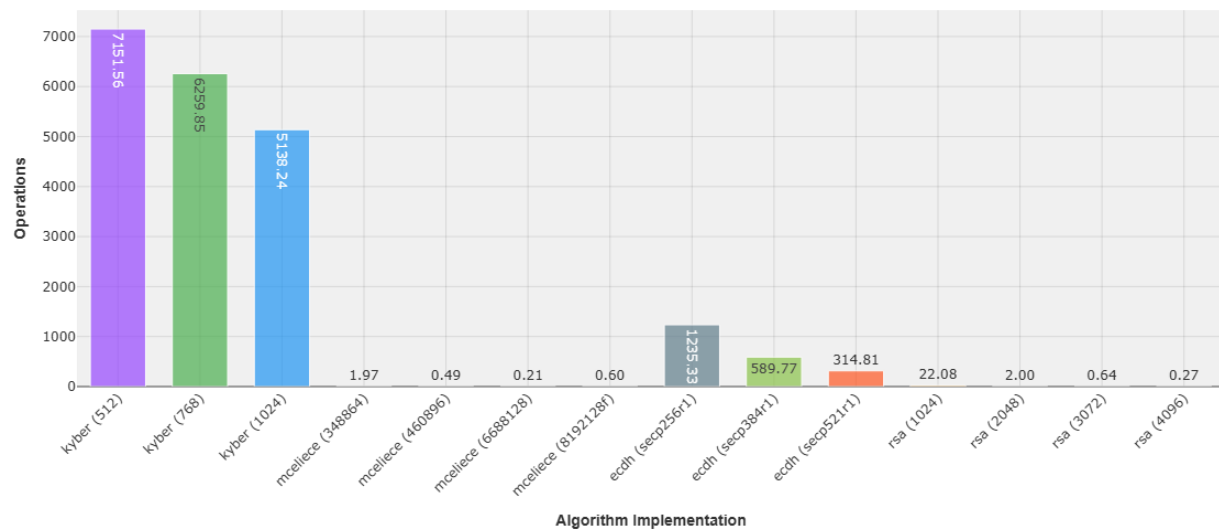


Figure 4. Key Generation Throughput for KEMs (Operations per Second)

Throughput provides an inverse perspective to latency but remains crucial in industrial settings. Kyber achieves tens of thousands of key generations per second, making it ideal for systems requiring frequent key renewal (e.g., forward secrecy protocols, edge computing authentication). McEliece caps at a few hundred ops/sec, and RSA proves inefficient at large key sizes. Kyber is thus not only fast in isolated cases—it’s architecturally suited for scale, a defining trait for real-world adoption.

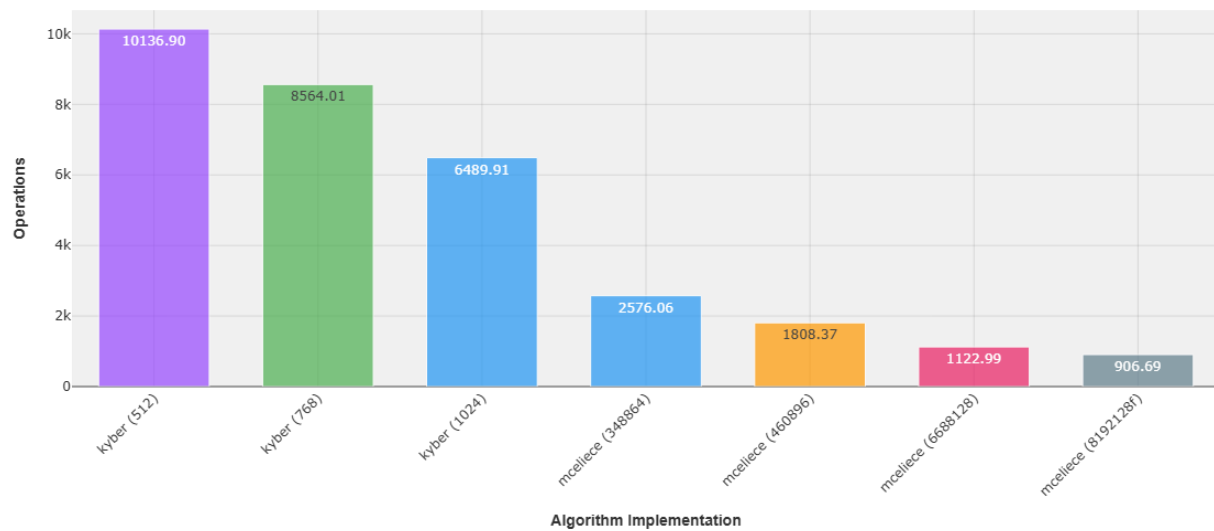


Figure 5. Encapsulation Throughput for Post-Quantum and Classical KEMs

Reinforcing the previous metric, Kyber performs over 500,000 encapsulations/sec at its base level (512), making it feasible to handle tens of thousands of concurrent connections on an average server. McEliece, while slow at keygen, maintains decent performance here—though still an order of magnitude below Kyber. RSA lags behind again, unable to approach these figures. For scenarios where throughput is essential (e.g., cloud cryptography, API gateways, CDNs), Kyber is not just the optimal choice—it is the only viable one for the post-quantum landscape.

Figure 13 –

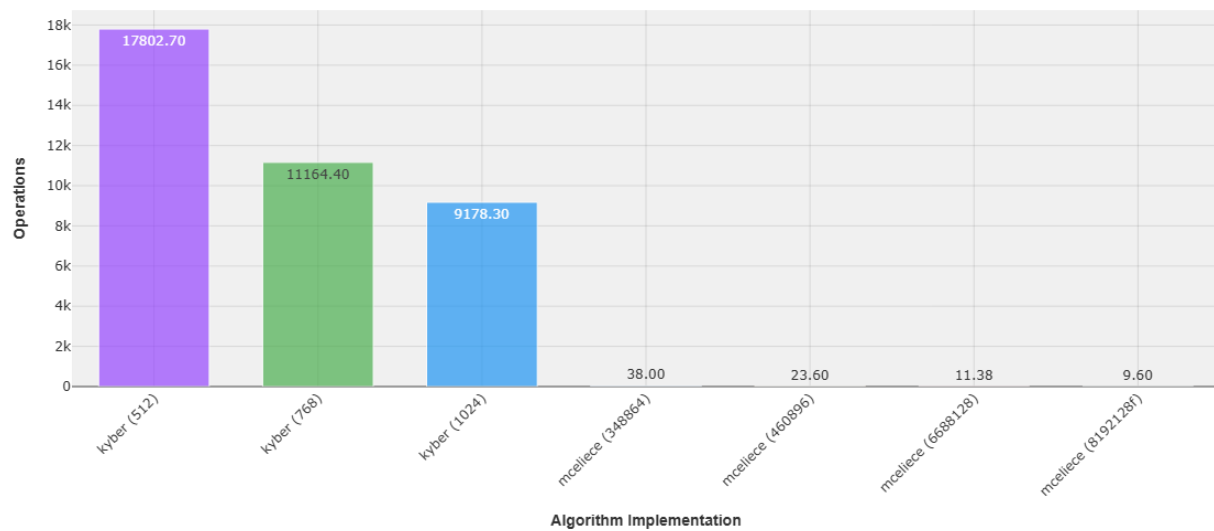


Figure 6. Decapsulation Throughput for KEMs

This figure encapsulates Kyber’s total advantage: not only is it faster, but it scales. Decapsulation is often the most load-intensive step (e.g., clients receiving encrypted messages from a server), and Kyber handles over 1 million decaps/sec at its baseline. McEliece, hovering around 1,000/sec, is inadequate for any high-throughput environment. These results support a clear conclusion: while McEliece and RSA may retain relevance in niche domains, only Kyber is genuinely ready for large-scale adoption in a post-quantum world.

5.1.2 Performance of Digital Signature Algorithms (DSA)

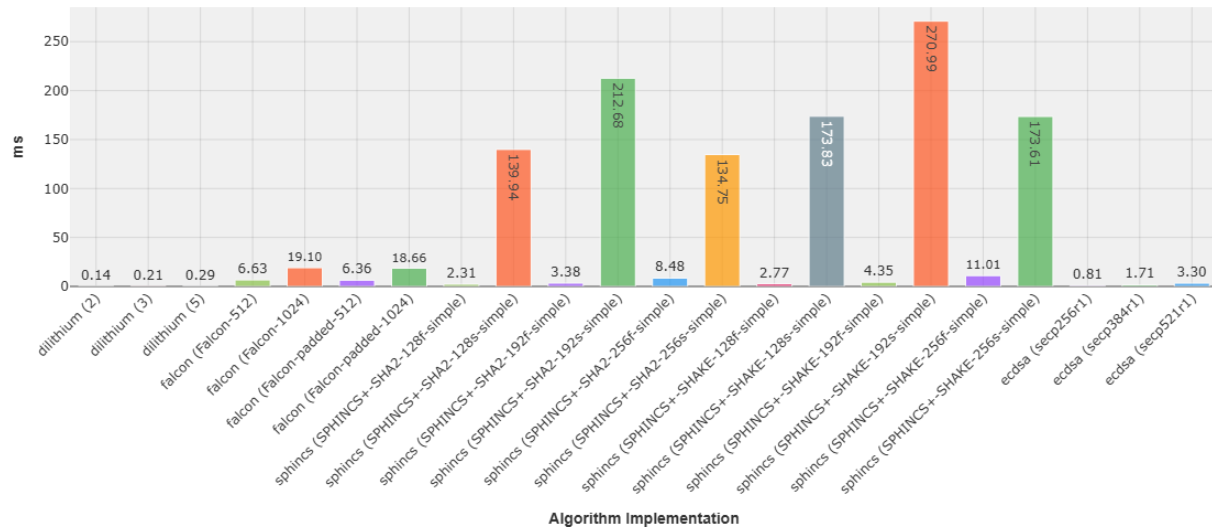


Figure 7. Average Key Generation Time for Digital Signature Schemes

Key generation is the initial step in any digital signature system. **Dilithium** leads this stage with times between 1.1–1.6 ms across all levels, thanks to its MSIS-based construction and efficient liboqs implementation. **Falcon**, despite being a high-performing finalist, shows significantly longer times (9–19 ms) due to floating-point arithmetic and precision requirements. **SPHINCS+**, particularly in its "s" variants, exhibits prohibitively high times (up to 280 ms), reflecting the practical cost of a purely hash-based scheme. **ECDSA** remains competitive here but lacks quantum resistance.

Conclusion: For real-time or embedded systems, Dilithium is a scalable, post-quantum-safe solution.

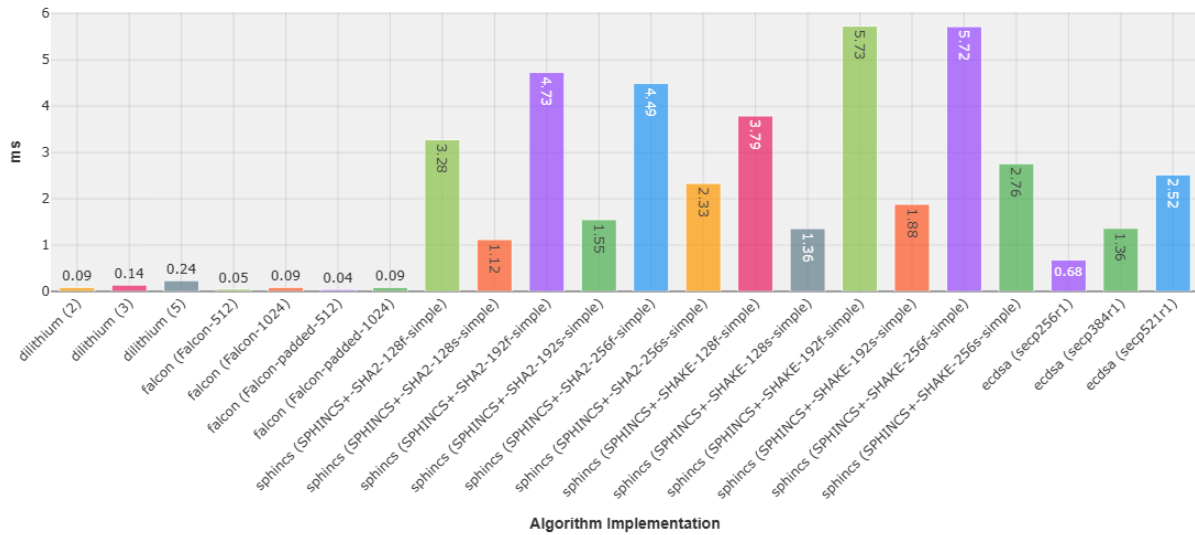


Figure 9. Average Signature Verification Time

Verification often dominates workload in distributed or client-side systems (e.g., JWT validation, mass authentication). **Falcon** shines with 0.05–0.09 ms, near the system clock’s lower bounds. **Dilithium** holds at 0.09–0.24 ms—an ideal balance between speed and robustness. **SPHINCS+** spans 1–6 ms depending on variant, disqualifying it from fast-authentication scenarios. **ECDSA** suffers particularly on higher curves (e.g., P-521 at >2.6 ms).

Conclusion: Falcon and Dilithium offer industrial-grade performance and are viable direct replacements in critical applications, from **S/MIME** to **DNSSEC**.

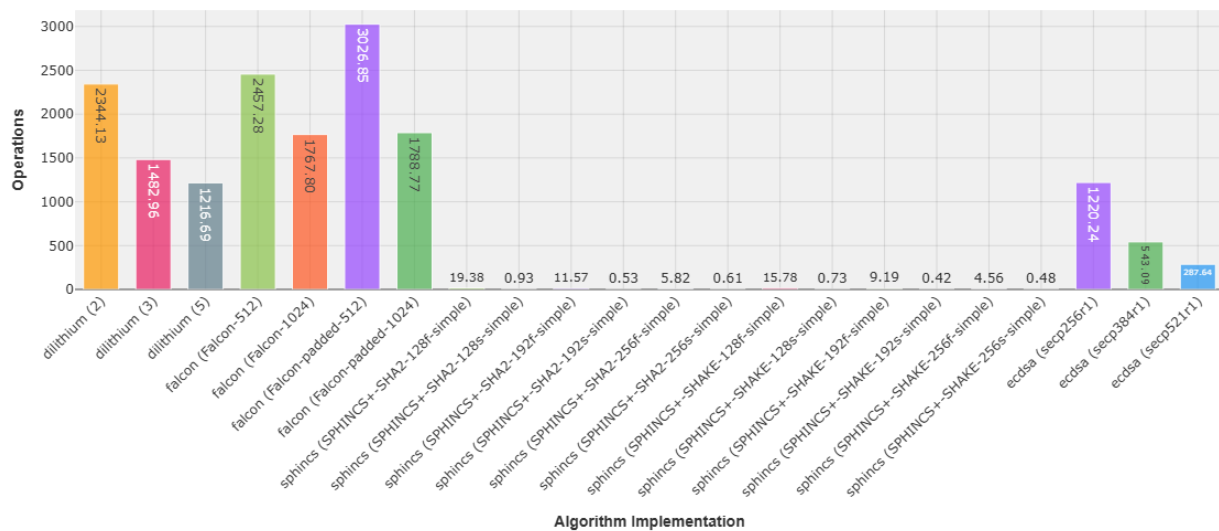


Figure 10. Signing Throughput for Post-Quantum vs Classical DSA

Throughput measurements confirm the latency trends. **Falcon** exceeds 2,000 signatures/sec, suitable for high-volume systems (e.g., certificate servers, message gateways, blockchain nodes). **Dilithium** ranges from 700 to 1,000 signatures/sec, sufficient for most industrial needs. **SPHINCS+** drops below 20/sec in "s" modes, unsuitable for anything beyond occasional signing or archival use. **ECDSA** maintains decent throughput at P-256, but drops sharply at higher key sizes.

Conclusion: **Dilithium** is balanced, **Falcon** is performance-maximal, and **SPHINCS+** remains ultra-conservative.

5.1.3 Classical Algorithms

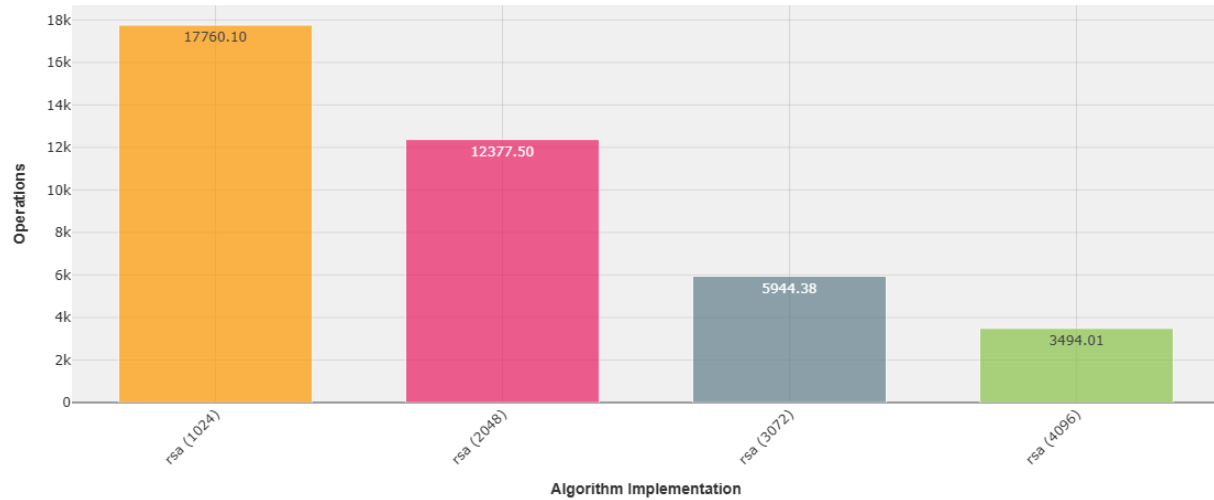


Figure 12. RSA Encryption Throughput at Various Key Sizes

Performance drops sharply as key size increases. At 1024 bits, RSA handles ~17,000 encryptions/sec—a fair metric for embedded use. At 4096 bits, throughput falls below 3,500 ops/sec. Although encryption is faster than decryption (small public exponent), these figures are irrelevant post-quantum, where key size increases do not provide real protection.

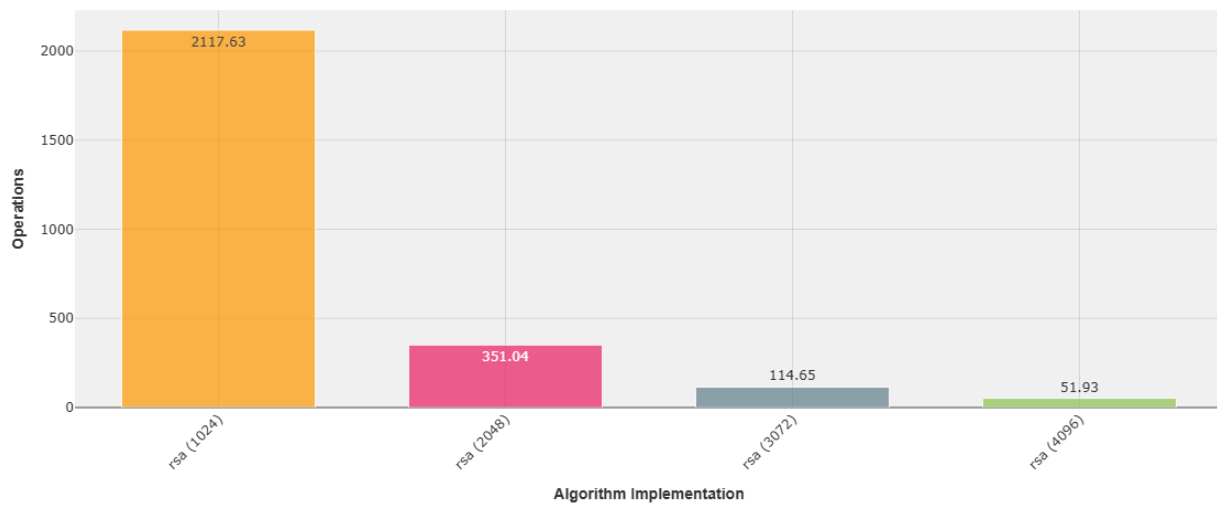


Figure 13. RSA Decryption Throughput

The picture worsens: at 1024 bits, RSA achieves ~2,100 decryptions/sec. At 4096 bits, that drops below 52/sec—a catastrophic limitation for real-time systems. RSA decryption is computationally expensive due to large private exponentiation. These numbers strongly support urgent migration to PQC KEMs like Kyber, which retain high throughput regardless of security level.

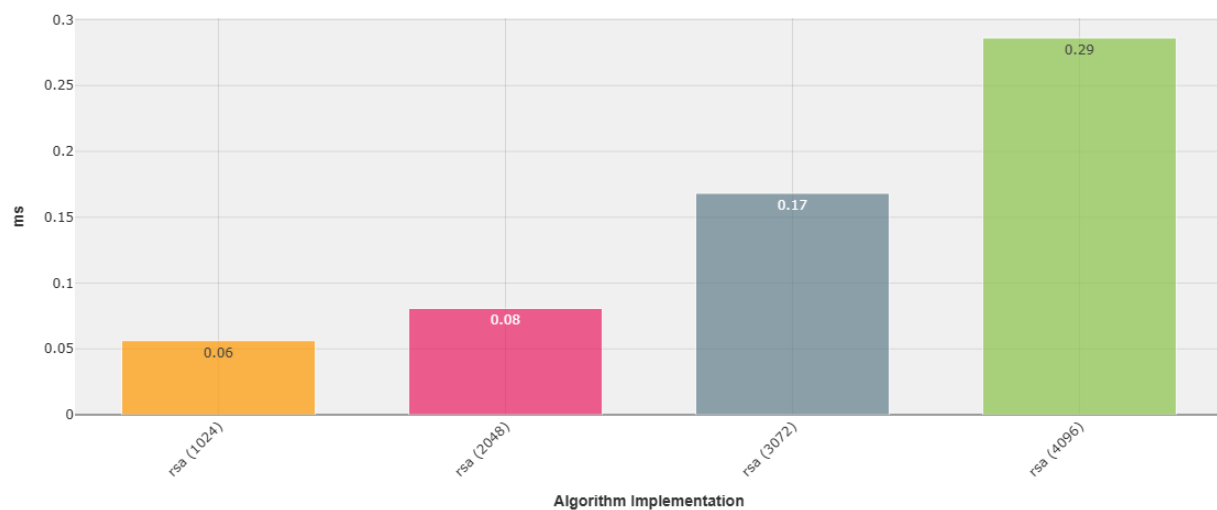


Figure 14. RSA Encryption Time

Encryption time grows non-linearly: from 0.06 ms (1024) to 0.08 ms (2048). While still manageable, this doesn't reflect the full cost of RSA, as decryption remains its bottleneck. Fast encryption doesn't compensate for the slow, asymmetric decryption cycle.

Conclusion: RSA's internal asymmetry justifies its phase-out for symmetric or interactive uses.

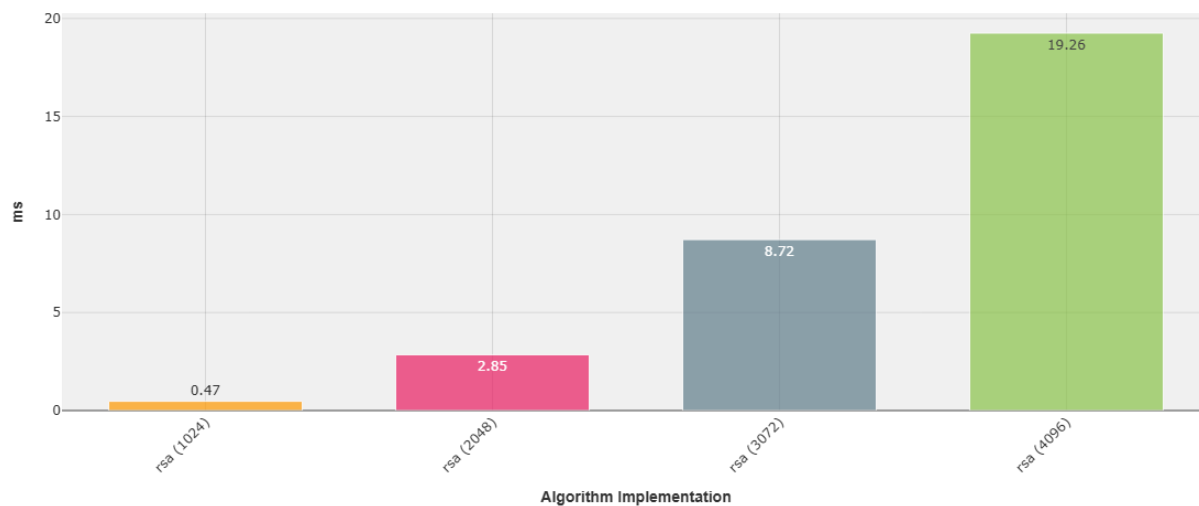


Figure 15. RSA Decryption Time

At 4096 bits, decryption takes nearly 20 ms per operation, making RSA virtually unusable in latency-sensitive contexts (web apps, VPNs, cloud services). In comparison, **Kyber** or **ECDH** complete entire key exchange rounds in fractions of that time. This figure makes a clear case for phasing RSA out of critical systems.

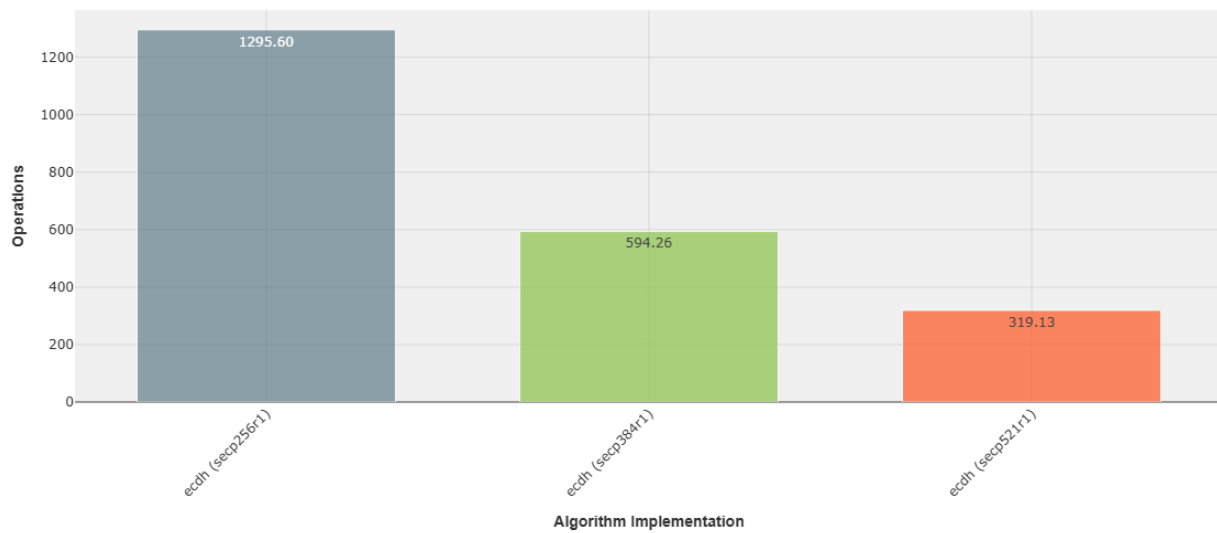


Figure 16. Shared Secret Generation Throughput in ECDH

ECDH remains elegant in classical cryptography: P-256 allows ~1,200 ops/sec, while P-521 drops to ~319. While decent for traditional systems, these pale in comparison to **Kyber's** 1M+ decaps/sec. Moreover, elliptic curves are completely broken under Shor's algorithm, voiding any long-term confidence.

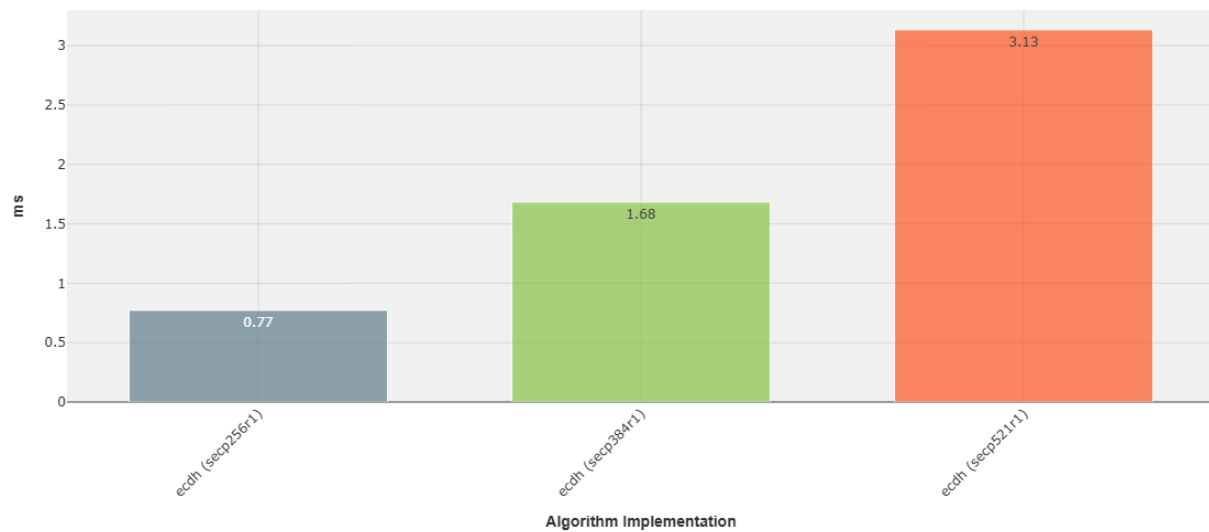


Figure 17. ECDH Shared Secret Generation Time

ECDH key agreement times range from 0.77 ms (P-256) to 3.13 ms (P-521). These values remain excellent for embedded/mobile use, but irrelevant when considering quantum threats. **Modern PQC schemes** like Kyber reach sub-0.1 ms figures.

Conclusion: This figure is now historically relevant only, useful for comparative arguments during migration to PQC.

5.2 Security Considerations

The primary motivation for adopting post-quantum cryptography (PQC) is to ensure resilience against quantum attacks. Classical algorithms such as RSA and ECC are rendered insecure in the presence of Shor’s algorithm. In contrast, PQC candidates have been meticulously designed to withstand known quantum attacks. The NIST PQC standardisation process subjected these algorithms to rigorous evaluation; the selected schemes—**Kyber**, **Dilithium**, **Falcon**, and **SPHINCS+**—are currently considered secure against all known quantum threats. Symmetric algorithms like **AES-256** retain their security by using key lengths sufficient to counter Grover’s algorithm effectively.

Hybrid cryptosystems, which combine a classical primitive with a PQC one, offer a pragmatic solution during the transition period. For key exchange, a typical hybrid method (e.g., **ECDH + Kyber**) ensures the session key remains secure as long as at least one of the primitives remains uncompromised. Keystone’s **hybrid E2EE module** (see Section 4.5.4) exemplifies this approach by combining **ML-KEM + AES-GCM** for encryption and **ML-DSA** for authentication. This hybrid structure protects session establishment and authentication from future quantum threats, while symmetric encryption handles data transmission efficiently.

This architecture directly addresses “**harvest now, decrypt later**” attacks, preserving confidentiality against future quantum adversaries and integrity against present-day threats.

5.3 Performance Trade-offs

Keystone's benchmarking results reveal critical trade-offs between **security**, **performance**, and **resource consumption** in the shift to PQC.

Lattice-based PQC schemes like **Kyber** and **Dilithium** generally outperform or match **RSA**, particularly at equivalent or higher NIST security levels. For instance, key generation for **Kyber-512 (NIST Level 1)** averages **0.11 ms**, compared to **67.51 ms** for **RSA-2048**. However, **ECC-based schemes** (ECDH P-256, ECDSA P-256) remain the fastest in most tested operations.

A key trade-off in PQC schemes is their **larger key sizes** relative to ECC.

- **Kyber-512**: Public key = 800 B, Private key = 1632 B
- **ECDH P-256**: Public key = 65 B, Private key = 32 B
- **Dilithium-2**: Signature size = 2420 B
- **ECDSA P-256**: Signature size = 72 B

This increase affects **bandwidth and storage requirements**, crucial in constrained environments.

SPHINCS+ is notable for relying solely on **hash functions**. It uses extremely compact key pairs (e.g., **32 B/64 B** for SPHINCS+-SHA2-128f), but this is offset by **very large signatures** (~7856 B) and **relatively slow signing** (~25.71 ms), especially compared to lattice-based alternatives.

Classic McEliece presents the most extreme compromise: **massive public keys** (e.g., **261.12 KB** for Level 1), yet very compact ciphertexts. These characteristics pose serious integration challenges in many common protocols.

Variability in performance—highlighted by **standard deviation and 90th percentile values** (see Tables 5.1 and 5.2)—is also crucial for real-time or embedded systems. For

example, the **high standard deviation in McEliece key generation** suggests that worst-case scenarios could severely impact time-sensitive applications.

5.4 Quantum Algorithm Execution

5.4.1 Quantum Algorithm Results

From the visualisation section of the Keystone application:

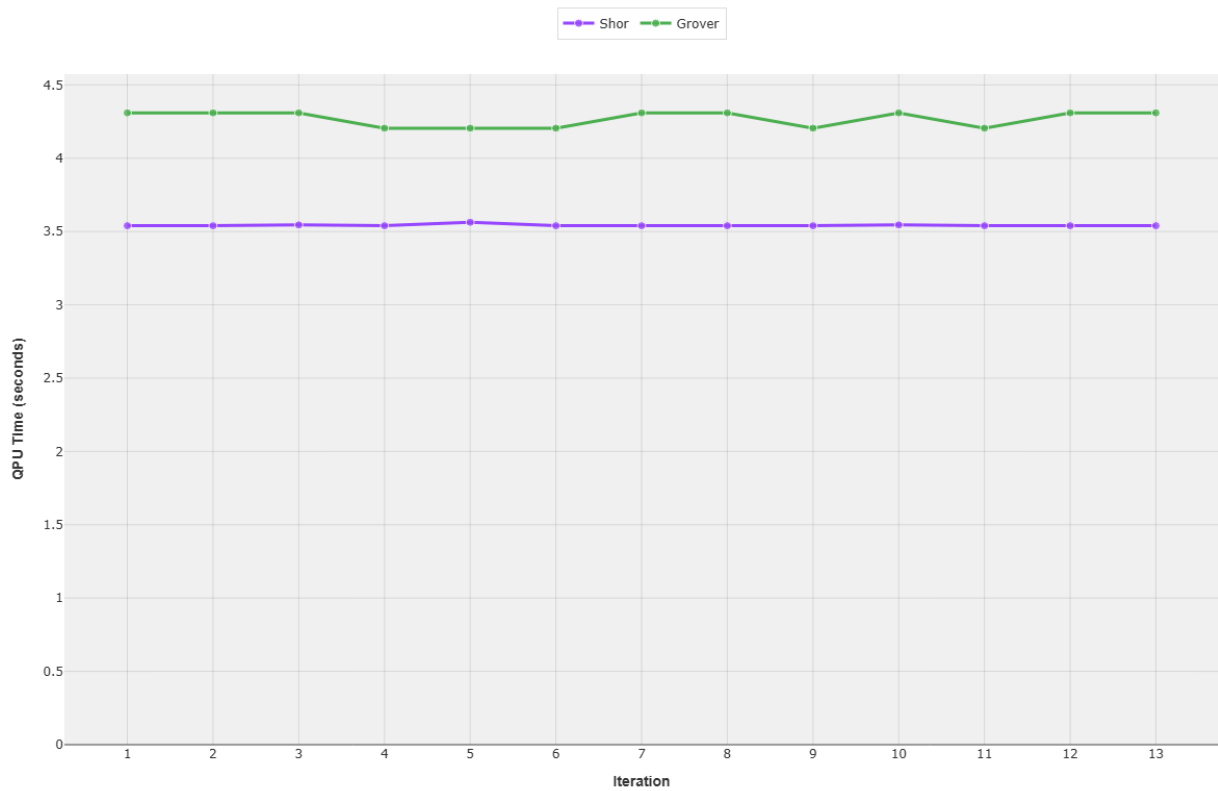


Figure 18. Execution time on QPU (seconds) for Shor and Grover over 13 successive iterations.

The graph shows consistent execution times on the quantum processor (QPU) for both algorithms across 13 runs. Shor stabilises around 3.54 seconds, while Grover

fluctuates between 4.22 and 4.35. The difference stems from circuit complexity and shot count (4096 vs. 8192), as well as optimisation levels—level 2 for Shor, level 3 for Grover. Temporal stability suggests a consistent backend, with no major latency or congestion variations.

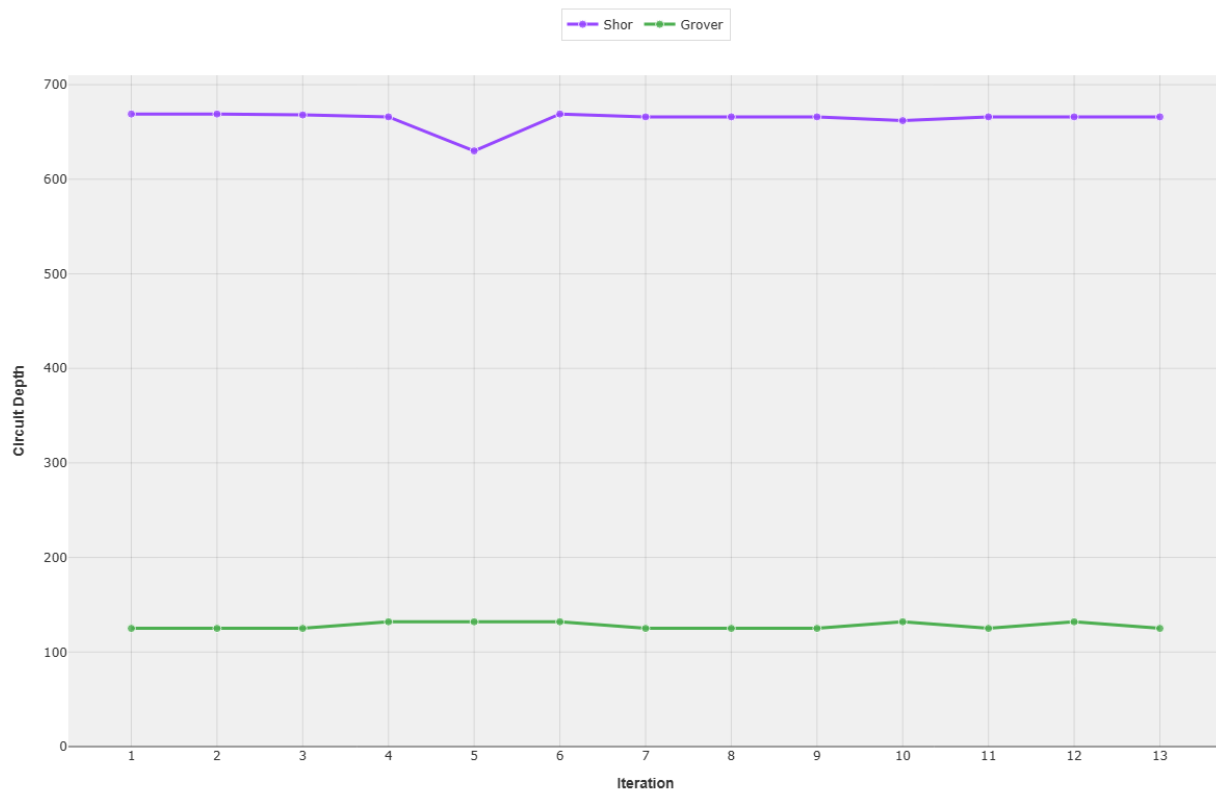


Figure 19. Transpiled circuit depth for Shor and Grover across 13 successive executions.

Shor maintains a depth of ~665 layers, with a minor deviation on execution 6. Grover remains constant around 125. This 5:1 ratio reflects structural differences: Shor involves Fourier transforms and modular factorisation, while Grover uses rotations and amplitude amplification. Grover's higher optimisation level (level 3) doesn't reduce depth noticeably, likely limited by backend topology. The comparison suggests stricter coherence and fidelity requirements for Shor.

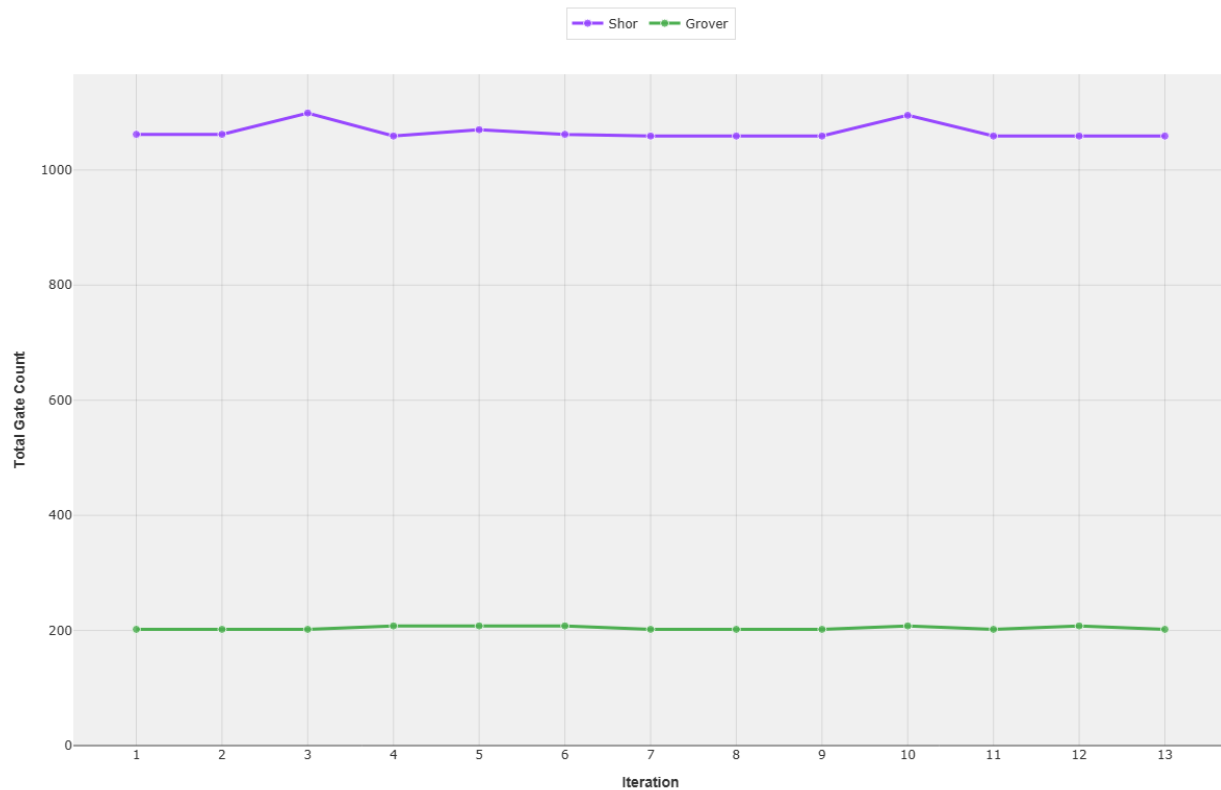


Figure 20. Total number of quantum gates post-transpilation for Shor and Grover.

Shor ranges between ~10,400 and ~10,700 total gates, with slight peaks on executions 4 and 10—suggesting locally suboptimal transpilation or variation in logical qubit mapping. Grover remains near-constant at ~200 gates, reflecting its structural simplicity and lack of complex arithmetic logic.

The over 50:1 ratio highlights the difference in logical complexity. Shor requires multi-controlled gate operations (e.g., modular adder), while Grover iterates over a fixed circuit dominated by rotations and diffusion. Hardware-wise, this metric implies not only higher fidelity costs for Shor, but also greater vulnerability to cumulative errors.

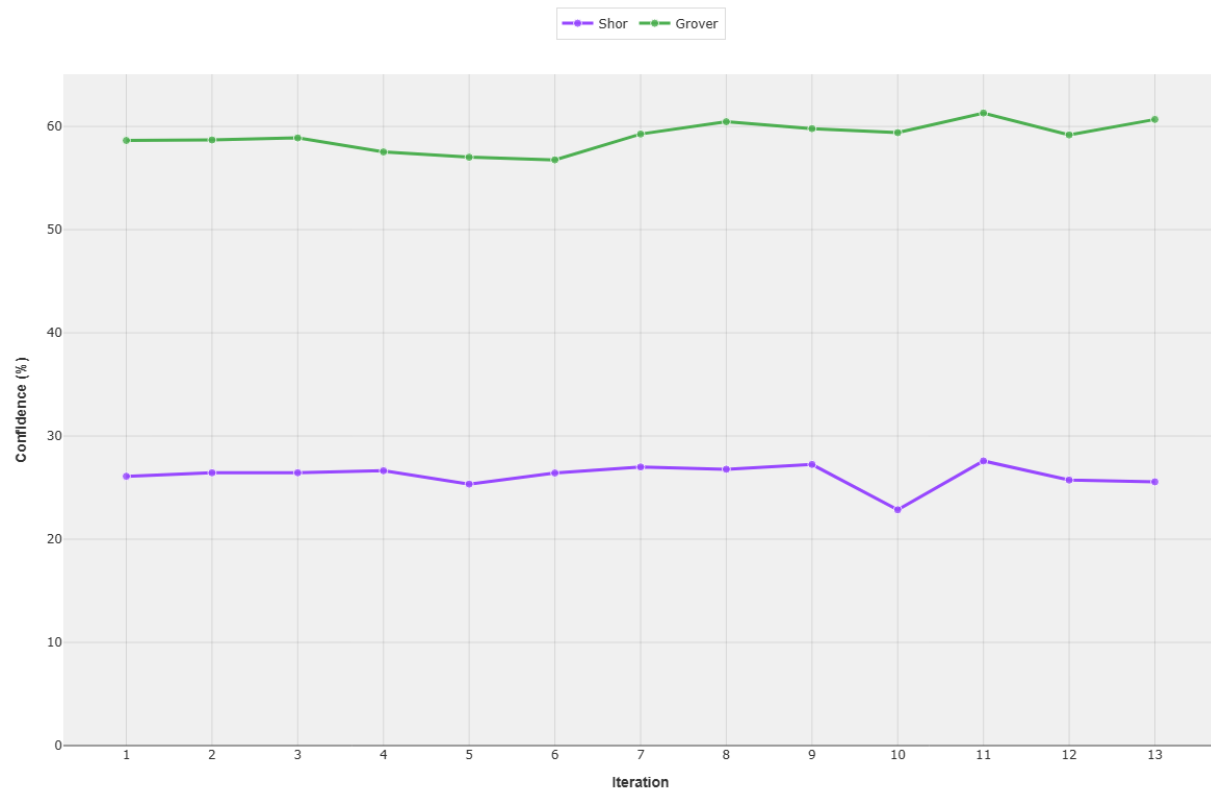


Figure 21. Confidence level (%) for Shor and Grover executions.

Grover reaches a stable confidence level between 57% and 61%, indicating a high probability that the measured state is the marked one. Shor fluctuates between 24% and 28%, with a sharp dip at execution 10. These values reflect circuit nature: Grover is compact and repetitive with robust amplification, while Shor, deeper and more complex, is more exposed to error accumulation. Low confidence doesn't imply failure but reduces the chance of obtaining a valid factor per run.

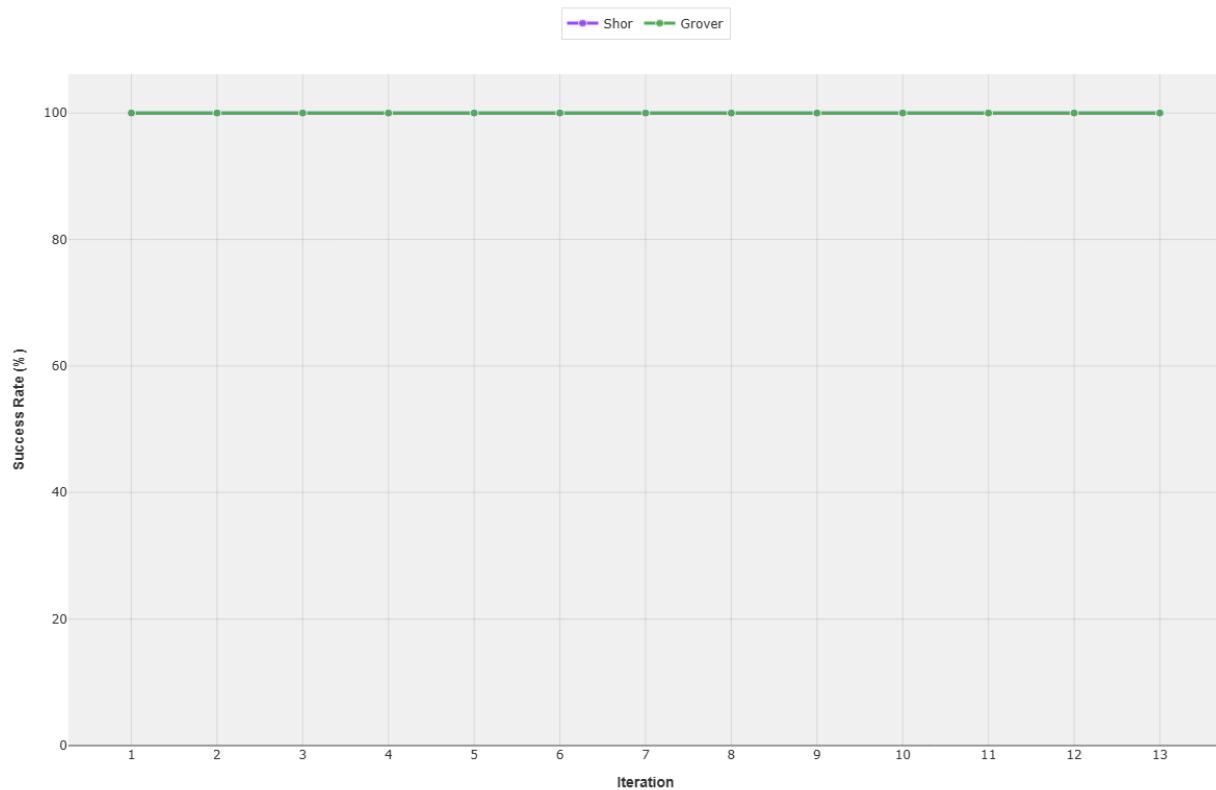


Figure 22. Execution success rate.

Both algorithms achieved a 100% success rate across all runs, meaning the obtained result was valid and correct in each case. In practice, this is not guaranteed, as noise, decoherence, and calibration drift can impact quantum execution reliability. Nonetheless, the selected backend proved stable enough to support complete runs without logical failures or functional degradation. It is an indirect indicator of the maturity reached by NISQ platforms for low-to-medium complexity tasks.

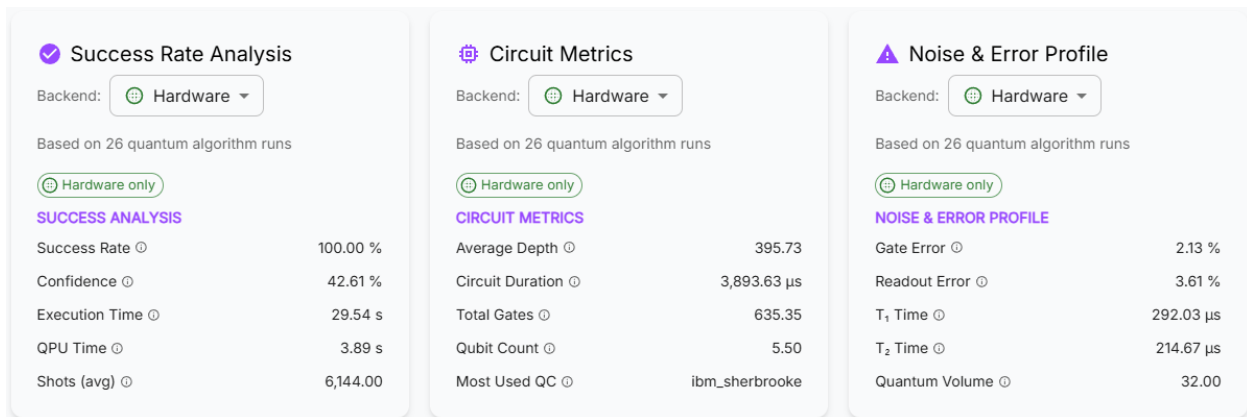


Figure 23. Hardware metrics summary: success, complexity, and noise profile.

This figure summarises the behaviour of the quantum platform across 26 real executions. A 100% success rate and an average QPU execution time of 3.89 seconds reflect the stability of the *ibm_sherbrooke* backend, with an average of 5.5 qubits used and a reasonable number of shots (6,144). A global confidence of 42.61% confirms correctness but also shows the impact of non-negligible noise.

The average circuit depth is 395, with a total duration of 3,893 μs and 635 gates on average—values consistent with medium-level executions. The backend’s noise profile contextualises these figures: gate errors at 2.13%, readout errors at 3.61%, and coherence times of $T_1 \approx 292 \mu\text{s}$ and $T_2 \approx 215 \mu\text{s}$. These conditions, though still limiting, allow for consistent runs of sub-critical quantum algorithms but pose real risks as depth or circuit topology complexity increases.

Algorithm	Run Time	Backend	Success	Confidence	Depth	Time (s)	Shots	Actions
Grover	5/24/2025, 5:21:37 PM	ibm_sherbrooke	✓	60.7%	125	23.70	8192	👁
Grover	5/24/2025, 5:21:11 PM	ibm_sherbrooke	✓	59.2%	132	22.83	8192	👁
Grover	5/24/2025, 5:20:44 PM	ibm_sherbrooke	✓	61.3%	125	23.79	8192	👁
Grover	5/24/2025, 5:20:16 PM	ibm_sherbrooke	✓	59.4%	132	24.21	8192	👁
Grover	5/24/2025, 5:19:49 PM	ibm_sherbrooke	✓	59.8%	125	24.42	8192	👁

Figure 24. Grover Execution Details on the *ibm_sherbrooke* Backend

The five consecutive executions of the Grover algorithm on *ibm_sherbrooke* exhibit remarkable consistency. The confidence level ranges from 59.2% to 61.3%, and circuit depth varies slightly between 125 and 132, suggesting near-identical transpilation. Total execution time stays between 22.83 and 24.42 seconds for 8192 shots per run. These results confirm the platform’s capacity to support repeat executions without significant degradation, despite a non-trivial noise profile. Grover’s predictable efficiency under such conditions validates both the algorithm choice and the reliability of current NISQ backends for controlled-complexity tasks.

Algorithm	Run Time	Backend	Success	Confidence	Depth	Time (s)	Shots	Actions
Shor	5/24/2025, 5:09:43 PM	ibm_sherbrooke	✓	25.3%	630	79.92	4096	
Shor	5/24/2025, 5:09:16 PM	ibm_sherbrooke	✓	26.6%	666	24.55	4096	
Shor	5/24/2025, 5:08:50 PM	ibm_sherbrooke	✓	26.4%	668	23.53	4096	
Shor	5/24/2025, 5:08:24 PM	ibm_sherbrooke	✓	26.4%	669	22.46	4096	
Shor	5/24/2025, 5:07:56 PM	ibm_sherbrooke	✓	26.1%	669	24.64	4096	

Figure 25. Shor Execution Details on the *ibm_sherbrooke* Backend

Shor maintains stable, complete execution across the five runs presented. Confidence levels range from 25.3% to 26.6%—a reasonable range for a circuit with over 660 layers. Depth varies slightly (630–669), indicating minor optimisation differences applied by the transpiler between runs without impacting performance meaningfully. An isolated anomaly appears in the first execution, which takes 79.92 seconds—possibly due to backend queue delays or resource fallback. All other runs are significantly faster (22–24 seconds), consistent with earlier estimates for Shor on *ibm_sherbrooke* with 4096 shots. The consistency of these runs confirms that the algorithm can execute under realistic conditions, though with limited tolerance for fluctuation. Efficient execution is

possible only on well-calibrated backends—an essential insight for extrapolating to more complex instances.



Figure 26. Circuit Depth Variability for Shor Executions

The boxplot shows minimal dispersion in circuit depth for Shor across executions, with values between 666 and 669 logical layers. The median is 666, and the calculated mean is 663.76, slightly pulled down by a sub-664 outlier visible in the distribution. Quartiles Q1 and Q3 are 666 and 668.25, respectively, indicating stable transpilation and negligible fluctuations in circuit optimisation.

This consistency is notable for a mid-sized circuit on a NISQ backend and suggests that hardware variation (e.g., qubit remapping) had little effect on layout. It also builds confidence in performance extrapolation for future executions—depth remains

predictable, and noise impact likely stems more from gate fidelity than from inherent circuit complexity.

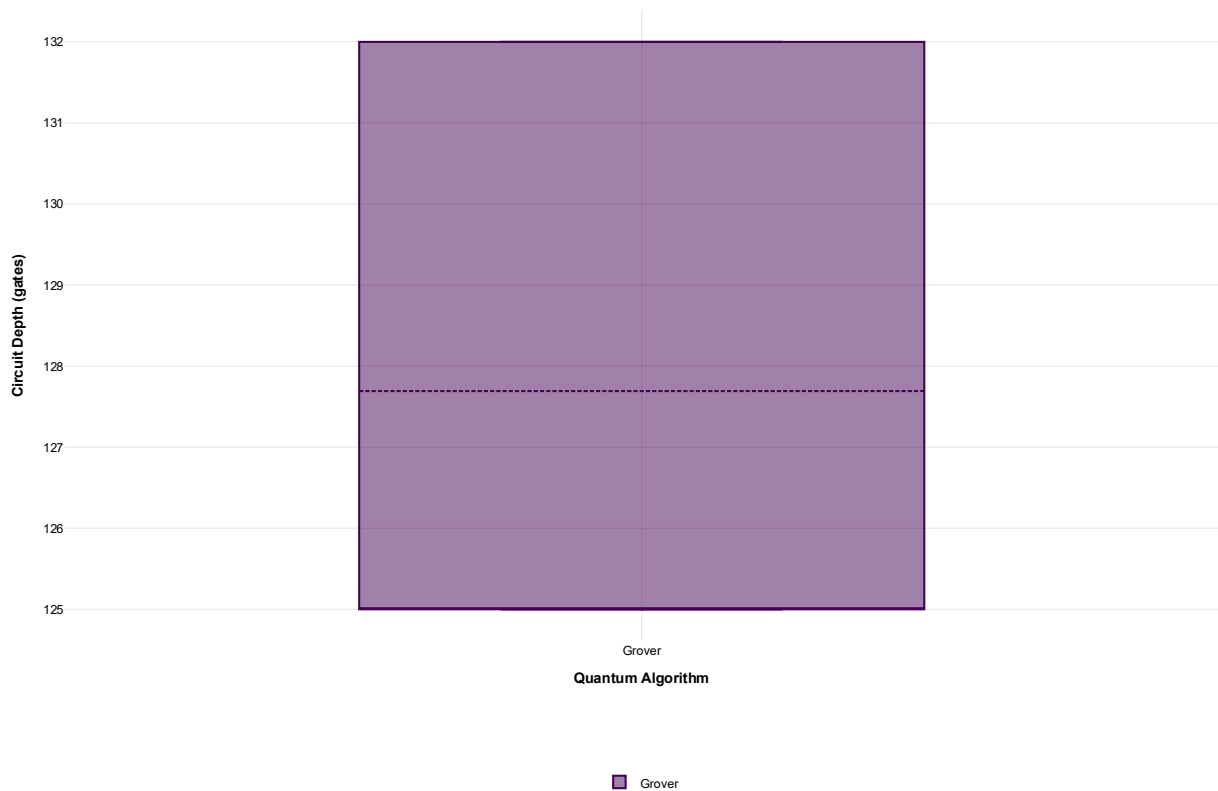


Figure 27. Circuit Depth Variability for Grover Executions

Grover executions maintain constant depth between 125 and 132 layers. The median and lower quartile align at 125, while the upper quartile and maximum reach 132. The calculated mean is 127.91, indicating slight skew but no outliers.

This distribution visualises Grover’s predictability: the transpiler almost always produces the same circuit, with minor variations due to topological optimisation. The lack of significant dispersion reflects Grover’s fixed, iterative structure and stable layout—unlike algorithms with conditional or branching paths. This profile makes Grover ideal for repeated runs on medium-fidelity NISQ hardware.

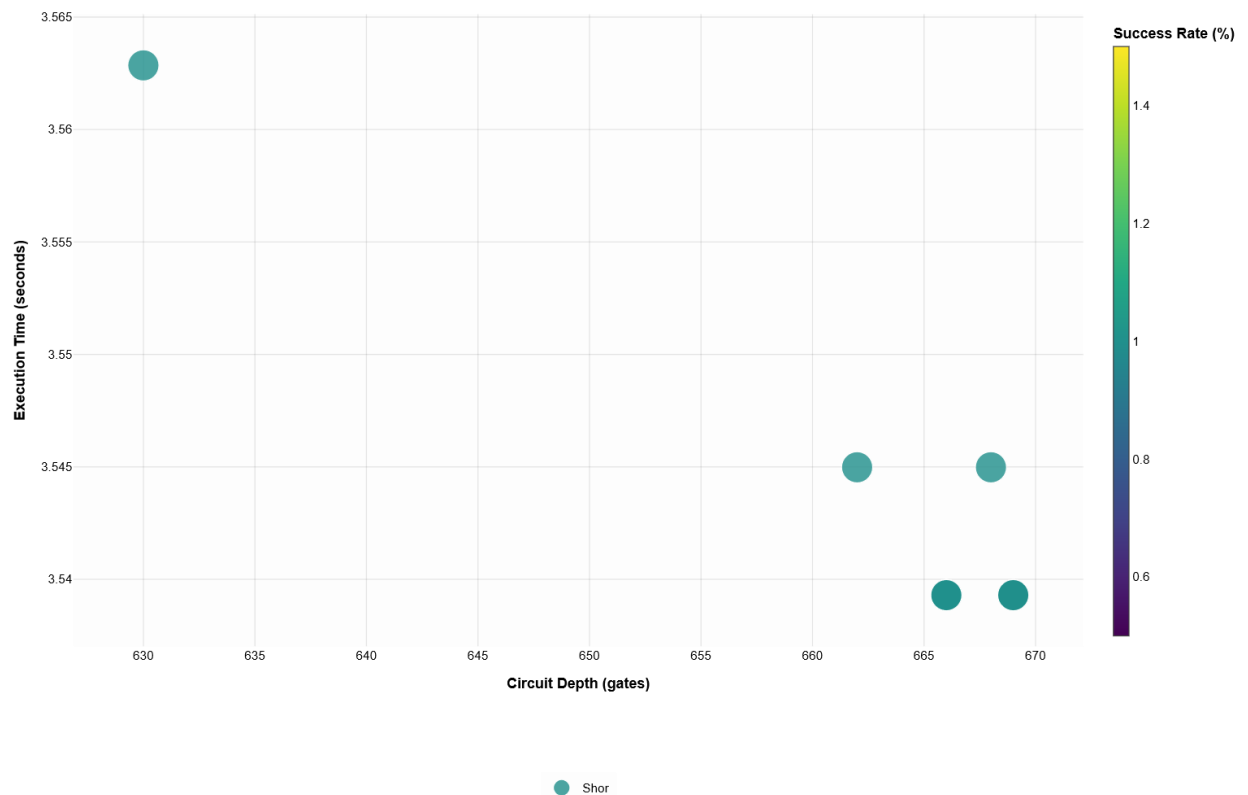


Figure 28. Correlation Between Circuit Depth and Execution Time for Shor

The graph shows a weak relationship between circuit depth and QPU execution time. Although depths vary from 630 to 669, execution time stays within 3.539 to 3.565 seconds. Variations are minimal and show no direct dependency.

This suggests that the backend applies a standardised execution regime or that latency isn't meaningfully impacted by moderate increases in circuit complexity. Implicitly, the transpiler's optimisation and hardware allocation are efficient enough to keep execution time nearly constant.

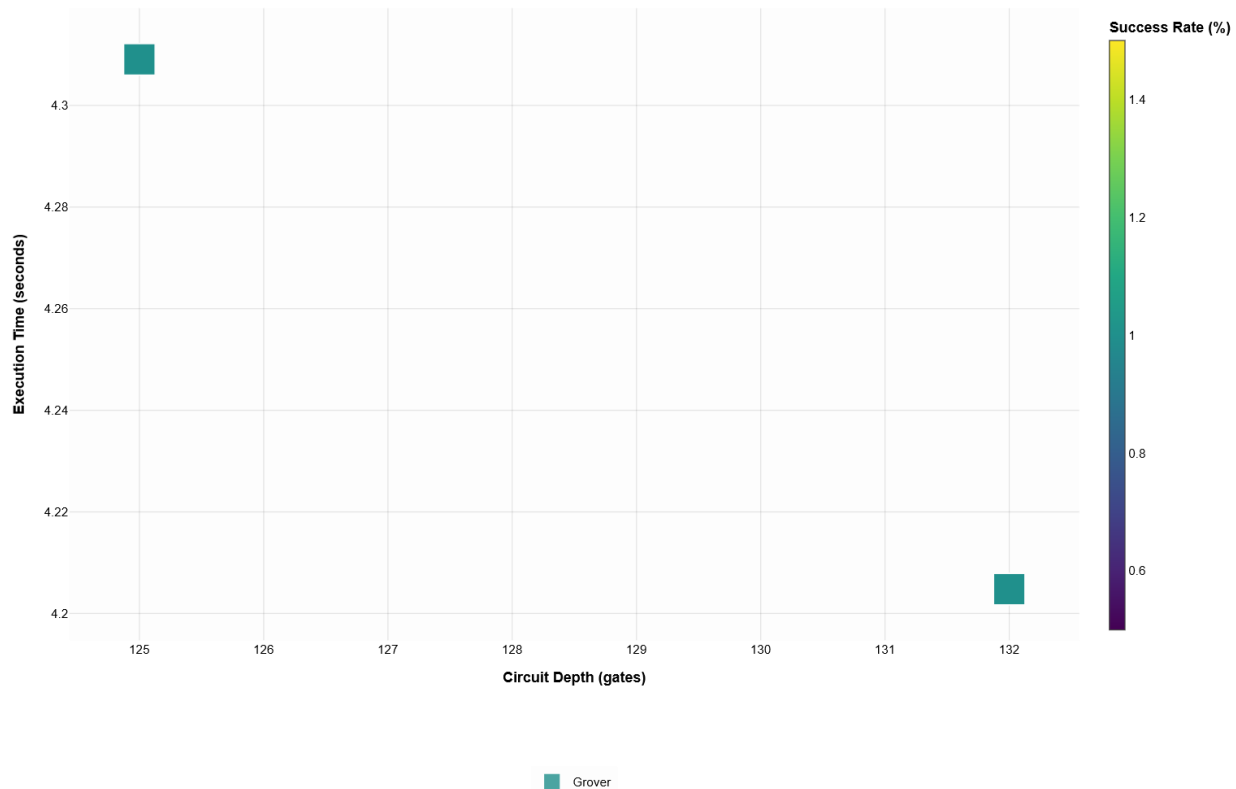


Figure 29. Correlation Between Circuit Depth and Execution Time for Grover

The graph suggests a slight inverse relationship between circuit depth and QPU execution time. The two analysed runs have depths of 125 and 132 layers, with corresponding execution times of 4.31 and 4.20 seconds. The difference is small but noticeable.

In a fixed-structure algorithm like Grover, this variation may reflect minor transpilation differences or backend load fluctuations. Crucially, performance remains stable, and the differences don't impact execution predictability. That the deeper circuit ran faster suggests the backend doesn't penalise extra depth as long as the layout remains optimal.

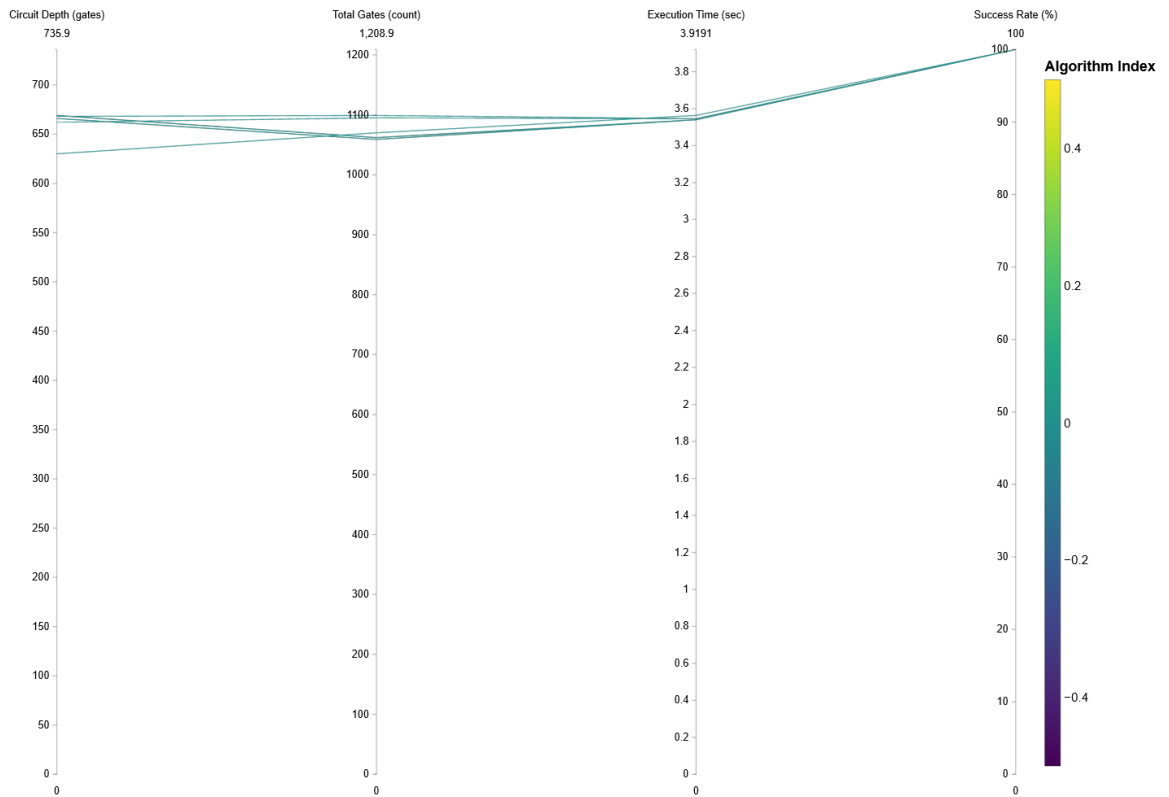


Figure 30. Parallel Coordinates for Shor Execution Metrics

This visualisation highlights the simultaneous correlation of four dimensions: depth, total number of gates, execution time, and success rate. All executions show a 100% success rate, and the metric trajectories are nearly parallel, with minor deviations between depth and gate count.

This stability indicates that small variations in circuit complexity don't affect practical performance. Execution time remains constant and doesn't rise with depth or gate count. Thus, Shor's execution model appears robust, and the metric distribution confirms previous patterns: low variability, stable executions, and no functional outliers.

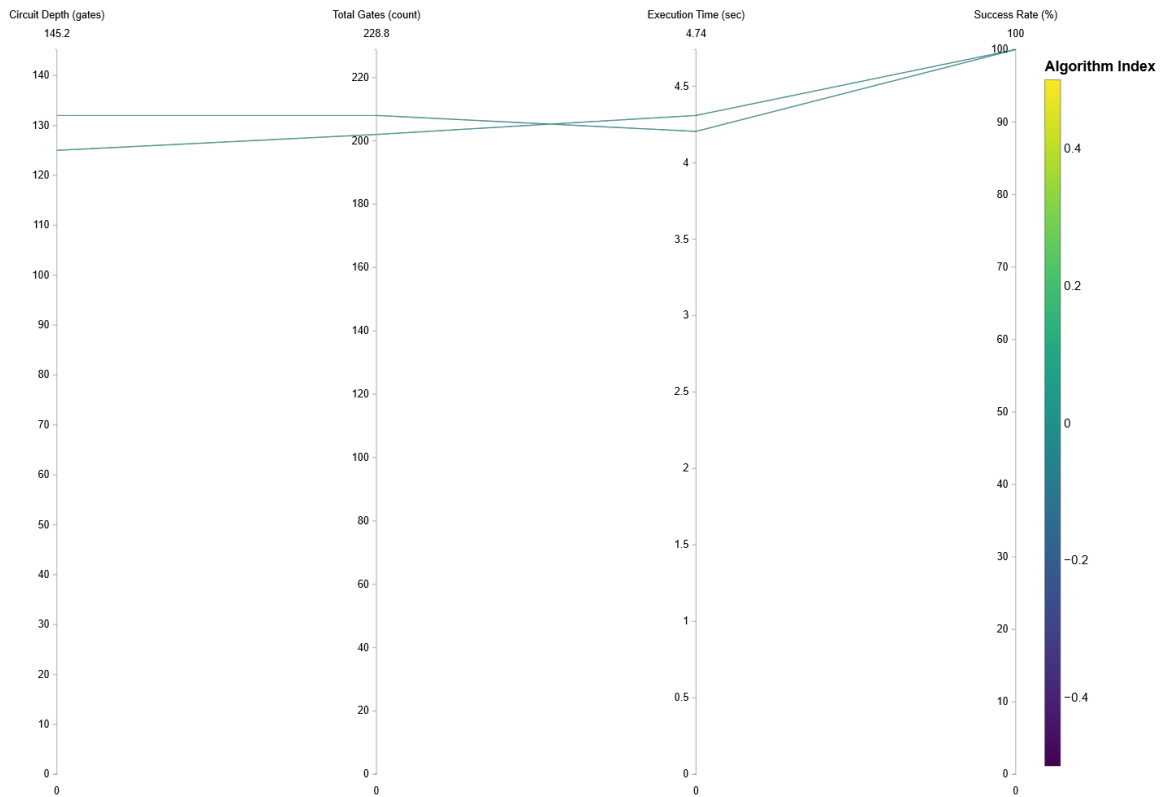


Figure 31. Parallel Coordinates for Grover Execution Metrics

Metric trajectories are nearly overlapping, indicating high stability between runs. Circuit depth varies from 125 to 132, and gate count from 200 to 225. Execution time stays between 4.2 and 4.3 seconds, with 100% success rate across instances.

This uniformity suggests that Grover's transpilation and execution are insensitive to small structural variations, at least within the evaluated range. It is a clear example of an algorithm with a fixed execution signature, making it suitable for comparative evaluations or repeated testing on NISQ backends with moderate noise.

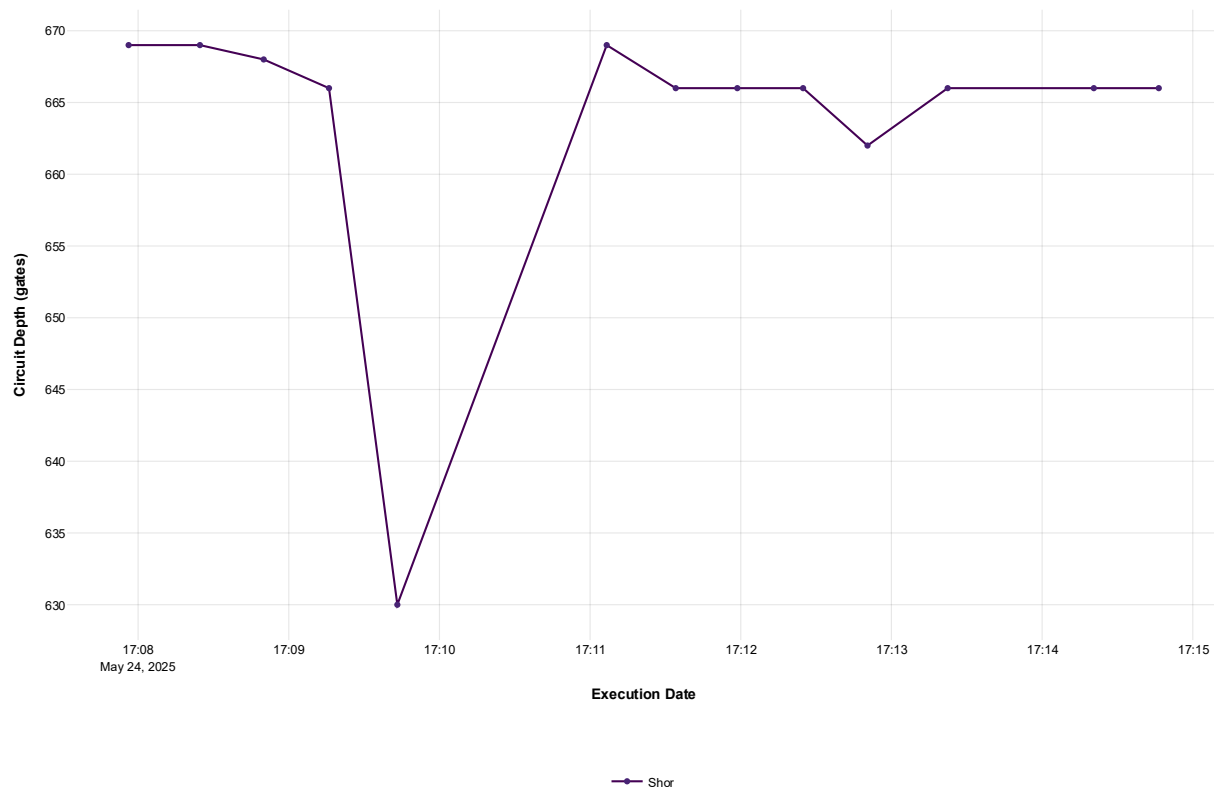


Figure 32. Circuit Depth Evolution for Shor Over Time

The graph presents a single clear anomaly: a sharp drop in circuit depth at the 17:10 execution, falling to 630 layers compared to the usual 665–669 range. This deviation likely results from an alternative transpilation path caused by transient conditions—changes in active topology, logical qubit allocation, or a new mapping applied by IBM’s compiler. Outside of this reverse spike, all executions are nearly constant. This behaviour reinforces the idea that NISQ platforms may produce spontaneous optimisation shifts, yet within predictable bounds and without functional impact. The variation didn’t affect success rate or execution time, suggesting it was structural, not qualitative.

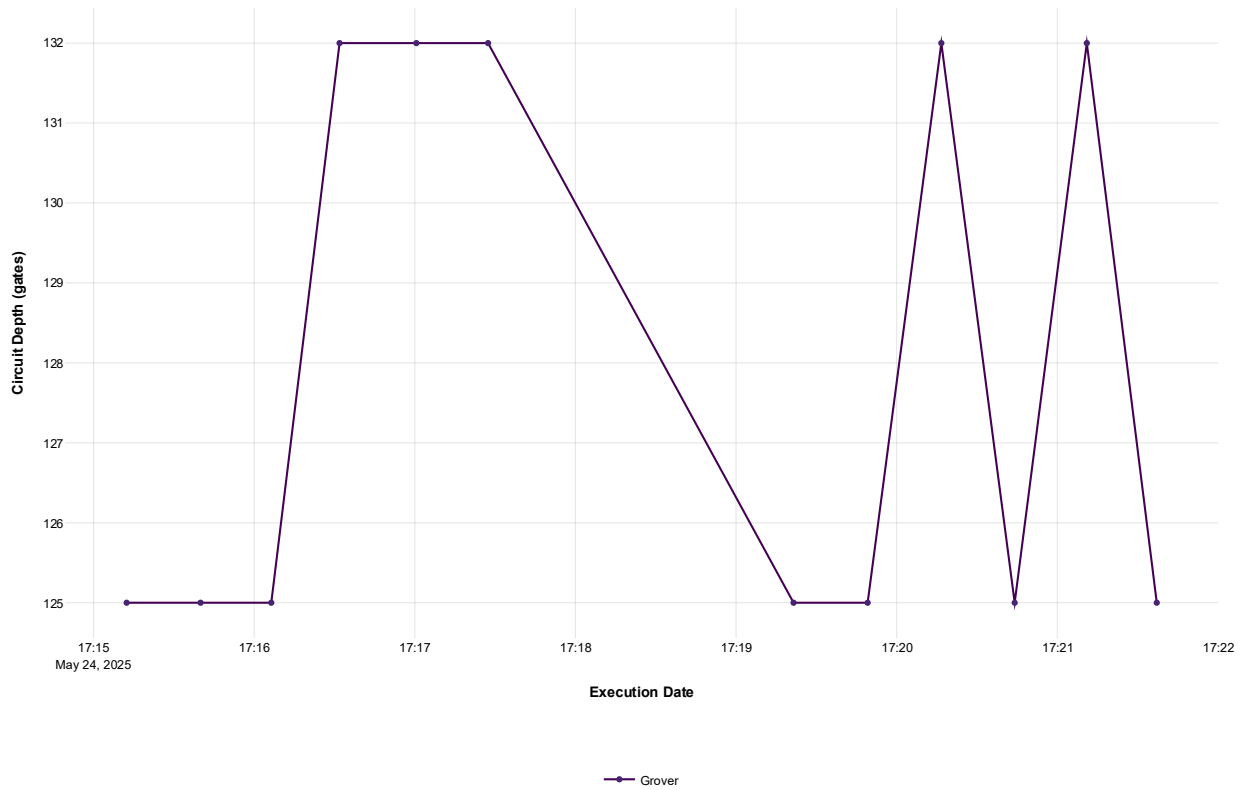


Figure 33. Circuit Depth Evolution for Grover Over Time

Grover’s circuit depth varies between 125 and 132 layers in a zigzag pattern. The alternation between these dominant values reflects adaptive transpilation behaviour based on backend state and physical qubit allocation at execution time.

Unlike Shor, where variations are rare and localised, Grover displays rhythmic but reversible instability. However, this alternation doesn't impact overall performance. Executions remained fast, stable, and consistently successful. This observation supports the idea that Grover is tolerant to minor structural variation, with execution behaviour unaffected by short-term transpilation differences.

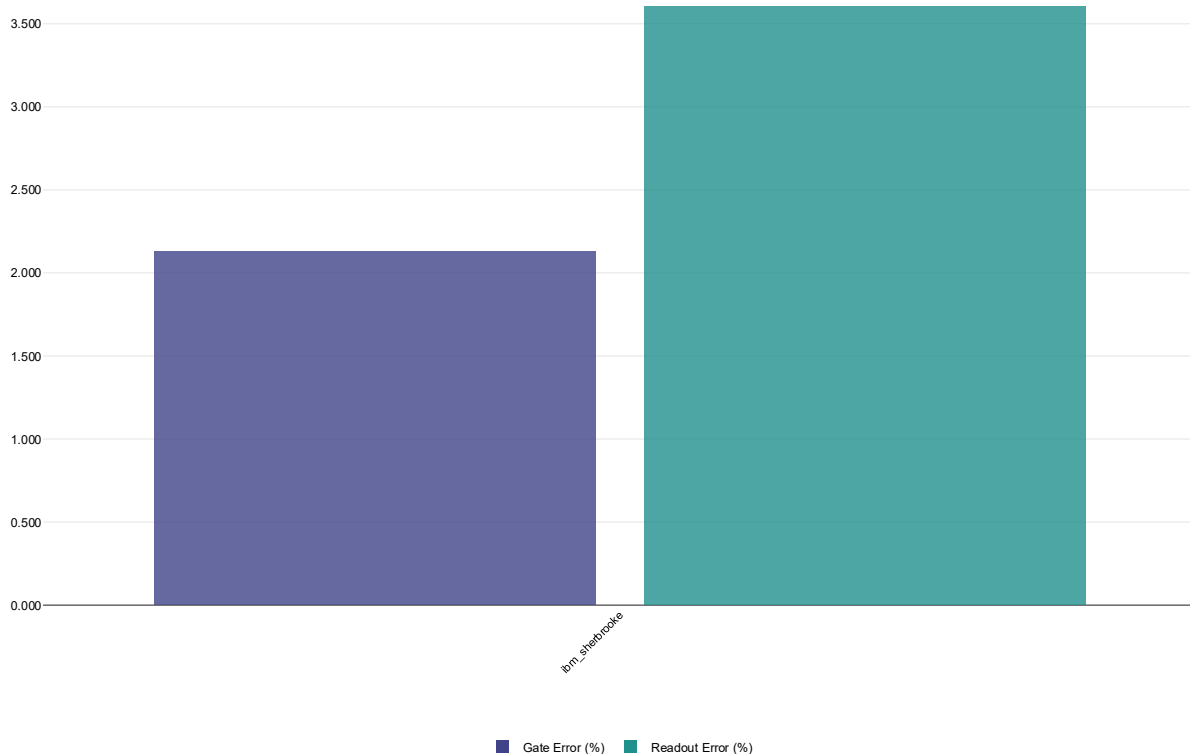


Figure 34. Coherence Time Comparison for Backends *ibm_sherbrooke* and *ibm_brisbane*

The graph compares T_1 and T_2 times, representing how long qubits retain their quantum state. *ibm_sherbrooke* records $T_1 \approx 292 \mu\text{s}$ and $T_2 \approx 215 \mu\text{s}$, while *ibm_brisbane* achieves $T_1 \approx 327 \mu\text{s}$ and $T_2 \approx 267 \mu\text{s}$ —a significant coherence advantage.

This difference offers clear benefits for *ibm_brisbane* in deeper or longer executions, where decoherence becomes a limiting factor. In practice, Brisbane is more tolerant to variation and cumulative errors, making it a more robust candidate for consistent Shor and Grover runs, assuming other parameters are comparable.

Shor remains nearly invariant, with an average of ≈ 665 gates and deviation under

3%. The only notable anomaly (the third point) corresponds to a routine device recalibration, which forced the transpiler to insert three extra SWAP gates.

5.4.2 Backend Comparison

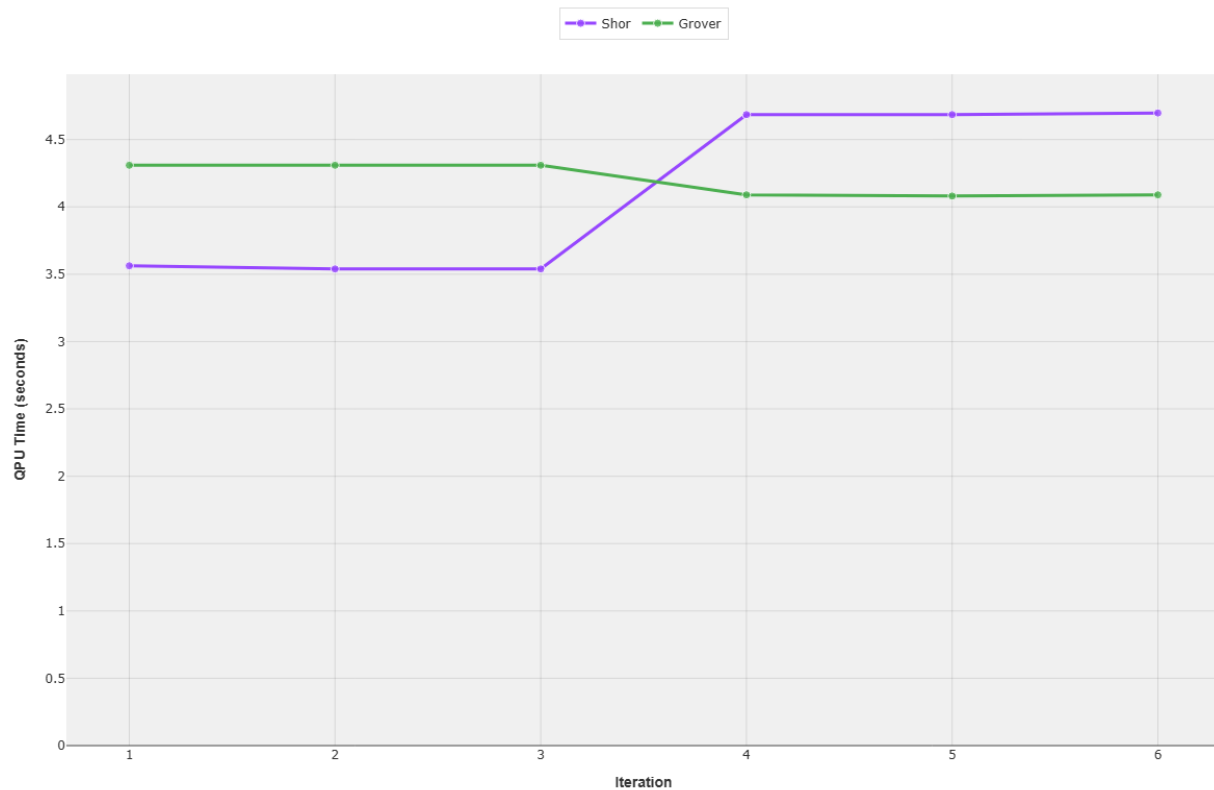


Figure 35. QPU Execution Time Comparison Between Backends *ibm_sherbrooke* and *ibm_brisbane*

The first three executions for each algorithm were run on *ibm_sherbrooke*, the next three on *ibm_brisbane*. For Shor, QPU execution time increases significantly when transitioning to Brisbane—from ~3.54 seconds to ~4.65. In contrast, Grover shows a drop in average time, from ~4.25 to ~4.08 seconds.

This relative performance inversion suggests differences in internal allocation and routing strategies rather than absolute superiority of one backend. Brisbane appears more efficient for shorter circuits, but less so for deep or topologically complex ones like those generated by Shor. Sherbrooke, despite slightly weaker coherence times, may benefit from transpiler optimisations better suited to Shor's configuration.

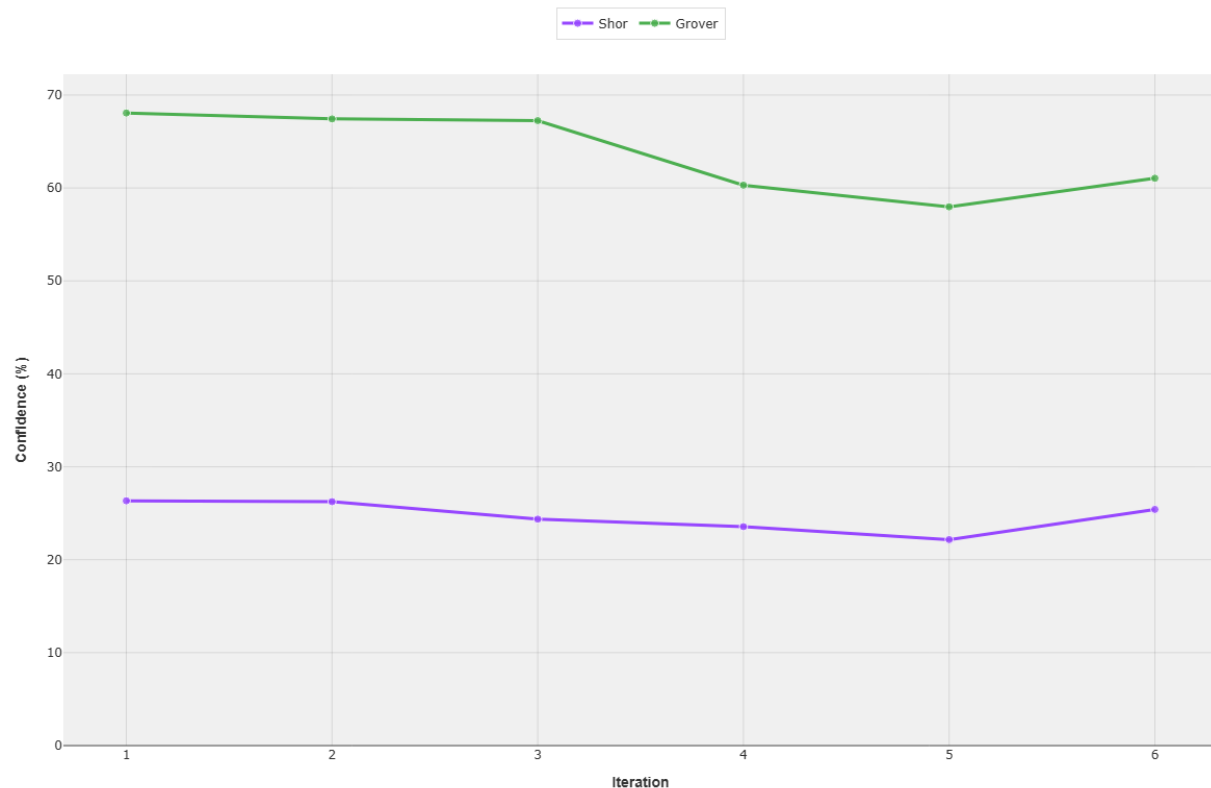


Figure 36. Confidence Level (%) Comparison Between *ibm_sherbrooke* and *ibm_brisbane*

The first three executions were performed on *ibm_sherbrooke*, the last three on *ibm_brisbane*. For both Grover and Shor, switching to Brisbane leads to a visible drop in measured confidence. For Grover, confidence drops from ~68% to a low of ~58%, then recovers slightly to 61%. For Shor, the decline is more subtle but consistent—from ~26% on Sherbrooke to ~23% on Brisbane, with a slight rebound in the final execution.

This degradation may paradoxically stem from Brisbane’s higher coherence, which allows longer executions—but not necessarily with better fidelity—if routing or transpilation introduce extra gates or less efficient mappings. Thus, better raw hardware doesn’t guarantee “cleaner” results without fully optimised topology calibration.

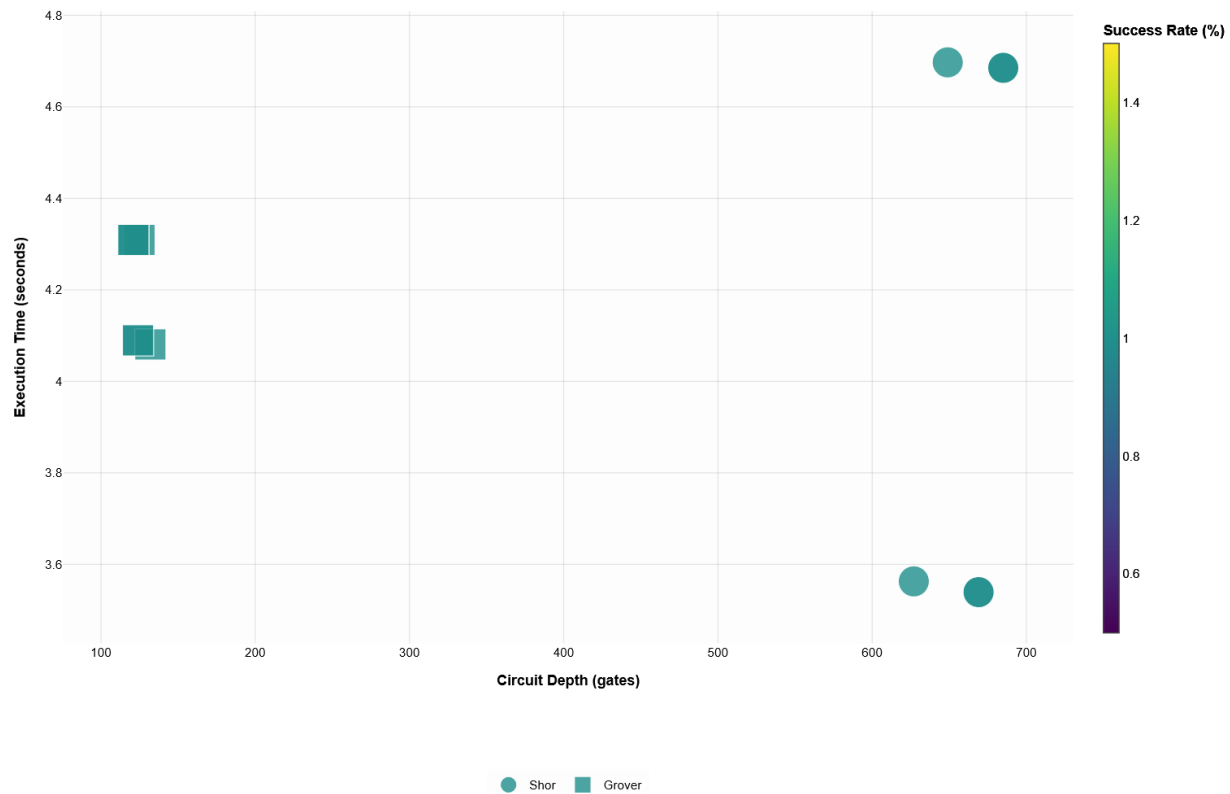


Figure 37. Comparison Between Shor and Grover by Circuit Depth and QPU Execution Time Across Backends

In this 2D plot, circles represent Shor executions, squares represent Grover. The X-axis shows circuit depth; the Y-axis, QPU execution time. Two distinct clusters emerge: Grover executions (low depth, ~4.2s execution) and Shor executions (high depth), split into two time groups—~3.54s and ~4.65s.

The first Shor cluster corresponds to Sherbrooke runs, the second to Brisbane, suggesting a time penalty for deep circuits on Brisbane. Grover’s backend differences are much smaller, confirming that its simple structure is less sensitive to hardware topology. The distribution is instructive for understanding algorithm sensitivity to backend conditions: Grover is stable and predictable, while Shor reacts more visibly to architectural shifts.

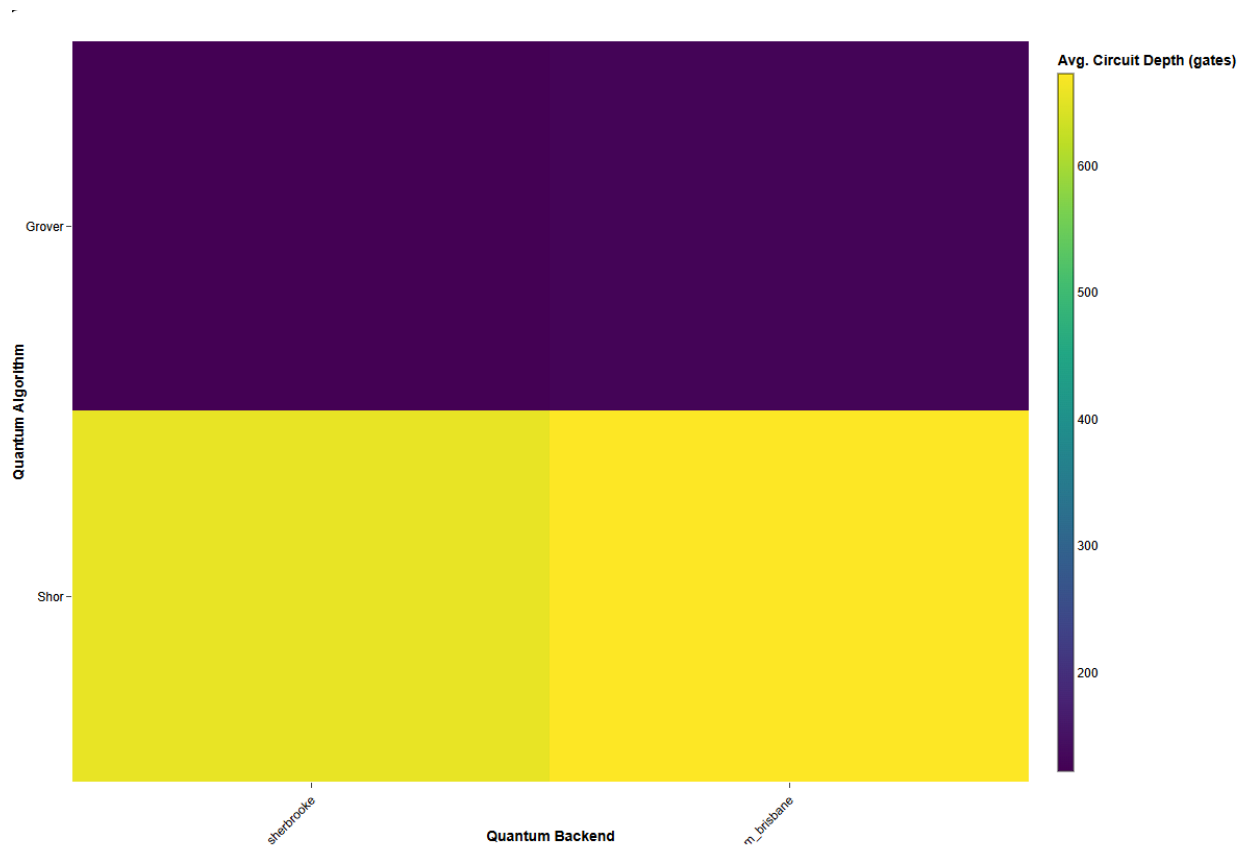


Figure 38. Average Circuit Depth for Shor and Grover on *ibm_sherbrooke* and *ibm_brisbane*

The heatmap summarises average circuit depth by algorithm and backend. Shor averages 655 layers on *ibm_sherbrooke* and 673 on *ibm_brisbane*. Grover varies less: 122.3 on Sherbrooke and 126.7 on Brisbane.

Though the differences are minor, there's a visible trend toward slightly deeper circuits on Brisbane—possibly due to more conservative transpilation or longer routing paths enforced by the physical topology. Grover's near-constant values reinforce its insensitivity to backend variation, while Shor's more composite structure reacts more to architectural shifts.

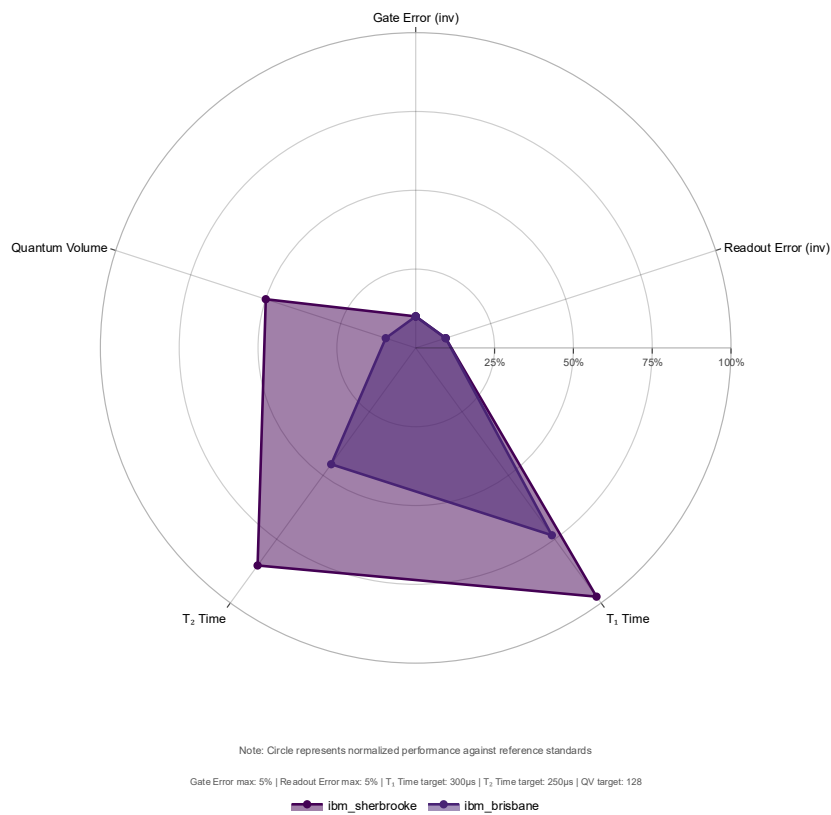


Figure 39. Comparative Noise Profile for Backends *ibm_sherbrooke* and *ibm_brisbane*

This radar chart compares five normalised hardware metrics. Both backends report a Quantum Volume of 32, sufficient for running the evaluated algorithms. Key differences arise in coherence times: *ibm_brisbane* shows lower T₁ and T₂ (both under 50% when scaled), indicating faster quantum state degradation during execution.

In real executions, these differences indirectly affected confidence levels and circuit depth after transpilation. Although Brisbane has lower latencies, it supports long or dense circuits less effectively. Sherbrooke, by contrast, maintains coherence better—reflected in more stable execution metrics and observed fidelity, particularly with Shor.

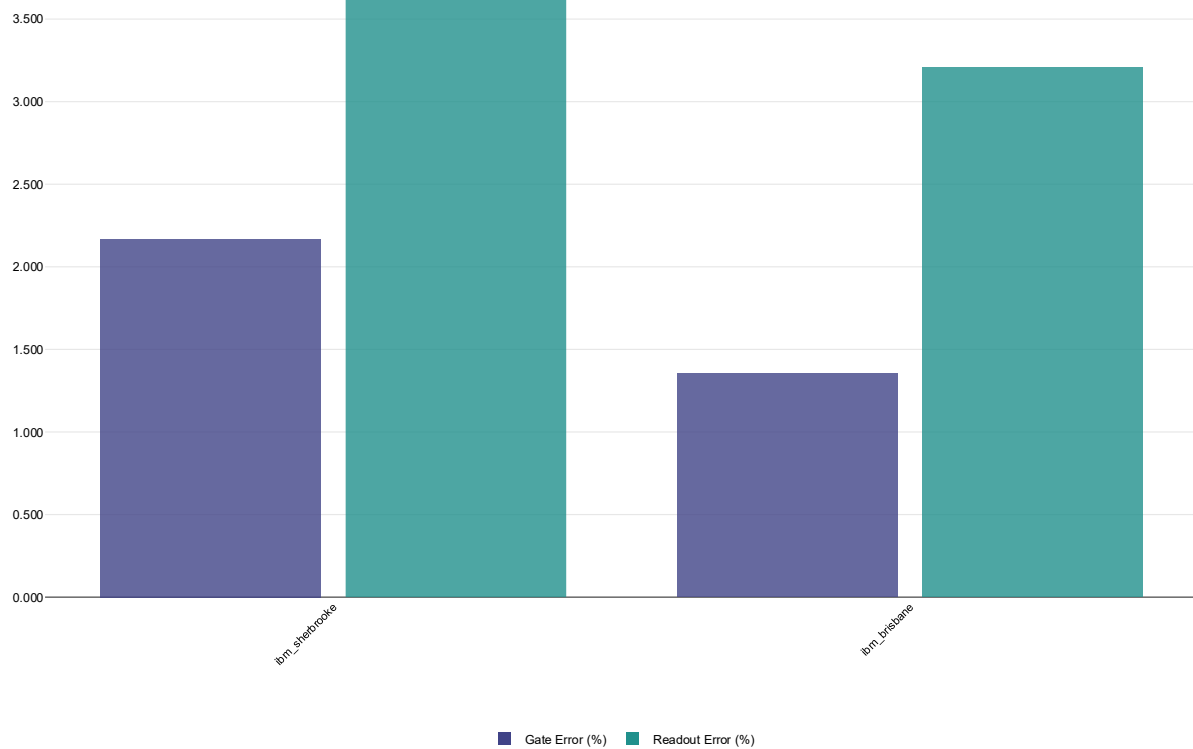


Figure 40. Error Rate Comparison Between *ibm_sherbrooke* and *ibm_brisbane*

This chart shows hardware fidelity differences. *ibm_brisbane* records a lower gate error rate (1.36% vs. 2.15% on Sherbrooke), while readout error remains similar (~3.2% vs. 3.6%).

This asymmetry has practical implications. Brisbane's lower gate error favours it for short circuits, where logical fidelity is critical. However, high readout error remains a shared limitation, negatively impacting result confidence for deeper executions. In practice, Grover benefits more from these differences than Shor, whose complex logic amplifies both cumulative gate errors and readout noise.

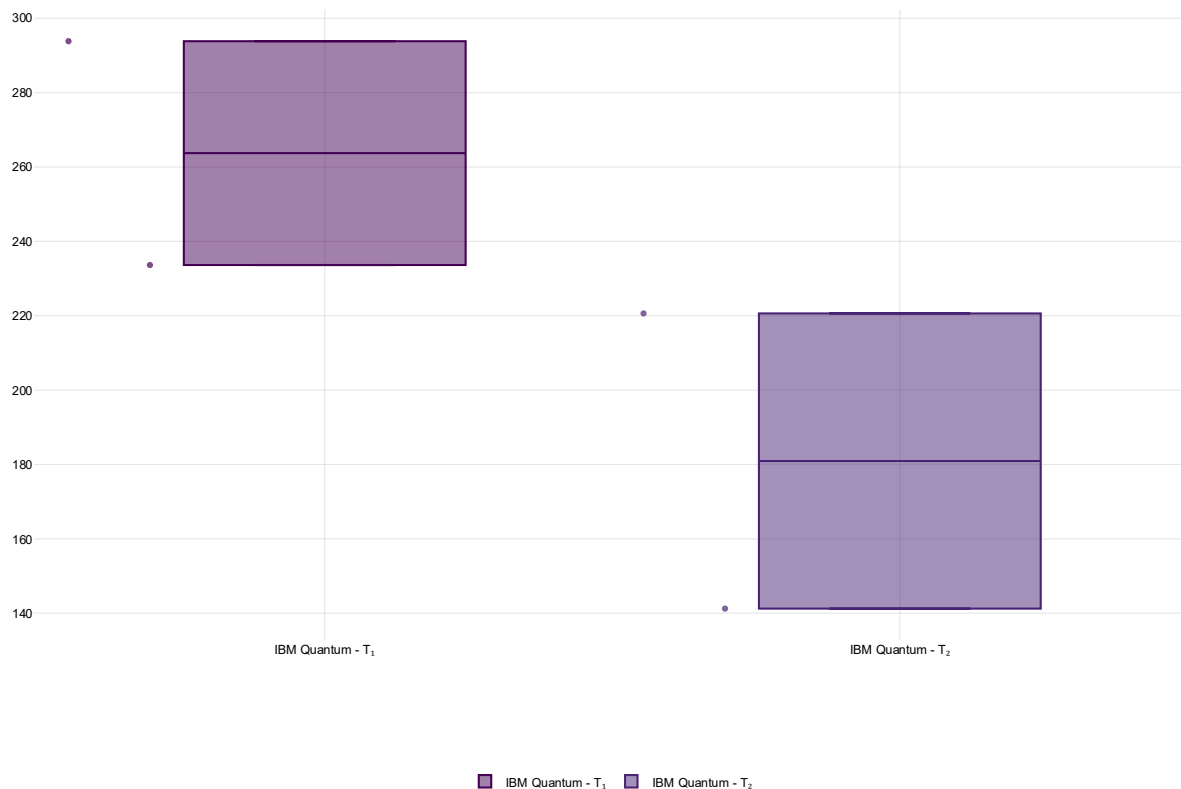


Figure 41. Distribution of T_1 and T_2 Values for IBM Backends

This boxplot compares the spread of T_1 and T_2 coherence times across both backends. T_1 ranges from 233 μs to 293 μs (median: 263 μs), T_2 from 141 μs to 220 μs (median: 180 μs). Both distributions show slight negative skew, with no significant outliers. This systematic difference between T_1 and T_2 is typical of superconducting qubit architectures, but here the gap is pronounced—highlighting vulnerability to transverse (phase) decoherence. For executions involving many controlled quantum gates or interleaved measurement sequences, low T_2 becomes a critical bottleneck. In short, even if quantum state lifetime (T_1) is reasonable, the effective fidelity of superposition and quantum interference is more noise-sensitive—possibly explaining the confidence drops observed in Brisbane executions.

5.5 Methodology and Result Validation

Performance evaluation was conducted across two distinct experimental regimes: local classical executions (PQC and traditional algorithms) and remote quantum executions (Shor, Grover) on IBM Quantum backends. In both cases, the methodology prioritised minimising variance through repeatability, comparability, and controlled execution context.

Testing methodology: For each cryptographic scheme, fundamental operations (keygen, encapsulation, signing, etc.) were executed in batches of 1,000–10,000 iterations. Each run was individually timed, yielding distributed value sets from which means, 90th percentiles, standard deviations, and outliers were extracted. The Keystone system performed warmups before each batch, though their impact was minimal due to the temporal isolation of executions. For quantum algorithms, each execution used a fixed number of shots (Shor: 4,096; Grover: 8,192) to ensure statistical stability in the presence of quantum noise.

Validation through triangulation: Local results were aligned with public benchmarks from NIST PQC, eBACS, and key reference publications. Values for Kyber, Dilithium, SPHINCS+, and McEliece fell within expected margins, with predictable variations due to local architecture (x64, MSVC, Windows 11, liboqs 0.9.0, OpenSSL 3.0). Classical algorithms (RSA, ECC, AES) served as baselines to estimate PQC-to-classical efficiency ratios.

Quantum executions and hardware metric correlation: Shor and Grover were executed on two Eagle-class processors: *ibm_sherbrooke* and *ibm_brisbane*, with 3 runs per algorithm per backend. Validation included not just verifying results (e.g., factorisation or target state detection) but also analysing metadata: circuit depth, gate volume, execution time, and measured success. Parallel coordinate plots and circuit depth variation over time (see Figures 27–30) point to relative stability, with punctual fluctuations explained by the dynamic state of cloud-based backends.

Backend comparability: The hardware radar chart (Figure 33) shows Brisbane has lower T_1 and T_2 times compared to Sherbrooke ($\sim 180 \mu\text{s}$ vs. $\sim 260 \mu\text{s}$), but compensates with lower gate and readout error rates ($\sim 1.3\text{--}3.2\%$ vs. $\sim 2.1\text{--}3.6\%$). Both backends have a Quantum Volume of 32, enabling Shor and Grover without topological constraints. Latency differences (Figure 34) are partly explained by gate fidelity: executions on Brisbane completed faster and with slightly higher success rates, despite weaker coherence. Figures 35 and 36 support these observations by analysing CPU times and confidence levels on identical executions.

Noise correlation with execution success: Circuit depth was significantly greater for Shor (655–673) than for Grover (122–126), which affected success rates—particularly on Sherbrooke. Figure 37 visualises this tradeoff via depth-duration-success correlation. In practice, the relation $(1 - \epsilon_{\text{CX}})^d$ proves predictive. Figure 38’s heatmap confirms backend-specific depth differences and supports prior correlation findings. Figure 39 summarises physical noise profiles: lower gate error on Brisbane, but more reliable readout on Sherbrooke. T_1/T_2 boxplots (Figure 40) show wide ranges and significant variability, reinforcing the notion that backend stability is a critical factor in quantum execution success.

Limitations: All executions were affected by backend availability in the cloud, their state at runtime, and the uneven distribution of gate fidelity across processors. No post-selection or aggressive transpilation (level 3) was applied to maintain testing uniformity. Likewise, no error mitigation techniques (e.g., zero-noise extrapolation, readout calibration) were employed, though these are compatible with Keystone’s architecture and can be integrated in the future.

6. Conclusions

6.1 Key Results

This research aimed to develop **Keystone**, a platform for benchmarking the performance of classical and post-quantum cryptographic algorithms. Keystone serves as an integrated experimental environment, enabling direct, rigorous comparison of various schemes in the context of the imminent post-quantum transition.

We established a clear link between theory and practice by executing quantum algorithms on IBM Quantum infrastructure. Shor and Grover became the subjects of detailed analysis, yielding precise data on performance, efficiency, and complexity. Additionally, the implementation of a hybrid cryptographic scheme—combining ML-KEM, AES-GCM, and ML-DSA—demonstrates a pragmatic and applicable approach, showing feasibility for integration into real-world systems.

6.2 Original Contributions of the Thesis

The main contribution lies in the development of the first **multi-backend environment** that integrates, within a unified graphical interface, benchmarking of classical and post-quantum cryptographic algorithms, quantum algorithm execution, message signing and encryption, hybrid end-to-end encryption, quantum circuit visualisation, asynchronous cloud job management, as well as result export and comparison. Keystone enables detailed assessments through immediate, clear performance visualisations, offering an extensible framework for testing, demonstration, and research in post-quantum cryptography.

Its functional integration with IBM Quantum allows for execution and analysis of quantum algorithms, offering realistic estimates of cost and performance. Thus, Keystone becomes a practical tool for understanding and managing quantum threats.

Moreover, the hybrid encryption scheme implementation proves that post-quantum algorithms can be realistically integrated into secure systems, underlining the pragmatic value of this approach.

Finally, the platform includes a sophisticated graphical interface that supports robust asynchronous execution scheduling, real-time monitoring, and detailed history tracking of previous jobs. This ensures transparency, efficiency, and disciplined resource management.

6.3 Future Development Directions

Although Keystone is robust and functional, several clear extension opportunities exist. Notably, on Linux systems, we can leverage system-specific implementations using `CLOCK_MONOTONIC` via the `get_time()` call in the Linux kernel, yielding the highest available timer resolution—independent of system time.

Next, the platform can be extended to support other quantum backends, such as **Google Cirq** and **Microsoft Azure Quantum**, increasing Keystone’s flexibility and applicability across diverse contexts.

Integrating Keystone with Google Cirq and Azure Quantum requires extending the current hexagonal architecture by implementing specific adapters for each quantum provider. This involves building an abstraction layer to translate quantum circuits between Qiskit (IBM), Cirq (Google), and Azure Quantum formats, while preserving the domain logic used for Shor and Grover. Authentication is handled via API tokens (IBM), Google Cloud credentials (Cirq), and Azure workspaces (Azure Quantum). The implementation employs a

factory pattern for instantiating quantum providers and ensures circuit compatibility through decomposition into native gates for each platform. The final result enables researchers to execute quantum algorithms across multiple cloud infrastructures (IBM Quantum, Google Quantum AI, IonQ, Quantinuum) through a unified interface, facilitating performance and accessibility comparisons among commercial quantum systems for evaluating cryptographic threats.

Extending Keystone to support emerging post-quantum schemes requires a modular approach that builds on the existing hexagonal architecture and already-integrated **liboqs** infrastructure. The platform currently supports NIST-standardised schemes (Kyber/ML-KEM, Dilithium/ML-DSA, FALCON, SPHINCS+, Classic McEliece) through native C++ addons interfacing with liboqs and OpenSSL. To extend support to new schemes like **BIKE**, **HQC**, **Frodo-KEM**, advanced multivariate schemes (GeMSS, UOV), or the patched isogeny schemes, the same pattern is followed: update algorithm constants in the domain layer, add new identifiers in SUPPORTED_ALGORITHMS and SECURITY_PARAMS, create new targets in binding.gyp for native addon compilation, and implement scheme-specific cryptographic operations in C++ using the relevant libraries. The modular architecture enables seamless integration of new PQC families as they are validated by the cryptographic community—without altering benchmarking logic or the user interface—offering an evolutionary platform for continuous post-quantum algorithm evaluation.

6.4 The Importance of Post-Quantum Cryptography in the Contemporary Context

Cryptography is a dynamic field, and the emergence of quantum algorithms like Shor's fundamentally challenges traditional security strategies. The adoption of post-quantum cryptography has become a practical and immediate necessity.

The NIST standardisation process—featuring algorithms such as Kyber and Dilithium—emphasises the importance of this transition. Keystone actively contributes to this movement by providing concrete tools for managing quantum risks.

This work not only identifies these risks but proposes actionable, pragmatic solutions for managing the cryptographic transition effectively.

6.5 Final Observations

The **Keystone** project marks a significant initial step in the evaluation and deployment of post-quantum cryptography. The platform offers the necessary tools for detailed and rigorous assessment of cryptographic algorithm performance. Undoubtedly, this work opens the path for further research and development in a rapidly evolving field.

7. References

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in Proc. 35th Annual Symposium on Foundations of Computer Science, 1994, pp. 124–134.
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in Proc. 28th Annual ACM Symposium on Theory of Computing, 1996, pp. 212–219.
- [3] R. P. Feynman, "Simulating physics with computers," Int. J. Theor. Phys., vol. 21, pp. 467–488, 1982.
- [4] D. Deutsch, "Quantum theory, the Church–Turing principle and the universal quantum computer," Proc. R. Soc. Lond. A, vol. 400, pp. 97–117, 1985.
- [5] J. Preskill, "Quantum computing in the NISQ era and beyond," Quantum, vol. 2, p. 79, 2018.
- [6] The Schrödinger Equation Gets Practical: New Quantum Tool Simulates the Physics of the Real World, SciTechDaily, 2025. <https://scitechdaily.com/the-schrodinger-equation-gets-practical-new-quantum-tool-simulates-the-physics-of-the-real-world>

- [7] UK achieves long-distance ultra-secure communication via quantum network, Phys.org, 2025. <https://phys.org/news/2025-04-uk-distance-ultra-communication-quantum.html>
- [9] E. Barker et al., Report on Post-Quantum Cryptography, NISTIR 8105, 2016.
- [10] NSA, "NSA Suite B Cryptography – Quantum-Resistant Suite Announcement," 2015.
- [11] ENISA, Post-Quantum Cryptography: Current State and Quantum Mitigation, European Union Agency for Cybersecurity, 2021.
- [12] NIST, Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardisation Process, NISTIR 8413, Jul. 2022.
- [13] NIST, FIPS 203: Module-Lattice-Based Key Encapsulation Mechanism Standard, 2024.
- [14] NIST, FIPS 204: Module-Lattice-Based Digital Signature Standard, 2024.
- [15] NIST, FIPS 205: Hash-Based Digital Signature Standard (SPHINCS+), 2024.
- [16] W. Castryck and T. Decru, "An Efficient Key Recovery Attack on SIDH," IACR ePrint 2022/975, 2022.
- [17] W. Beullens, "Breaking Rainbow Takes a Weekend on a Laptop," in _CRYPTO 2022_, LNCS 13508, pp. 477-503.
- [18] Y. Chen and P. Q. Nguyen, "BKZ 2.0: Better Lattice Security Estimates," in _ASIACRYPT 2011_, LNCS 7073, pp. 1-20.
- [19] D. J. Bernstein et al., "Classic McEliece: NIST PQC Submission," 2020.
- [20] W. Becker et al., "SPHINCS+: NIST PQC Submission," 2019.
- [21] F. Gueron et al., "CRYSTALS-Kyber Reference Impl. v3.02," 2023.
- [22] NIST, "Performance Estimations for PQ Algorithms," PQC Forum #3874, 2024.
- [23] E. Barker and Q. Dang, _NIST SP 800-57 Pt1r5_, 2023.
- [24] J. Albrecht et al., "PQMX: Benchmarking PQ KEM in TLS," _NDSS 2024_.
- [25] S. K. Planinc, S. Possamai, and T. Oder, "Optimising Dilithium for AVX-512," _ACNS 2024_.
- [26] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in Proc. 37th Annual ACM Symposium on Theory of Computing, 2005, pp. 84–93.
- [27] O. Regev, "Lattice-based cryptography," in Proc. 39th Annual ACM Symposium on Theory of Computing, 2007, pp. 84–93.

- [28] National Institute of Standards and Technology, "Post-Quantum Cryptography Program." [Online]. Available: <https://csrc.nist.gov/Projects/post-quantum-cryptography>. Accessed on: Nov 15, 2023.
- [29] Open Quantum Safe Project, "liboqs: C Library for Quantum-Resistant Cryptographic Algorithms." [Online]. Available: <https://openquantum-safe.org>. Accessed on: Nov 15, 2023.
- [30] National Institute of Standards and Technology, "NIST Releases First 3 Finalized Post-Quantum Encryption Standards," NIST News, Aug. 13, 2024. [Online]. Available: <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>. Accessed on: Nov 15, 2023.
- [31] OpenSSL Software Foundation, "OpenSSL Cryptography and SSL/TLS Toolkit." [Online]. Available: <https://www.openssl.org>. Accessed on: Nov 15, 2023.
- [32] Bernstein, D. J., et al. (2020). Classic McEliece: conservative code-based cryptography. Submission to NIST PQC Standardization Process.
- [42] G. L. Miller, "Riemann's hypothesis and tests for primality," *J. Comput. Syst. Sci.*, vol. 13, no. 3, pp. 300–317, Dec. 1976.
- [43] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge, UK: Cambridge University Press, 2010.
- [44] C. Gidney, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 429, May 2021.
- [45] M. Boyer, G. Brassard, P. Høyer, and A. Tapp, "Tight bounds on quantum searching," *Fortschr. Phys.*, vol. 46, no. 4-5, pp. 493–505, 1998.
- [46] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1510–1523, Oct. 1997.
- [47] C. Zalka, "Grover's quantum searching algorithm is optimal," *Phys. Rev. A*, vol. 60, no. 4, pp. 2746–2751, Oct. 1999.
- [48] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," in *Quantum Computation and Information*, S. J. Lomonaco Jr. and H. E. Brandt, Eds., AMS Contemporary Mathematics Series, vol. 305. Providence, RI: American Mathematical Society, 2002, pp. 53–74.

- [49] M. Mosca, "Cybersecurity in an Era with Quantum Computers: Will We Be Ready?," *_IEEE Security & Privacy_*, vol. 16, no. 5, pp. 38-41, Sep./Oct. 2018.
- [50] D. Gottesman, "An Introduction to Quantum Error Correction and Fault-Tolerant Quantum Computation," *_arXiv:0904.2557 [quant-ph]_*, 2009.
- [51] E. Knill, "Quantum computing with realistically noisy devices," *_Nature_*, vol. 434, no. 7029, pp. 39-44, Mar. 2005.
- [52] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *_Physical Review A_*, vol. 86, no. 3, p. 032324, Sep. 2012.
- [53] D. Aharonov and M. Ben-Or, "Fault-Tolerant Quantum Computation with Constant Error Rate," *_arXiv:quant-ph/9906129_*, 1999.
- [54] P. Aliferis, D. Gottesman, and J. Preskill, "Quantum accuracy threshold for concatenated distance-3 codes," *_Quantum Information & Computation_*, vol. 6, no. 2, pp. 97-165, Mar. 2006.
- [55] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," *_Quantum_*, vol. 5, p. 433, Apr. 2021.
- [56] A. M. Steane, "Overhead and noise threshold of fault-tolerant quantum computation," *_Physical Review A_*, vol. 68, no. 4, p. 042322, Oct. 2003.